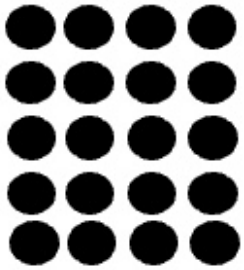
NHL STENDEN
university of applied sciences

COMPUTER SCIENCE

FRANCISCO PIMENTEL



DATE:

SEPTEMBER 12, 2025

STUDENT N.º 5708915

Francisco.ponte.pimentel@nhlstenden.student.com

Table of Contents

Image Index 5
Introduction 7
Introduce Arduino 8
How to install? 9
After downloaded 10
Materials 11
LED 11
Button 13
74HC595 Shift Register 15
7-Segment LED Displays 17
7-Segment 1-Digit LED Counter 18
7-Segment 4-Digit multiplexed LED Display 18
Liquid Crystal Displays (LCD) 19
LM35 20
Water Level Sensor 22
DHT11 23
Week 1 25
Scratch 25
Hello, It's me 27
Hello, It's me - Result: 27
Micro Bit 28
Self-Reflection 1 29
Week 2 30
Mario Less 30
Mario Less - Result: 30
Mario More 31
Mario More - Result: 31
Cash 32
Cash - Result: 32
Mario UNO 33
Mario UNO - Result: 33
Self-Reflection 2 34
Week 3 34
Set up guide Arduino in VSCode 34
Scrabble 40
Scrabble - Result: 40
Readability 41
Readability - Result: 41
Caesar 42
Caesar - Result: 42
Morse Code 43
Morse Code - Result: 43
Self-Reflection 3 45

Week 4 ..... *46*

Sort **46**

Plurality **47**

Plurality - Result: 47

Tideman **48**

Tideman - Result: 48

Traffic Lights **49**

Traffic Lights - Result: 49

Self-Reflection 4 ..... **51**

Week 5 ..... *52*

Filter – Less **52**

Filter Less - Result: 52

Recover **55**

Recover - Result before: 55

Result after: 56

Displaying Uptime **57**

Displaying Uptime - Result: 57

Self-Reflection 5 ..... **60**

Week 6 ..... *60*

Git Together **60**

Inheritance **61**

Inheritance - Result: 61

Jumbo Clock **62**

Jumbo Clock - Result: 62

Self-Reflection 6 ..... **64**

Week 7 ..... *65*

Speller **65**

Speller - Result: 65

Weather Station **66**

Weather Station - Result: 66

Self-Reflection 7 ..... **69**

Codes *70*

Hello, It's Me – Code **70**

Mario Less - Code **70**

Mario More - Code **71**

Cash - Code **72**

Mario UNO - Code **73**

Scrabble - Code **75**

Readability - Code **76**

Caesar – Code **77**

Morse Code - Code **79**

Plurality - Code **82**

Tideman – Code **84**

Traffic Lights - Code **88**
Filter Less - Code **91**
Recover - Code **93**
Displaying Uptime - Code **95**
Git Together - Code **98**
Inheritance - Code **100**
Jumbo Clock - Code **103**
Speller – Code **106**
Weather Station - Code **108**
Bibliography ***112***
Check, check, triple check ***113***

Image Index

Figure 1 - Fritzing diagram of the circuit....11
Figure 2 - Fritzing diagram of the circuit....13
Figure 3 - The 74HC595 IC....15
Figure 4 - Fritzing diagram....16
Figure 5 - Displaying numbers 0-9....17
Figure 6 - Number to be displayed and port data....17
Figure 7 - 7 Segment 1-digit....18
Figure 8 - 7 Segment 4-digit....18
Figure 9 - Standard I2C-based LCD....19
Figure 10 - Circuit View | LCD....19
Figure 11 - LM35 temperature sensor chip....20
Figure 12 - Circuit View | LM35....21
Figure 13 - Water Level Sensor....22
Figure 14 - DHT11 module....23
Figure 15 - Game_1....25
Figure 16 - Game_2....26
Figure 17 - Hello, It's me | Result....27
Figure 18 - Micro:bit....28
Figure 19 - Mario Less | Result....30
Figure 20 - Mario More | Result....31
Figure 21 - Cash | Result....32
Figure 22 - Mario UNO | Result....33
Figure 23 - Scrabble | Result....40
Figure 24 - Readability | Result....41
Figure 25 - Caesar | Result....42
Figure 26 - Morse Code | Circuit View....43
Figure 27 - Morse Code | Schematic View....44
Figure 28 - Morse Code | Real View....44
Figure 29 - Plurality | Result....47
Figure 30 - Tideman | Result....48
Figure 31 - Traffic Lights – Circuit View....49
Figure 32 - Traffic Lights – Traffic Lights | Schematic View1....50
Figure 33 - Traffic Lights | Schematic View2....50
Figure 34 - Traffic Lights | Real View....51
Figure 35 - Filter Less | GrayScale....52
Figure 36 - Filter Less | Sepia....53
Figure 37 - Filter Less | Reflect....53
Figure 38 - Filter Less | Blur....54
Figure 39 - Recover | Result Before....55
Figure 40 - Recover | Result After....56
Figure 41 - Displaying Uptime | Circuit View....57
Figure 42 - Displaying Uptime | Schematic View1....58
Figure 43 - Displaying Uptime | Schematic View2....58
Figure 44 - Displaying Uptime | Schematic View3....59
Figure 45 - Displaying Uptime | Real View....59
Figure 46 - Inheritance | Result....61
Figure 47 - Jumbo Clock | Circuit View....62
Figure 48 - Jumbo Clock | Schematic View....62

Figure 49 - Jumbo Clock | Real View....63
Figure 50 - Speller | Result....65
Figure 51 - Weather Station | Real View | Main....66
Figure 52 - Weather Station | Real View | MaxT....66
Figure 53 - Weather Station | Real View | Water....67
Figure 54 - Weather Station | Schematic View....68

Introduction

My name is Francisco Ponte Pimentel, I'm 18 years old, and I'm currently pursuing a degree in Computer Science at NHL Stenden University. This portfolio showcases projects from two of the core areas of my academic work: Computer Science, developed through the Harvard course CS50x 2025, whose official documentation can be consulted at https://manual.cs50.io and Embedded Systems, where I relied heavily on the "Mastering Arduino" manual throughout my learning process.
I have always been driven by curiosity, not only to understand how things work, but why they are designed that way. This mindset naturally pushes me to go beyond the minimum requirements, explore new tools, and continuously refine my approach to problem-solving.
I value being an attentive listener and fast learner, as these qualities help me efficiently absorb information, adapt to new concepts, and collaborate effectively with others.
Whether you are here to explore my CS50 problem sets, embedded systems prototypes, or simply to understand how I think and build things, this portfolio is structured so that each project includes the necessary context to follow it, even without external materials.

Introduce Arduino

Arduino is an open-source microcontroller development system that combines hardware, an integrated development environment (IDE), and a vast collection of software libraries. This combination makes Arduino accessible to both beginners and professionals, allowing for the creation of electronic projects quickly and relatively easily. Thanks to its active community of programmers, engineers, and enthusiasts, there are numerous examples, tutorials, and libraries available, facilitating the implementation of various types of interfaces and sensors.

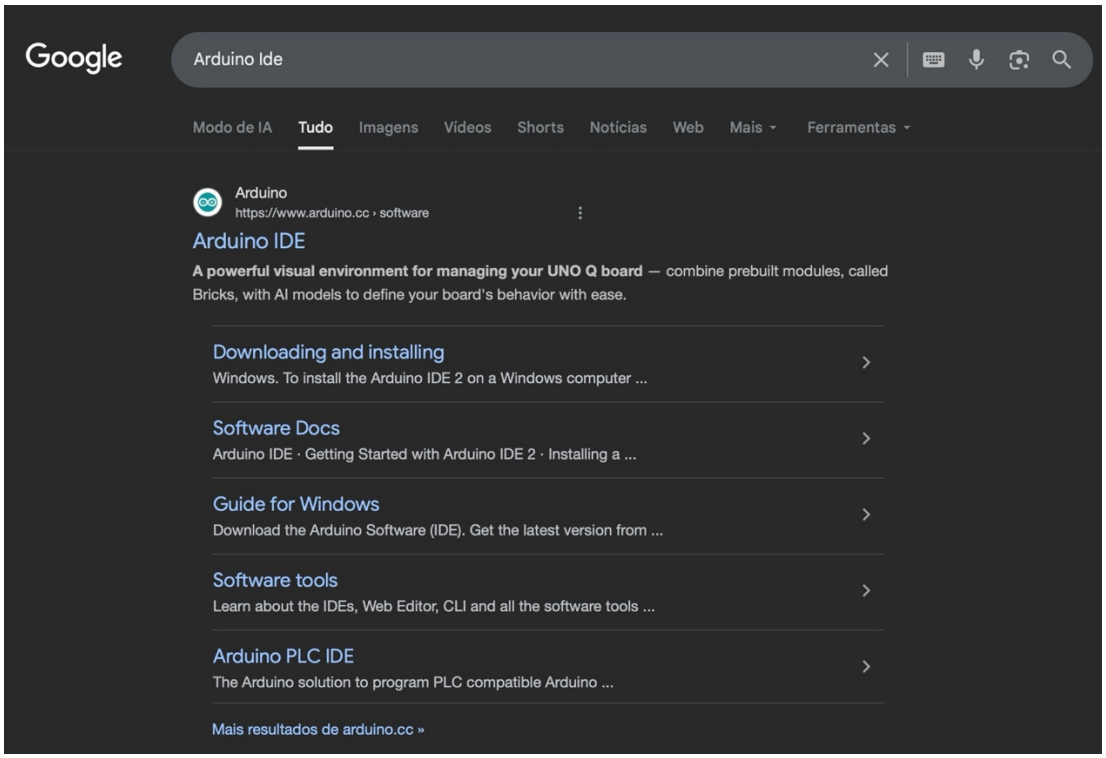
The Arduino Uno R3 is perhaps the most popular model in the Arduino family. Based on the 8-bit ATmega328P processor, it's ideal for beginner to intermediate level projects that don't require a lot of processing power or memory. The Uno R3 has become famous for its simple IDE and the ease of developing projects in a short time.

More recently, the Arduino Uno R4 was released, maintaining compatibility with the R3 but offering significantly improved specifications. The R4 is based on the 32-bit 48 MHz Cortex-M4 processor, with more SRAM and flash memory, a higher precision ADC, and an integrated DAC. It also adds support for the CAN bus, expanding the range of possible applications. Among the available variants, the Arduino R4 Wi-Fi stands out for its integrated wireless connectivity, enabling more direct and flexible Internet-connected projects.

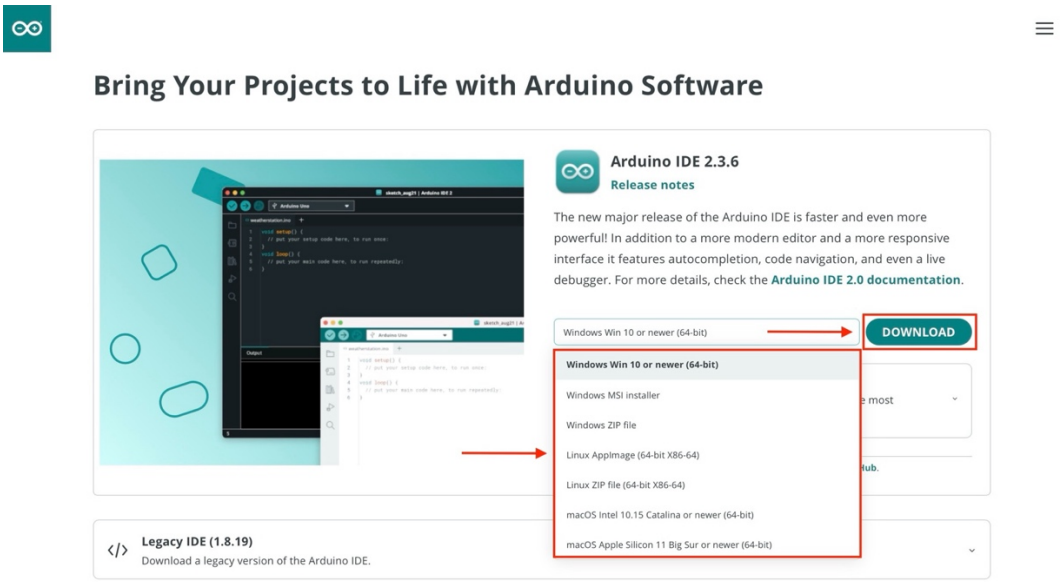
In short, the Arduino R4 Wi-Fi combines the ease of use of the traditional Arduino with greater processing power and connectivity capabilities, making it ideal for more advanced projects that require wireless communication and superior performance.

How to install?

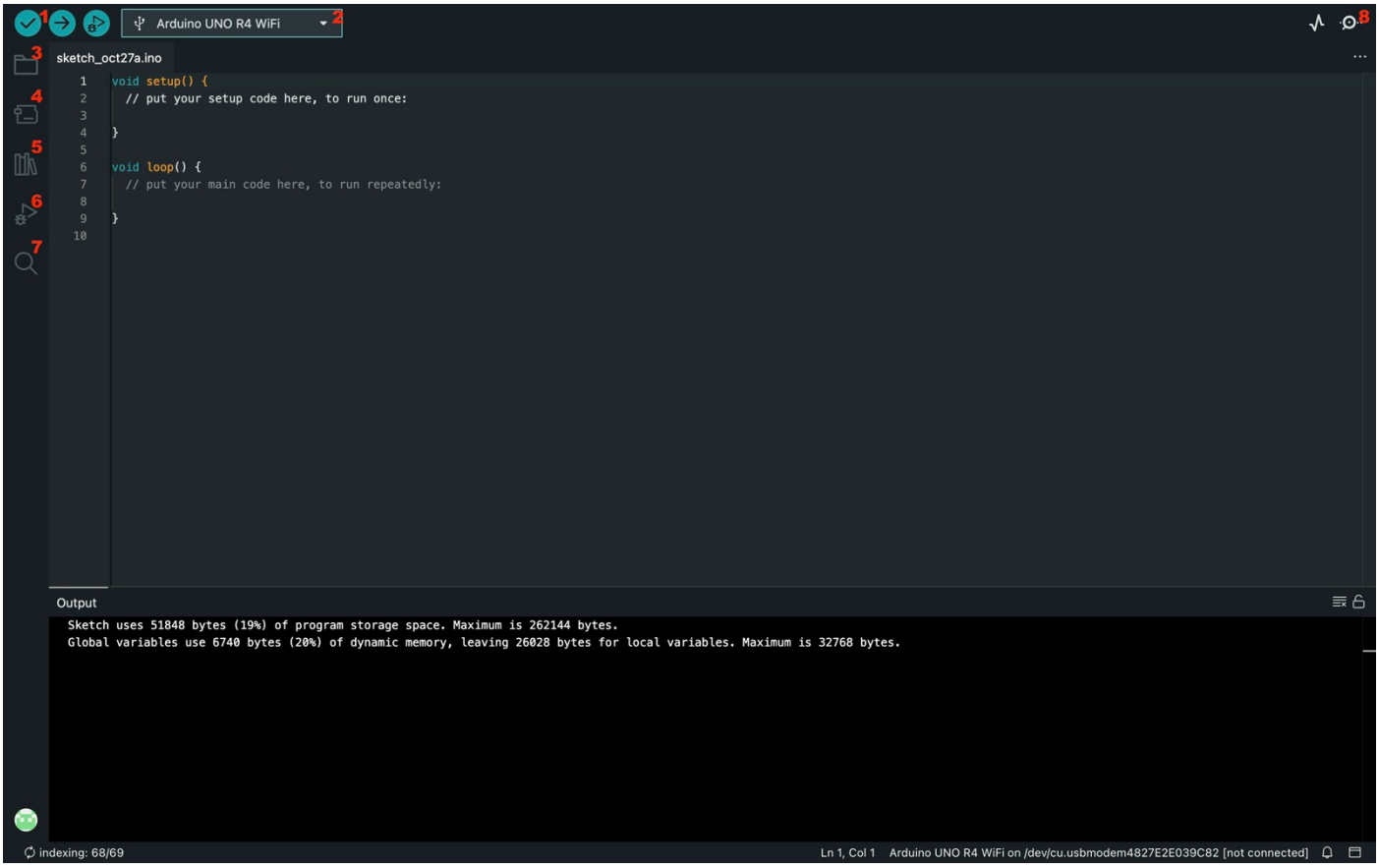
1. Search for Arduino IDE



2. Select your type and download it



After downloaded



Number	Caption	Description
1	Verify / Upload	Compile and upload your code to your Arduino Board
2	Select Board & Port	Detected Arduino boards automatically show up here, along with the port number
3	Sketchbook	Here you can find all your sketches locally stored on your computer
4	Board Manager	Browse through Arduino & third party packages that can be installed
5	Library Manager	Browse through thousands of Arduino libraries
6	Debugger	Teste and debug programs in real time
7	Search	Search for keywords in your code
8	Open Serial Monitor	Opens the Serial Monitor tool, as a new tab in the console

Materials

Here I will present the materials I used throughout the projects, as well as a brief explanation of how to use them.

LED

LEDs can be connected to the development board as either in current-sourcing mode or current-sinking mode. The following example shows a simple way to use LED with Arduino.

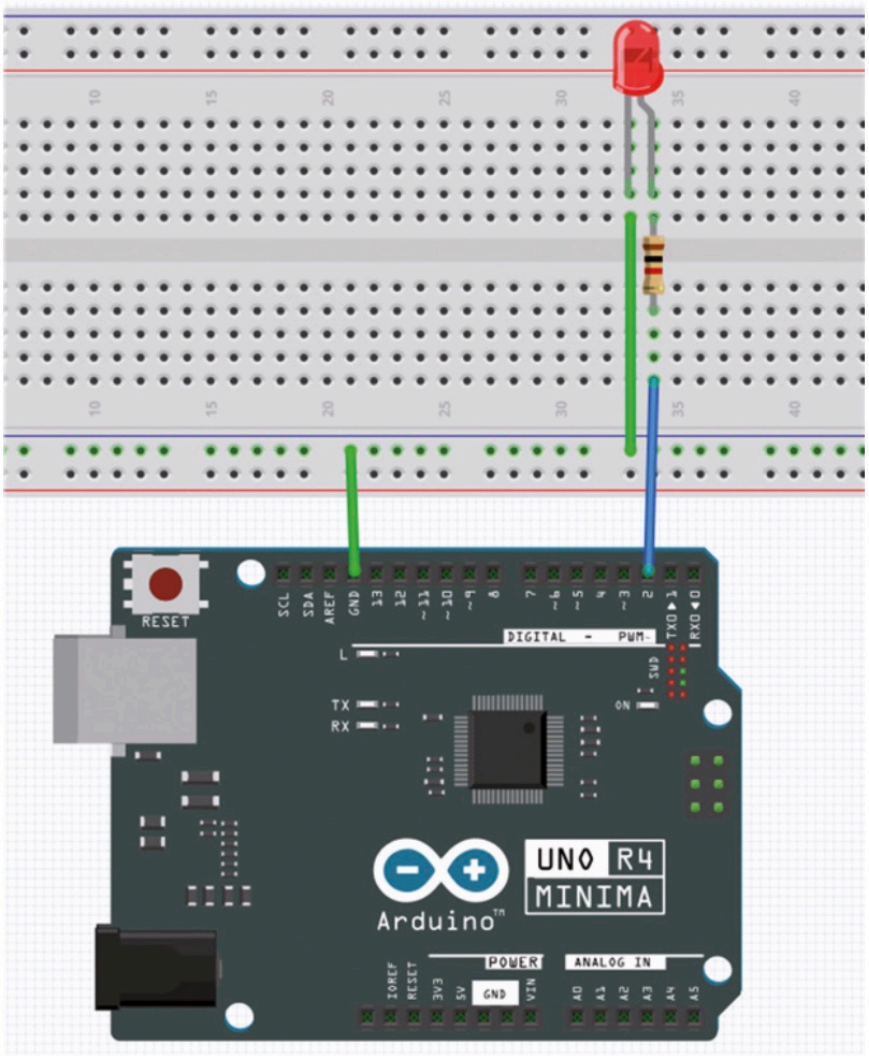


Figure 1 - Fritzing diagram of the circuit

```
//-------------------------------------------------------------------
// BLINKING AN EXTERNAL LED
// ========================
//
// This program blinks the LED connected to port 2 of the board
//
// Author: Dogan Ibrahim
// File : LEDext
// Date : June, 2023
//-------------------------------------------------------------------
#define ON HIGH                          // Define ON
#define OFF LOW                          // Define OFF
int LED = 2;                             // LED at port 2
void setup()
{
  pinMode(LED, OUTPUT);                  // Configure LED as output
}
void loop()
{
  digitalWrite(LED, ON);                 // LED ON
  delay(1000);                           // 1 second delay
  digitalWrite(LED, OFF);                // LED OFF
  delay(1000);                           // 1 second delay
}
```

Button

Buttons can be connected to the development board to provide digital input to the system. When pressed, they change their state between HIGH and LOW, allowing the Arduino to detect user interactions or trigger specific actions in the program. They are commonly used to start, stop, or control different parts of a project. The following example shows a simple way to use a button with Arduino.

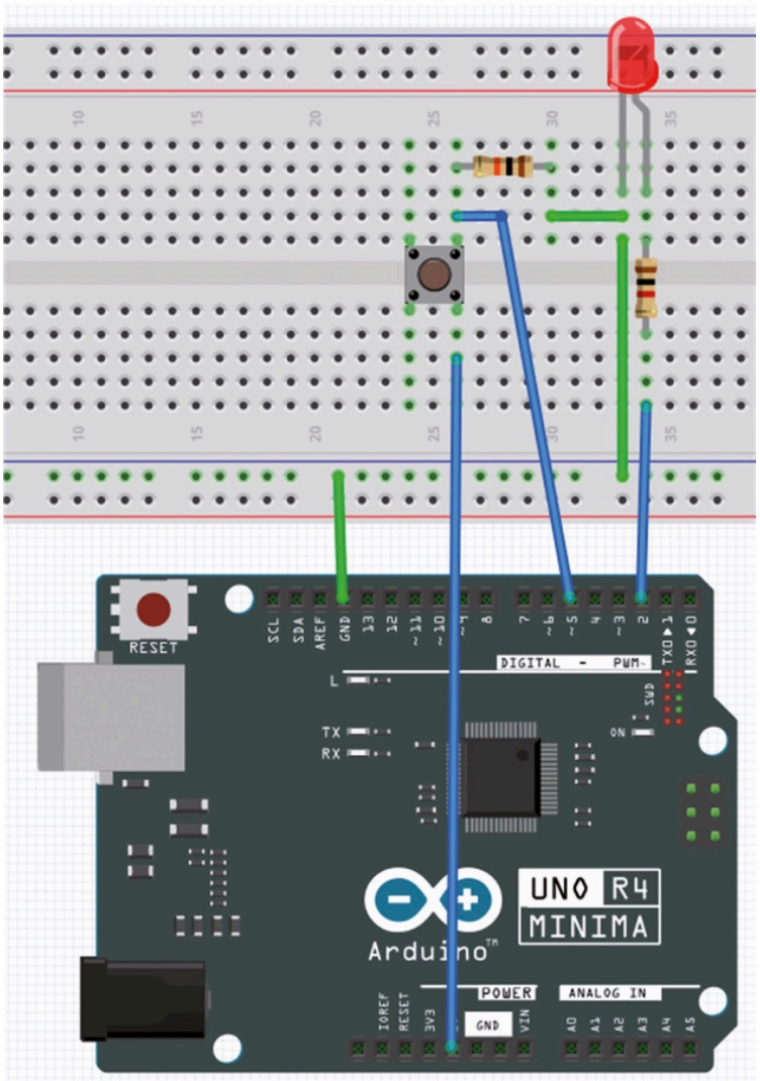


Figure 2 - Fritzing diagram of the circuit

```
//-------------------------------------------------------------------
// BUTTON AND LED
// ==============
//
// In this program an LED and a button are connected. Pressing the button
// turns ON the LED
//
// Author: Dogan Ibrahim
// File : Button
// Date : June, 2023
//-------------------------------------------------------------------
#define ON HIGH                                 // Define ON
#define OFF LOW                                 // Define OFF
int LED = 2;                                    // LED at port 2
int Button = 5;                                 // Button at port 5
void setup()
{
  pinMode(LED, OUTPUT);                         // Configure LED as output
  digitalWrite(LED, OFF);                       // LED OFF at beginning
  pinMode(Button, INPUT);                       // COnfigure Button as input
}
void loop()
{
  if (digitalRead(Button) == 1) {               // If Button is pressed
    digitalWrite(LED, ON);                      // LED ON
  } else {                                      // Otherwise
    digitalWrite(LED, OFF);                     // LED OFF
  }
}
```

74HC595 Shift Register

The 74HC595 Shift Register is a serial-in, parallel-out integrated circuit used to increase the number of digital outputs available on the Arduino. It allows multiple components, such as LEDs, to be controlled using only three pins from the board: one for data, one for the clock, and one for the latch signal. Data is sent serially into the register and then converted into parallel output after eight clock pulses.

This makes it highly efficient for projects requiring many outputs while conserving Arduino pins. Multiple 74HC595 chips can also be daisy-chained to control even more devices.

Pin	Name	Description
15 and 1 to 7	Q0 to Q7	parallel output
9	Q7S	used to daisy chain other chips
14	DS	serial data input
13	OE	Output enable (LOW to enable)
12	STCP	Latch (Data sent to output when HIGH)
11	SHCP	shift register clock (on the rising edge)
10	MR	Reset (LOW to clear)
8	GND	power supply ground
16	VCC	+5 V

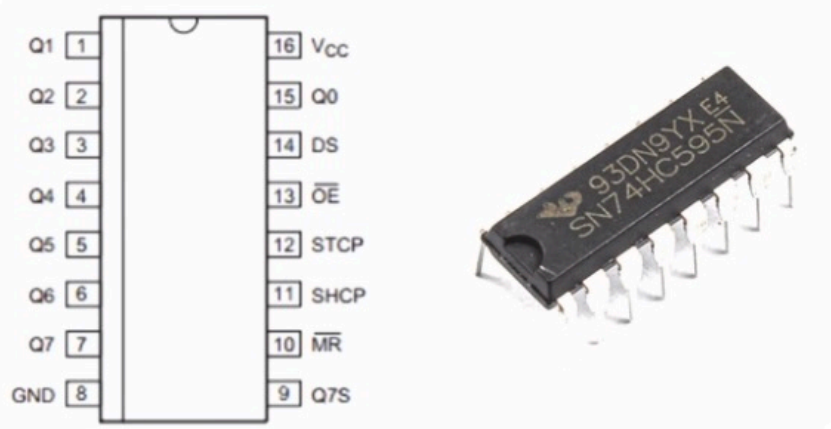


Figure 3 - The 74HC595 IC

The following example shows a simple way to use a shift register with Arduino.

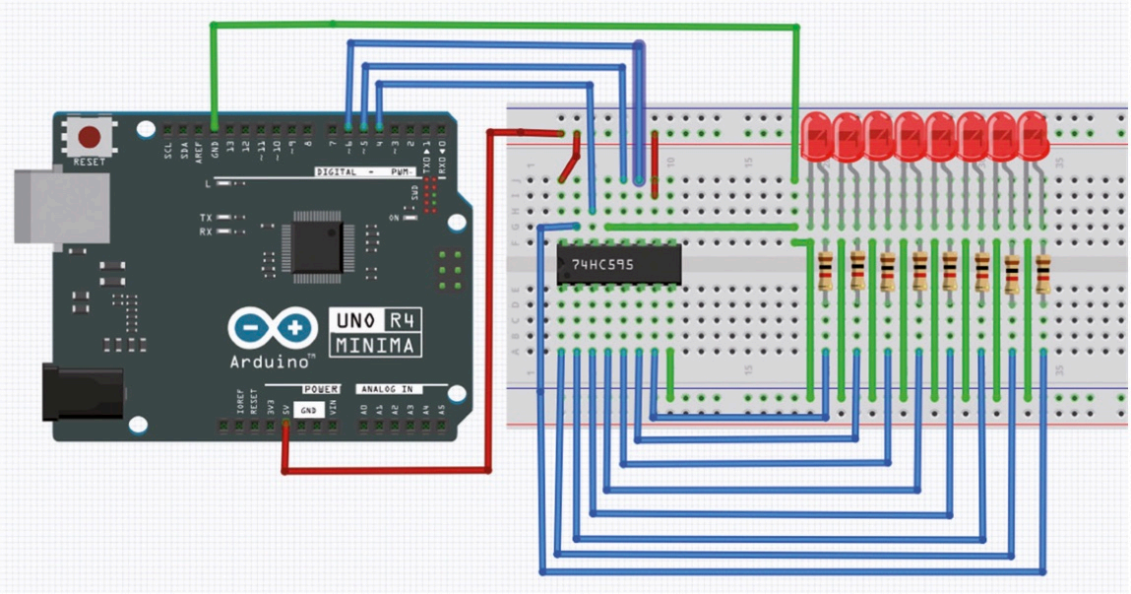


Figure 4 - Fritzing diagram

```
//-------------------------------------------------------------------------
// 74HC595 SHIFT REGISTER LED BINARY COUNTING
// ========================================
//
// In this project the 74HC595 shift register chip is used and 8 LEDs
// are connected to the chip.The program counts up in binary from 0 to
// 255 with 500 ms delay between each output
//
// Author: Dogan Ibrahim
// File : ShiftReg
// Date : June, 2023
//-------------------------------------------------------------------------
int STCP = 5;                                   // Latch pin
int SHCP = 6;                                   // Clock pin
int DS = 4;                                     // Data pin
void setup() {
  pinMode(STCP, OUTPUT);                        // Latch is output
  pinMode(SHCP, OUTPUT);                        // Clock is output
  pinMode(DS, OUTPUT);                          // Data is output
}
//
// The main program counts up from 0 to 255 and displays on the LEDs
//
void loop()
{
  for (int N = 0; N < 256; N++)
  {
    digitalWrite(STCP, LOW);                    // Latch LOW
    shiftOut(DS, SHCP, LSBFIRST, N);            // Shift out
    digitalWrite(STCP, HIGH);                   // Latch HIGH
    delay(500);                                 // 500 ms delay
  }
}
```

7-Segment LED Displays

A 7-Segment LED display is an electronic display module used to show numerical digits in devices such as clocks, calculators and counters. It consists of seven individual LED segments arranged in a figure-8 pattern. By turning specific segments on or off, the display can represent the number 0 through 9 and some letters.

These modules come in two main configurations: common-cathode and common-anode, which define how the segments are electrically wired and how they must be driven by a microcontroller or other logic circuit.

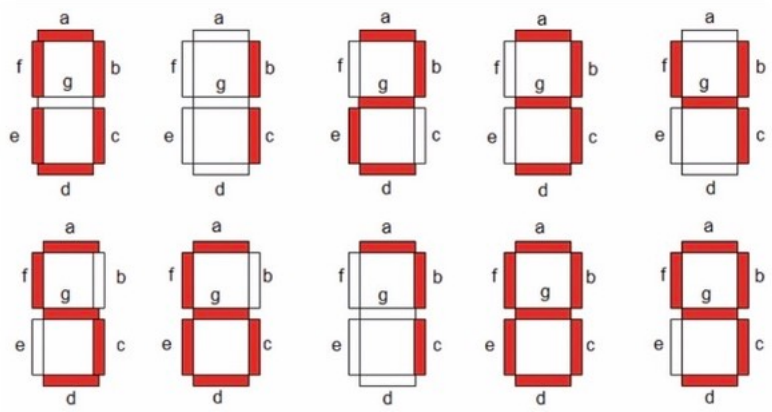


Figure 5 - Displaying numbers 0-9

Before using a 7-segment display, you must map the pins to the MCU ports. Table 4.1 shows the number to be displayed and the data to be sent to the port pins. Notice that the MSB bit (shown as x in the table) is not used and is set to 0.

Number	x g f e d c b a	Port data in Hex
0	0 0 1 1 1 1 1 1	0x3F
1	0 0 0 0 0 1 1 0	0x06
2	0 1 0 1 1 0 1 1	0x5B
3	0 1 0 0 1 1 1 1	0x4F
4	0 1 1 0 0 1 1 0	0x66
5	0 1 1 0 1 1 0 1	0x6D
6	0 1 1 1 1 1 0 1	0x7D
7	0 0 0 0 0 1 1 1	0x07
8	0 1 1 1 1 1 1 1	0x7F
9	0 1 1 0 1 1 1 1	0x6F

Figure 6 - Number to be displayed and port data

7-Segment 1-Digit LED Counter

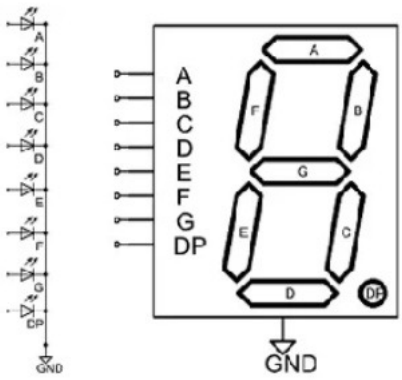


Figure 7 - 7 Segment 1-digit

7-Segment 4-Digit multiplexed LED Display

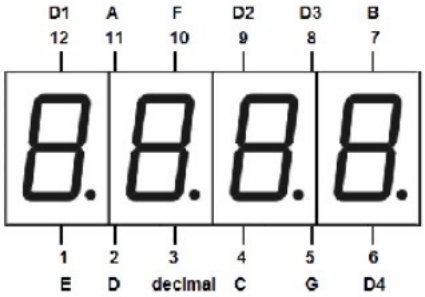


Figure 8 - 7 Segment 4-digit

Liquid Crystal Displays (LCD)

A Liquid Crystal Display (LCD) is a flat panel display technology commonly used in digital watches, calculators, monitors and embedded systems to visually present information. It works by controlling the light passing through liquid crystal material, which does not emit light itself but instead uses a backlight or ambient light to make the characters visible. LCDs are energy-efficient, lightweight, and capable of displaying numbers, text and even complex graphics.
They are available in formats such as character LCDs (e.g. 16x2 or 20x4) and graphic LCDs, making them very popular in microcontroller-based projects for user interfaces and data visualization.

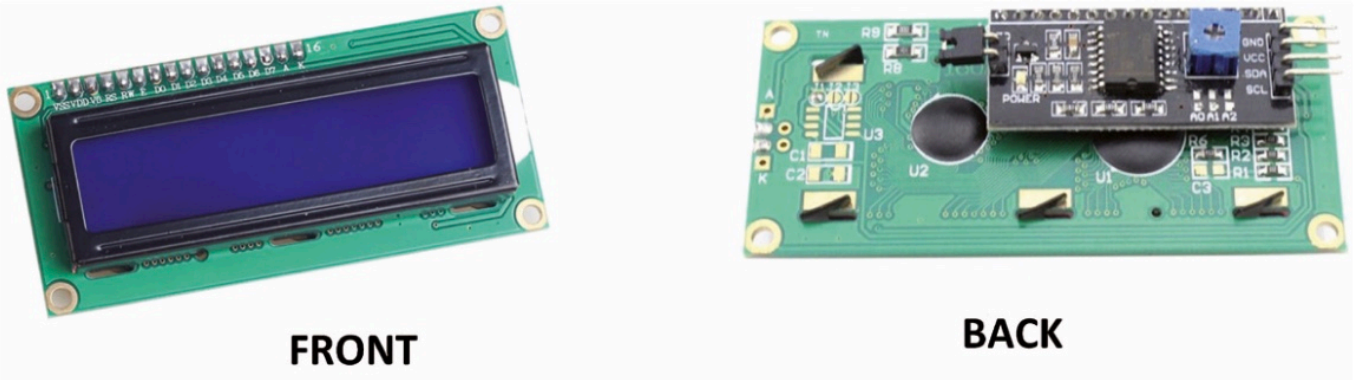


Figure 9 - Standard I2C-based LCD

The following example shows a simple way to use a shift register with Arduino.

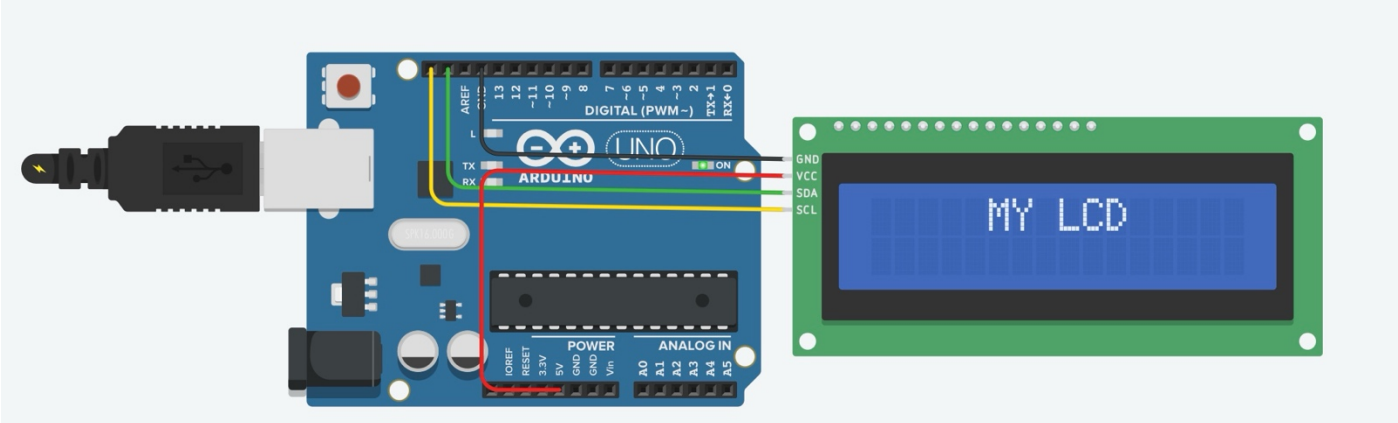


Figure 10 - Circuit View | LCD

```
//-------------------------------------------------------------------
// DISPLAYING TEXT ON THE LCD
// ==========================
//
// In this project the text MY LCD is displayed at the top row,
// starting column 5 of the LCD
//
// Author: Dogan Ibrahim
// File : LCD1
// Date : June, 2023
//-------------------------------------------------------------------
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  lcd.init();                          // Inicialize the LCD
  lcd.backlight();                     // Backlight ON
}
void loop() {
 lcd.setCursor(5, 0);                 // Row 0, Column 5
 lcd.print("MY LCD");                 // Display text
 while(1);                            // Wait here forever
}
```

LM35

LM35 is a 3-pin temperature sensor chip with pins: +V, GND, and OUT. The OUT pin is connected to analog input A0 of the Arduino Uno R4 development board. The analog-to-digital converter (ADC) on the processor is by default 10-bits wide, having 1024 quantization levels. Therefore, with a +5 V reference voltage, each quantization level corresponds to 4.88 mV. For example, 4.88 mV analog input corresponds to 10-bit digital data: 00 0000 0001, similarly, 9.76 mV corresponds to 00 0000 0010, and so on. The reading of the ADC must be multiplied by 1023/5000 to give the actual physical voltage in millivolts present at the analog input.

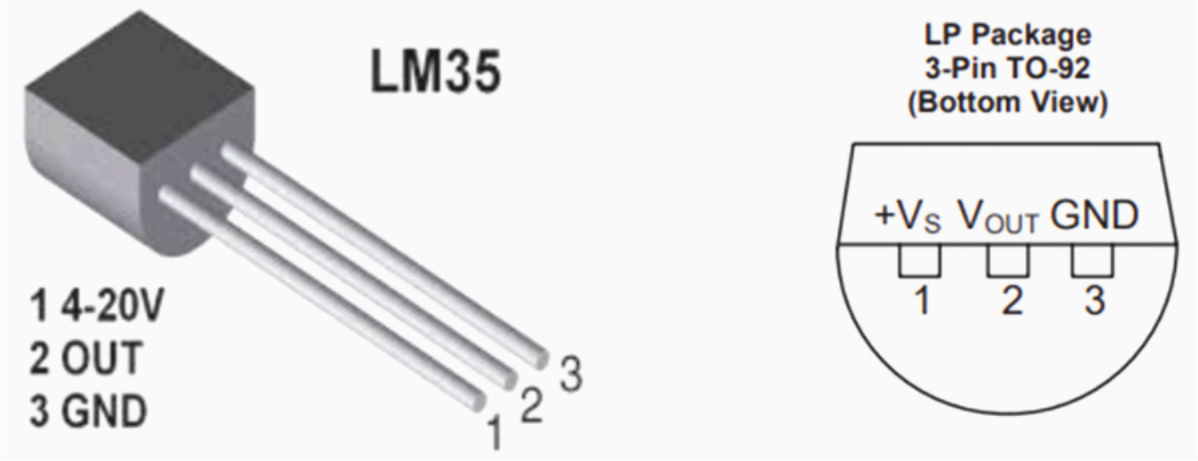


Figure 11 - LM35 temperature sensor chip

The following example shows a simple way to use a shift register with Arduino.

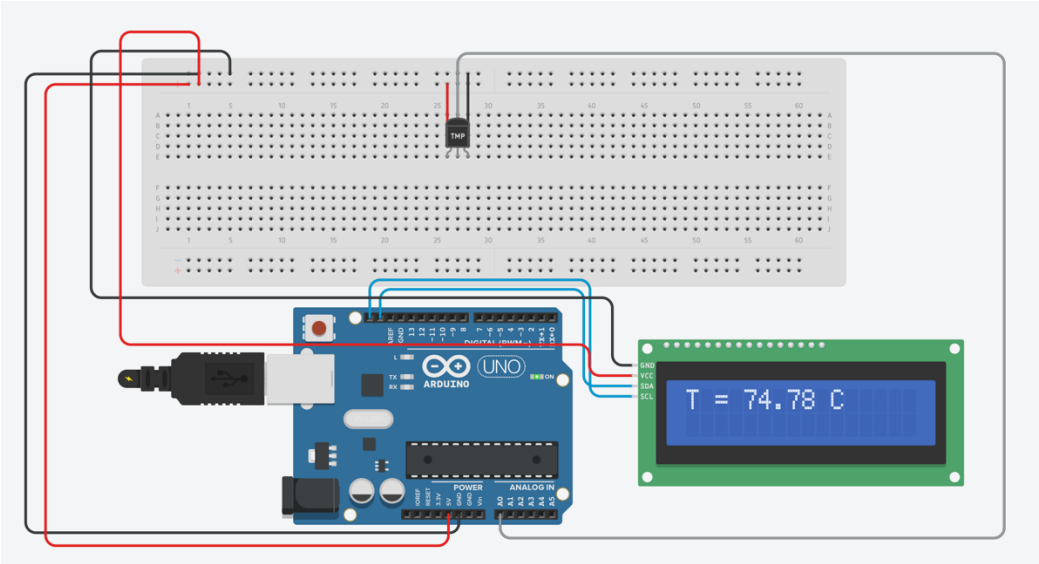


Figure 12 - Circuit View | LM35

```
//---------------------------------------------------------------------
// LM35 TEMPERATURE DISPLAY
// ========================
//
// In this project the LM35 temperature sensor chip is used to measure
// and display the ambient temperature on the LCD
//
// Author: Dogan Ibrahim
// File : LM35
// Date : June, 2023
//---------------------------------------------------------------------
#include <LCD_I2C.h>
LCD_I2C lcd(0x27, 16, 2);
#define LM35 A0                 // LM35 at port A0
int raw;
void setup()
{
lcd.begin();                   // Initialize LCD
lcd.backlight();                 // BAcklight ON
}
void loop()
{
raw = analogRead(LM35);              // Read temperature
float mV = 5000.0 * raw / 1023.0;        // in mV
float Temperature = mV / 10.0;          // True temperature
lcd.setCursor(0, 0);
lcd.clear();                   // Clear LCD
lcd.print("T = ");                // Display T =
lcd.print(Temperature);              // Display temperature
lcd.print(" C");                 // Display C
delay(2000);                   // Wait 2 seconds
}
```

Water Level Sensor

The water level sensor (Figure 6.19) is used to detect the presence of water, for example, the level of water in a tank, the level of water in a pool, to detect rainfall, to detect water leakage, and so on. The sensor has a series of ten exposed copper traces, five of which are power traces and five are sense traces. These traces are interlaced so that there is one sense trace between every two power traces. The traces are bridged by water when submerged in water or when water is present on the sensor. A small LED is lit on the sensor when power is applied.

The sensor has 3 pins: - (GND), + (power supply), and S (analog output). the basic specifications of the sensor are:

- Operating voltage: 3.3 V to 5 V
- Current: less than 20 mA

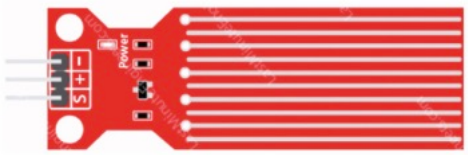


Figure 13 - Water Level Sensor

```
//-----------------------------------------------------------------------
// WATER LEVEL SENSOR
// ==================
//
// In this project a water level sensor is used. The program measures
// the water leproject measures the water level. Serial Monitor is used
// to tabulate the results
//
// Author: Dogan Ibrahim
// File : Waterlevel
// Date : June, 2023
//-----------------------------------------------------------------------
int POWER = 2;               // Power at port 2
#define Sensor A0
void setup()
{
 pinMode(POWER, OUTPUT);         // POWER is output
 Serial.begin(9600);           // Serial monitor
 delay(5000);
}
void loop()
```

```
{
 digitalWrite(POWER, HIGH);       // Apply power to sensor
 int raw = analogRead(Sensor);     // Read sensor value
 Serial.println(raw);          // Display ADC value
 delay(2000);              // One second delay
}
```

DHT11

The standard DHT11 is a 4-pin digital output device (only 3 pins are used) shown in Figure 6.38, having pins +V, GND, and Data. The Data pin is internallypulled up to +V through a 10-kΩ resistor. The chip uses a capacitive humidity sensor and a thermistor to measure the ambient temperature. Data output is available from the chip around every few seconds.

The basic features of DHT11 are:

- 3 to 5 V operation
- 2.5 mA current consumption (during a conversion)
- Temperature reading in the range 0-50 ºC with an accuracy of ±2 ºC
- Humidity reading in the range of 20-80% with 5% accuracy
- Breadboard compatible with 0.1-inch pin spacings

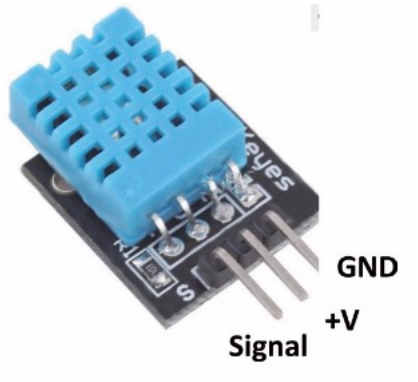


Figure 14 - DHT11 module

```
//-----------------------------------------------------------------------
// DHT11 TEMPERATURE AND HUMIDITY DISPLAY
// ======================================
//
// In this project a DHT11 temperature and humidity sensor chip is used.
// The data read is displayed on the LCD every 5 seconds
//
// Author: Dogan Ibrahim
// File : DHT11monitor
// Date : June, 2023
//-----------------------------------------------------------------------
#include "DHT.h"
#include <LCD_I2C.h>
LCD_I2C lcd(0x27, 16, 2);
float hum, temp;
#define Sensor 2               // DHT11 Sensor at pin 2
#define DHTTYPE DHT11 DHT          // DHT11 is used
dht(Sensor, DHTTYPE);
void setup()
{
lcd.begin(); // initialize the lcd
lcd.backlight(); // Backlight ON
dht.begin();
}
//
// Read the temperature and humidity. DHT11 sometimes fails to return
// data. Catch this condition and return correct status to teh calling
// program. Returning 1 is success, 0 is error
//
int ReadDHT11()
{
hum = dht.readHumidity();         // Read humidity
temp = dht.readTemperature();       // Read temperature in C
 // Check if any read failed and return status to caller to try again
 if (isnan(hum) || isnan(temp))
 {
  return 0;
 }
 else
  return 1;
 }
void loop()
{
 delay(2000);               // DHT11 is a slow device

 if(ReadDHT11() == 1)           // If successful read
 {
  lcd.clear();              // Clear LCD
  lcd.setCursor(0, 0);          // Cursor at top
  lcd.print("T=");            // Display T=
  lcd.print(temp);            // Display temperature
  lcd.print(" C");            // Display C
  lcd.setCursor(0, 1);          // Cursor at bottom
  lcd.print("H=");            // Display H=
  lcd.print(hum);            // Display humidity
  lcd.print(" %");            // Display %
  delay(5000);              // Wait 5 seconds
```

```
}
else                    // Failed to read
 delay(3000);               // Wait 3 seconds
}
```

Week 1

Scratch

Day	Logbook of work
Monday	I studied the website
Tuesday	I started experimenting with the commands
Wednesday	Created the game I submited
Thursday	
Friday	------

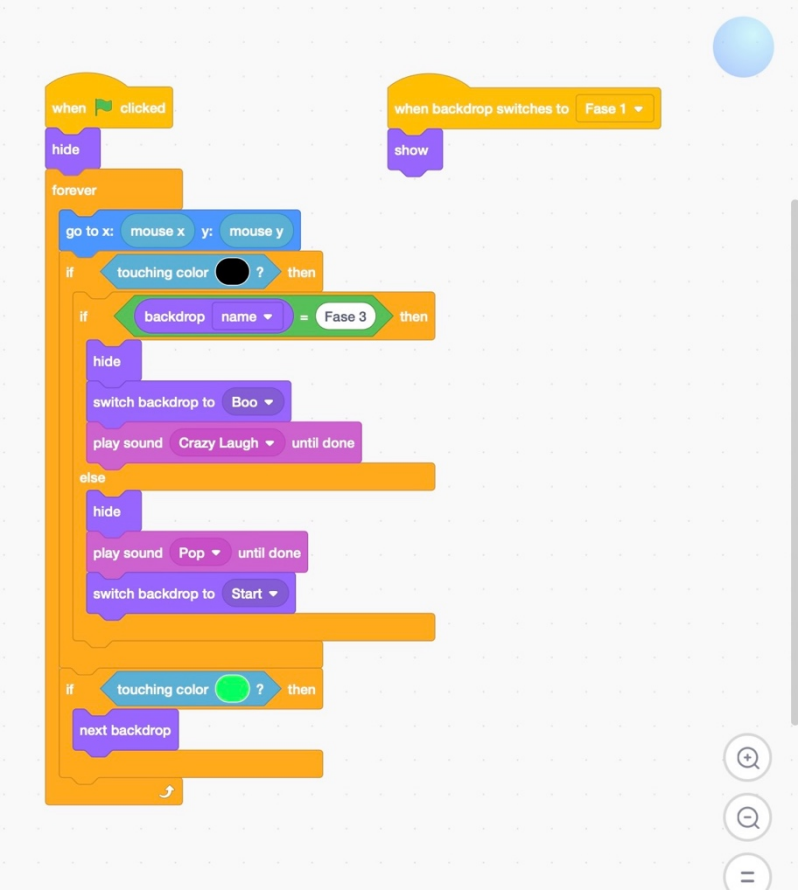


Figure 15 - Game_1

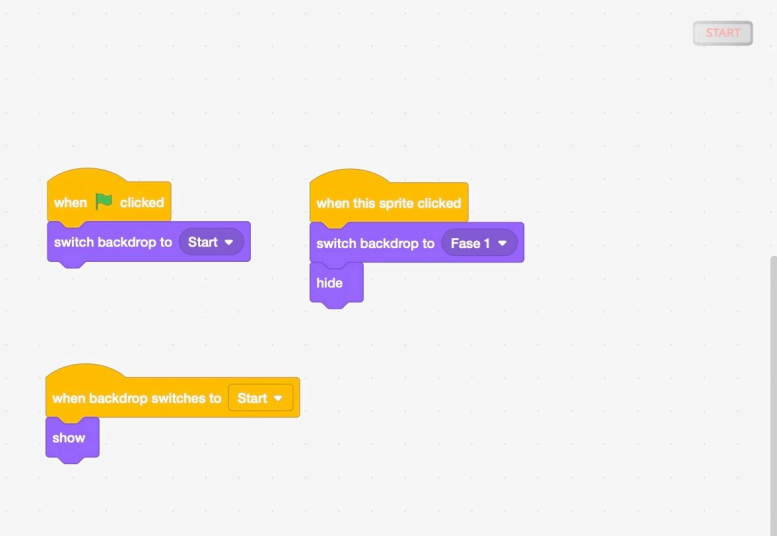


Figure 16 - Game_2

me50 / users / franciscpime / cs50 / problems / 2025 / x / scratch

#1 submitted a few seconds ago, Wednesday, September 10, 2025 1:26 PM CEST
check50 7/8 • 0 comments
tar.gz • zip

Access link: https://submit.cs50.io/check50/9605b8898580468e92904c10bcc96cb133c6ab3a

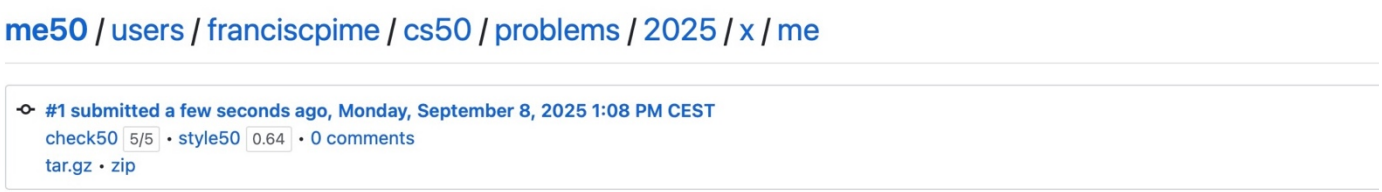
Access game: https://scratch.mit.edu/projects/1212787206

Hello, It’s me

Hello, It’s me - Result:

Figure 17 - Hello, It's me | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/me

Micro Bit

This program makes the Cutebot walk only on the black line and whenever it turns left or right it indicates using the LEDs. Additionally, whenever there is an obstacle within 15 centimeters, Cutebot stops, creating a safe distance between it and whatever is in front of it.

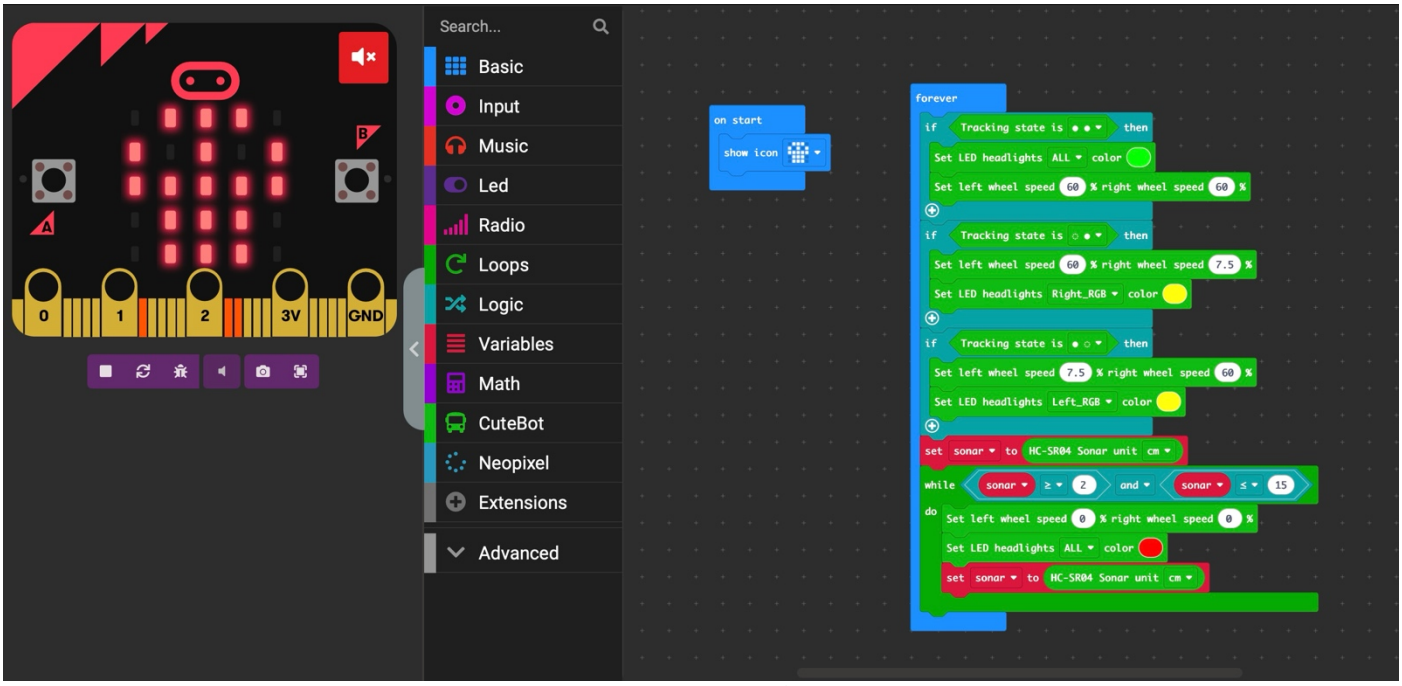


Figure 18 - Micro:bit

Access link: https://makecode.microbit.org/S69620-67556-83270-99053

Self-Reflection 1

My first week at university was really different from what I was used to in high school. Everything feels faster, and the teachers expect us to be much more independent. At the beginning, it was a bit overwhelming, but I actually like it because I can play my own work, and I feel more responsible for what I'm doing. It also made me realize that here the learning is more practical, which is way more motivating for me.

This week I had to deliver two assignments, went for computer science and another for Embedded Systems. For computer science, the task was to create a game in Scratch. I worked on it both at home and at university, and I asked some classmates to play it so I could get their feedback. That was really helpful, because they pointed out things I didn't notice, and I improved the game based on their opinions. I understood how important it is to listen to players, because the games actually grow when you listen to the community. For Embedded Systems, the challenge was programming the Cutebot so it behaves according to the instructions from our teachers. It took me some time to get it right, but in the end it worked, and I also shared ideas with classmates well asking for their help when I got stuck.

Overall, I feel like I learned a lot this week, not just about calling but also about teamwork and asking for feedback. In the future, I want to learn more about programming in real life situations in how to solve problems in creative ways.

Week 2

Mario Less

In the Mario Less problem, I was asked to build a right-aligned pyramid made of “#” characters, similar to the blocks in Super Mario. The program first prompts the user to enter a number between 1 and 8, which represents the height of the pyramid. Then, based on that value, it prints the pyramid line by line with the correct alignment.

Mario Less - Result:

Figure 19 - Mario Less | Result

Code

me50 / users / franciscpime / cs50 / problems / 2025 / x / mario / less

#1 submitted a few seconds ago, Thursday, September 11, 2025 8:47 AM CEST
check50 10/10 • style50 0.55 • 0 comments
tar.gz • zip

Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/mario/less

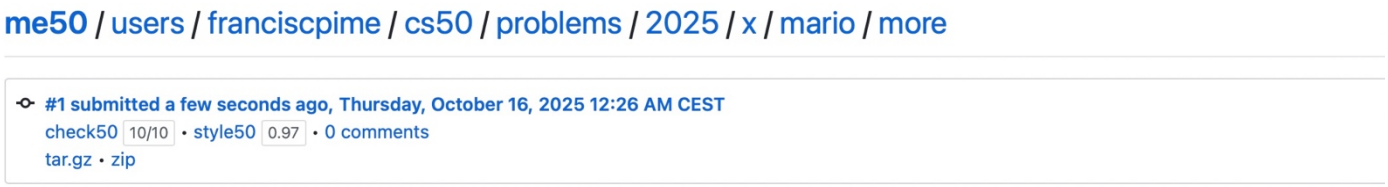
Mario More

In the Mario More problem, I had to build not just one, but two side-by-side pyramids made of “#” characters. Just like in Mario Less, the program asks the user for a number between 1 and 8, which represents the height. Then, it prints both pyramids aligned correctly, with a gap of exactly two spaces in between.

Mario More - Result:

Figure 20 - Mario More | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/mario/more

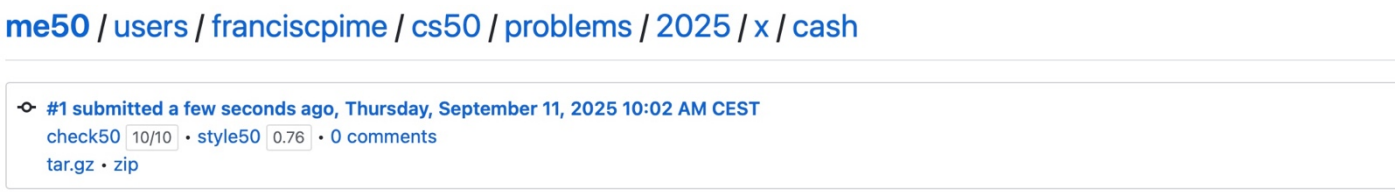
Cash

In the Cash problem, I was asked to calculate the minimum number of coins needed to give someone a certain amount of change. The program must prompt the user for a non-negative number of cents and then use a greedy algorithm, always subtracting the largest possible coin first (25, then 10, then 5, then 1) until no change remains.

Cash - Result:

Figure 21 - Cash | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/cash

Mario UNO

In this Arduino Project, I decided to recreate a Mario-style pyramid visual, but bring it to life using the Arduino UNO R4's built-in 12x8 LED matrix and a buzzer. The user simply enters a number between 1 and 8 through the Serial Monitor, and the board renders two perfectly symmetric pyramids mirrored diagonally across the LED matrix.
Each level of the animation also triggers an ascending buzzer tone, creating a dynamic and interactive audiovisual effect.

Mario UNO - Result:

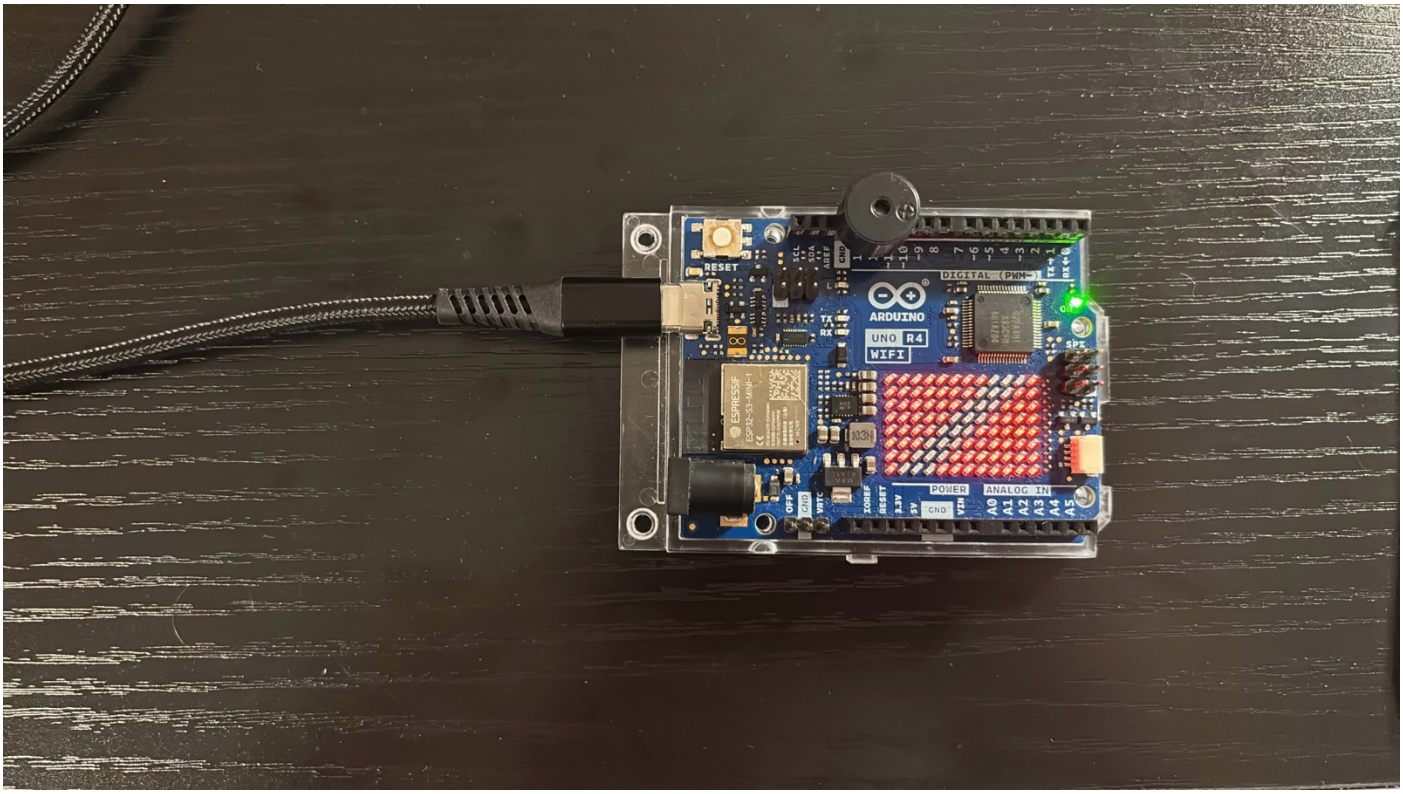


Figure 22 - Mario UNO | Result

Code

Self-Reflection 2

This week I started programming in C and also worked with Arduino for the first time. It was a bit intimidating at the beginning because the syntax and logic of C are quite different from what I was used to, but I slowly started to understand how things connect. For Computer Science, we had to complete two assignments and depending on which version we chose, we could get either G or a U. I decided to go with the easier ones, Mario Less and Cash, because I wanted to make sure I really understood the basics before jumping into something harder.

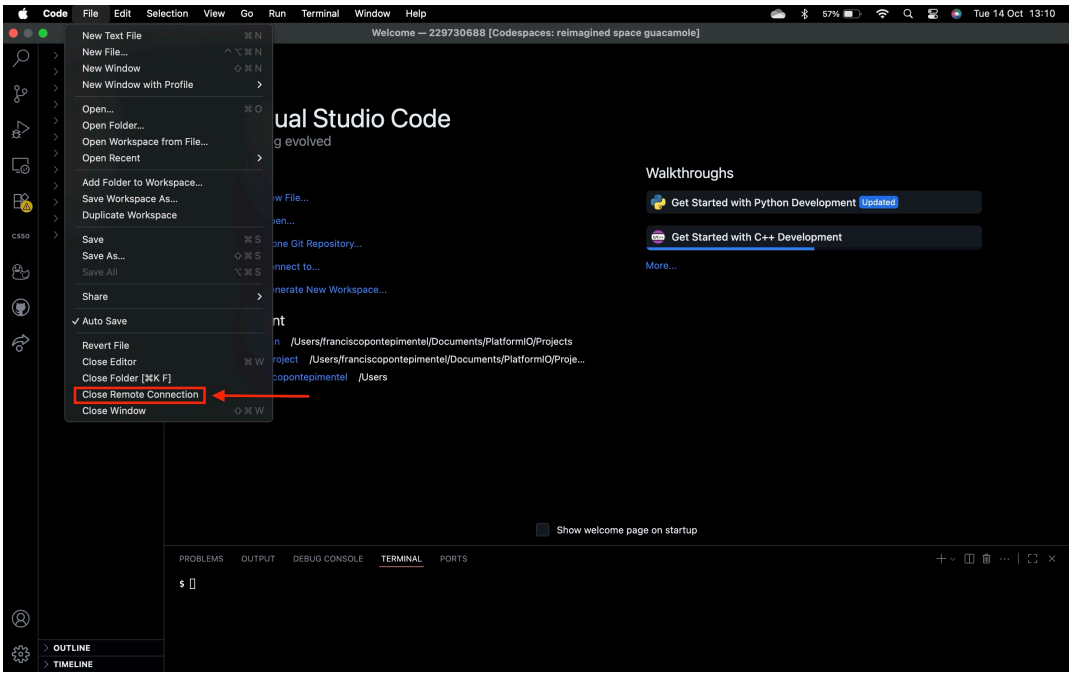
For Embedded Systems, we worked on a small project where we had to make two mirrored pyramids appear using Arduino, inspired by the Mario Less task. It was actually pretty cool to see the physical result of what I programmed. I struggled a bit at first with the code, but after some testing and asking questions to classmates, I got it working. That gave me a nice feeling of accomplishment.

This week taught me that understanding the fundamentals is super important. I also realized that programming requires patience – small mistakes can completely break your code, but fixing them is part of the learning process. I'd like to get better at problem-solving and debugging because I can already see how useful these skills will be later in my studies.

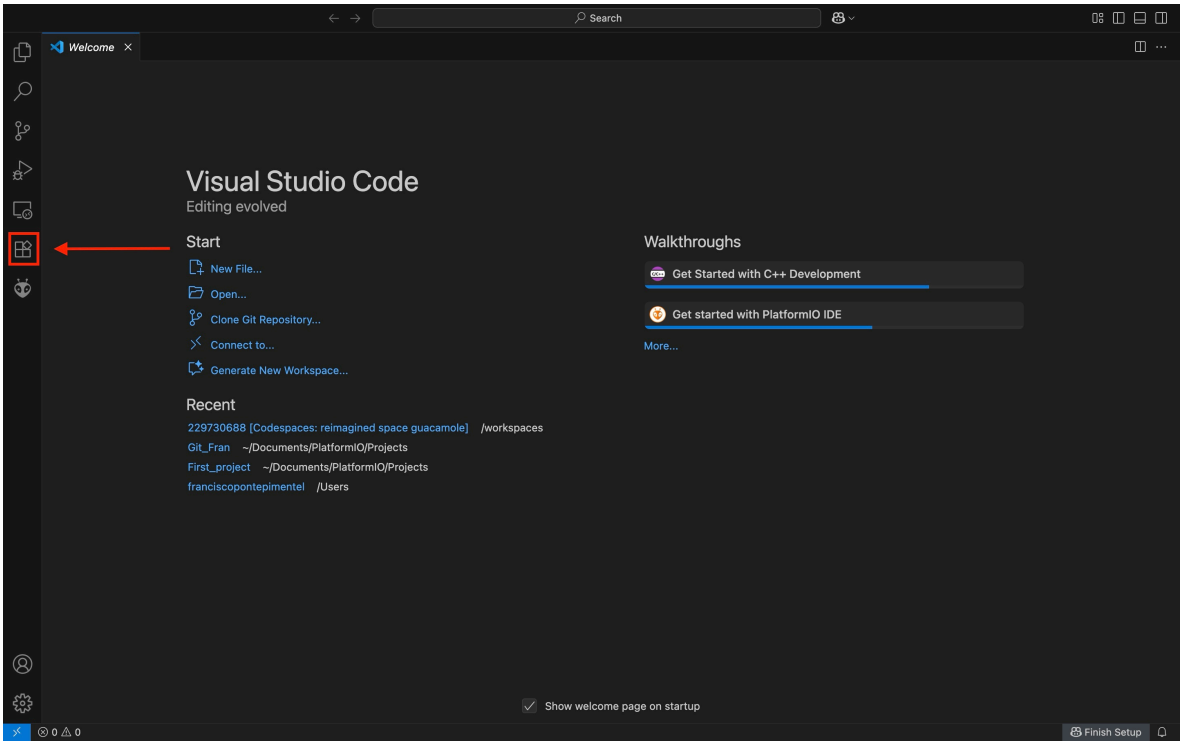
Week 3

Set up guide Arduino in VSCode

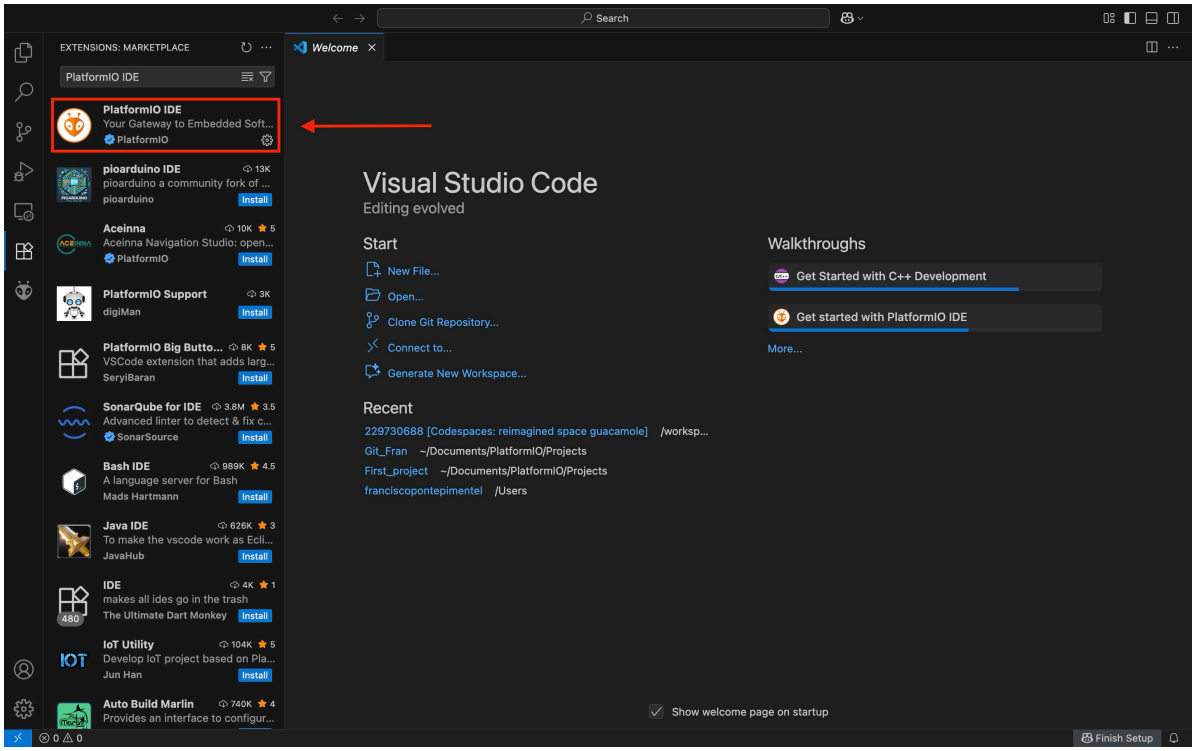
1. Close connection remotely



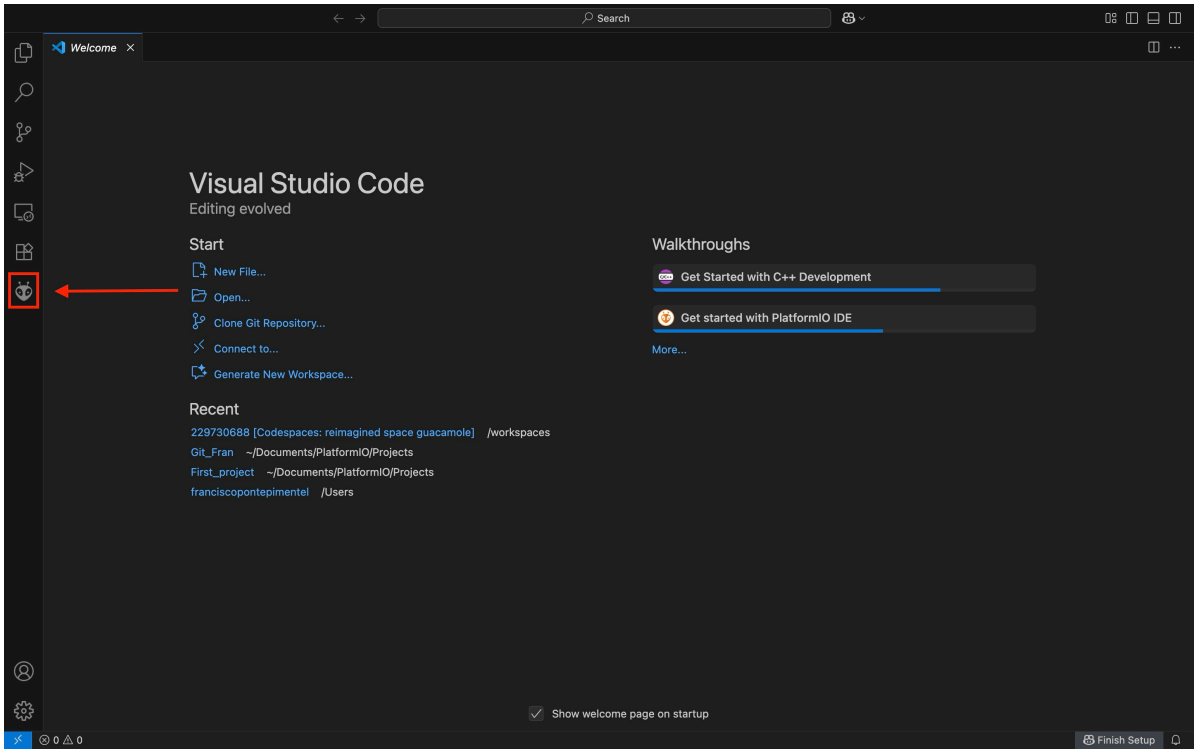
2. Open "Extensions”



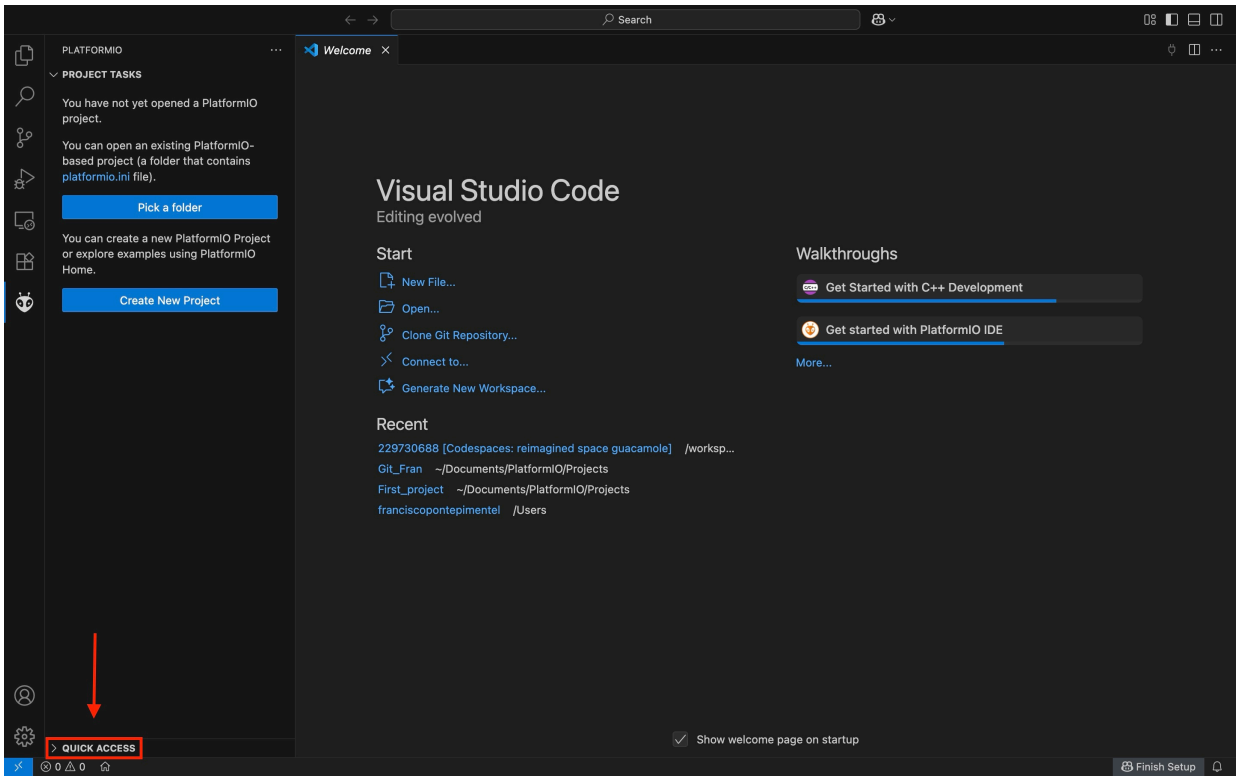
3. Install PlatformIO IDE. After installed, restart VScode



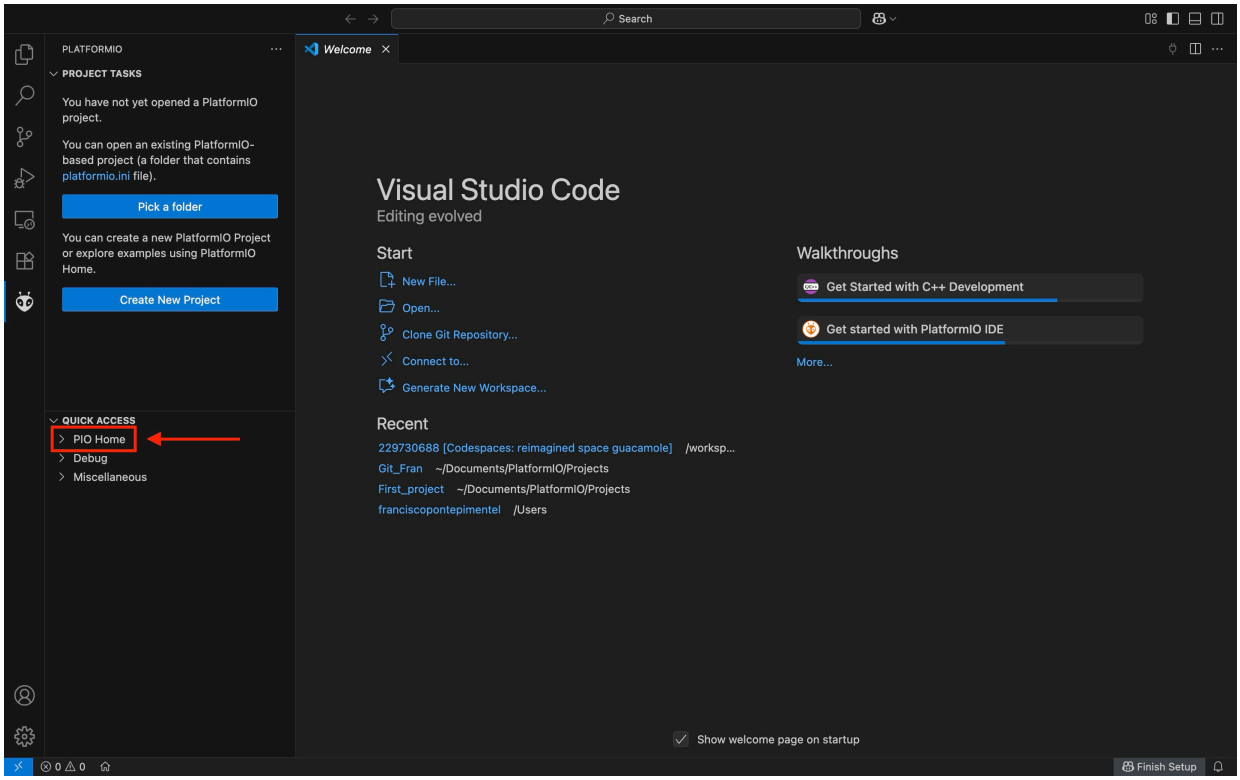
4. Click the platform IO icon



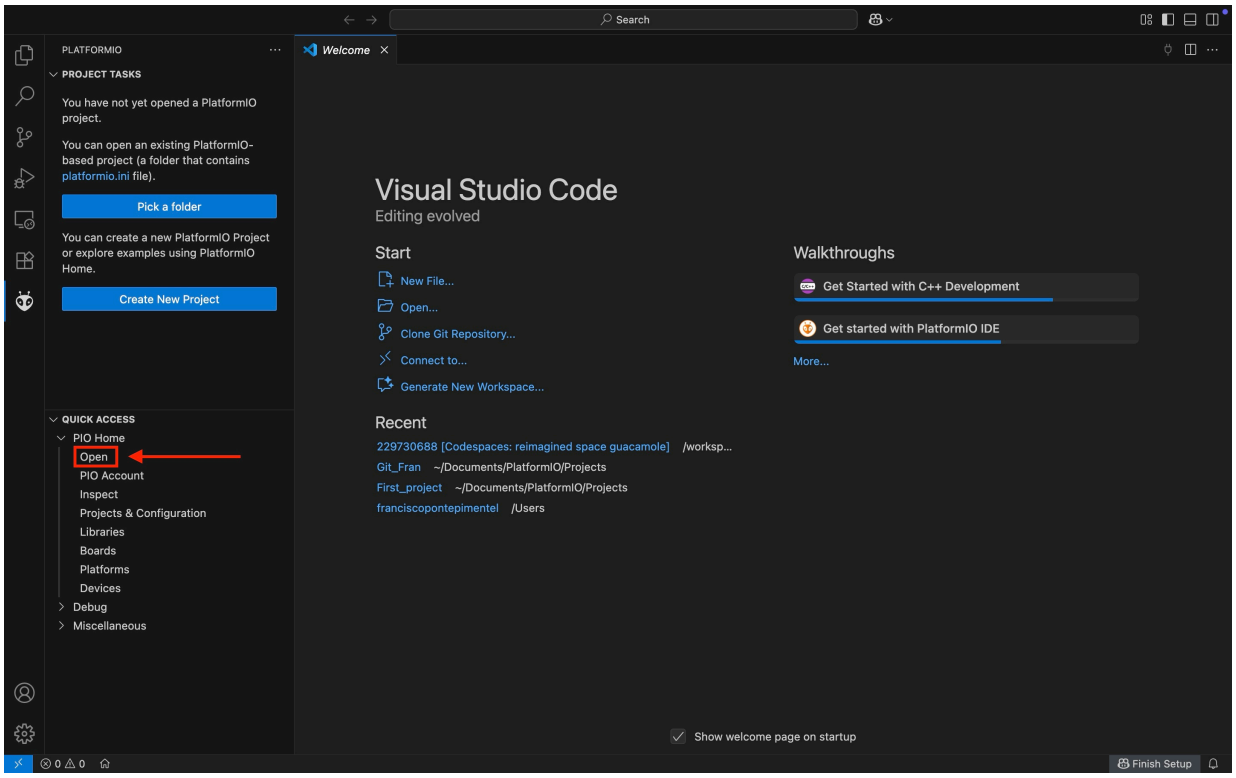
5. Click on “Quick Access”



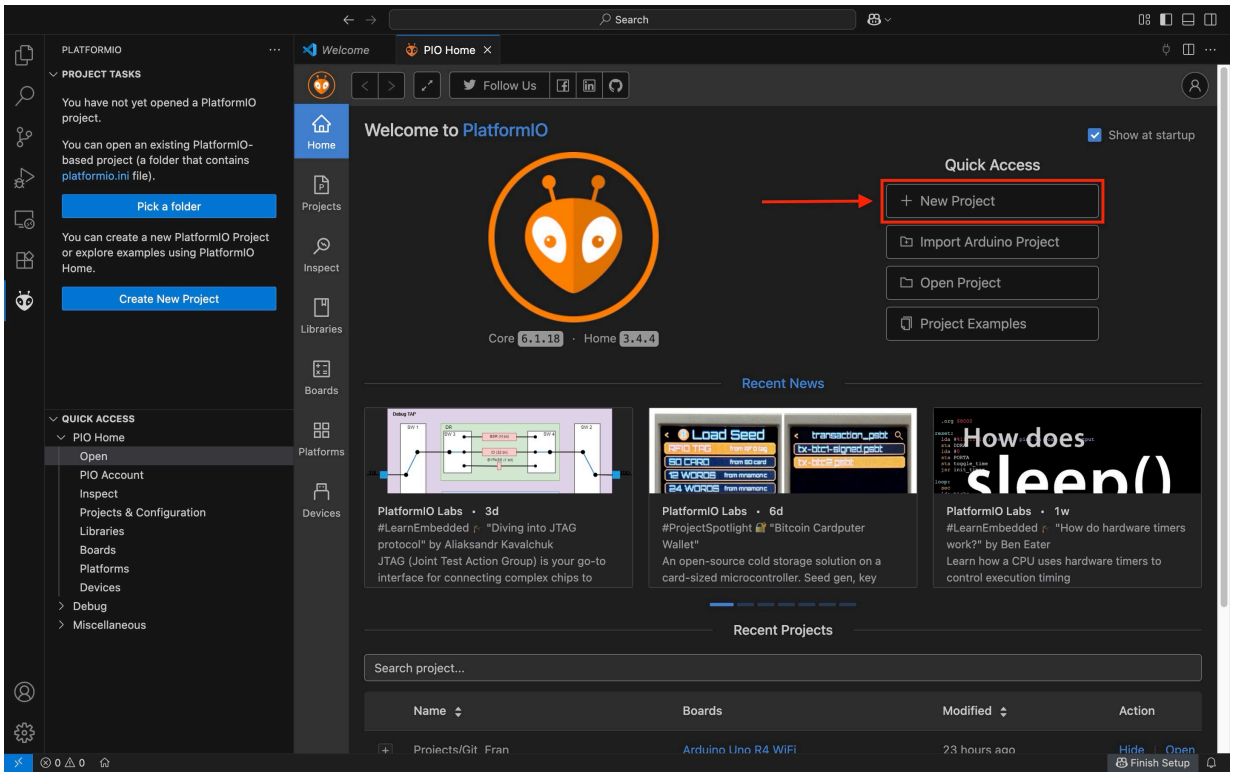
6. Click on “PIO Home”



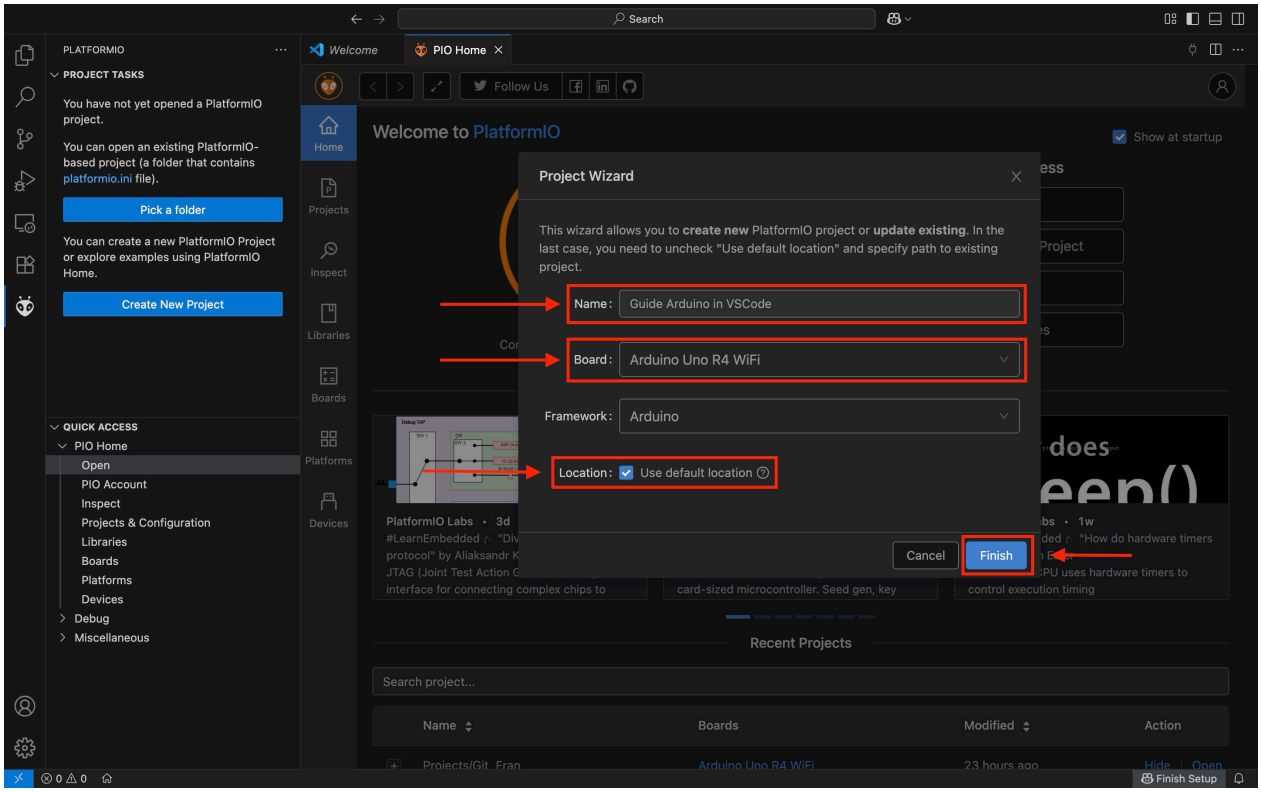
7. Click on “Open”



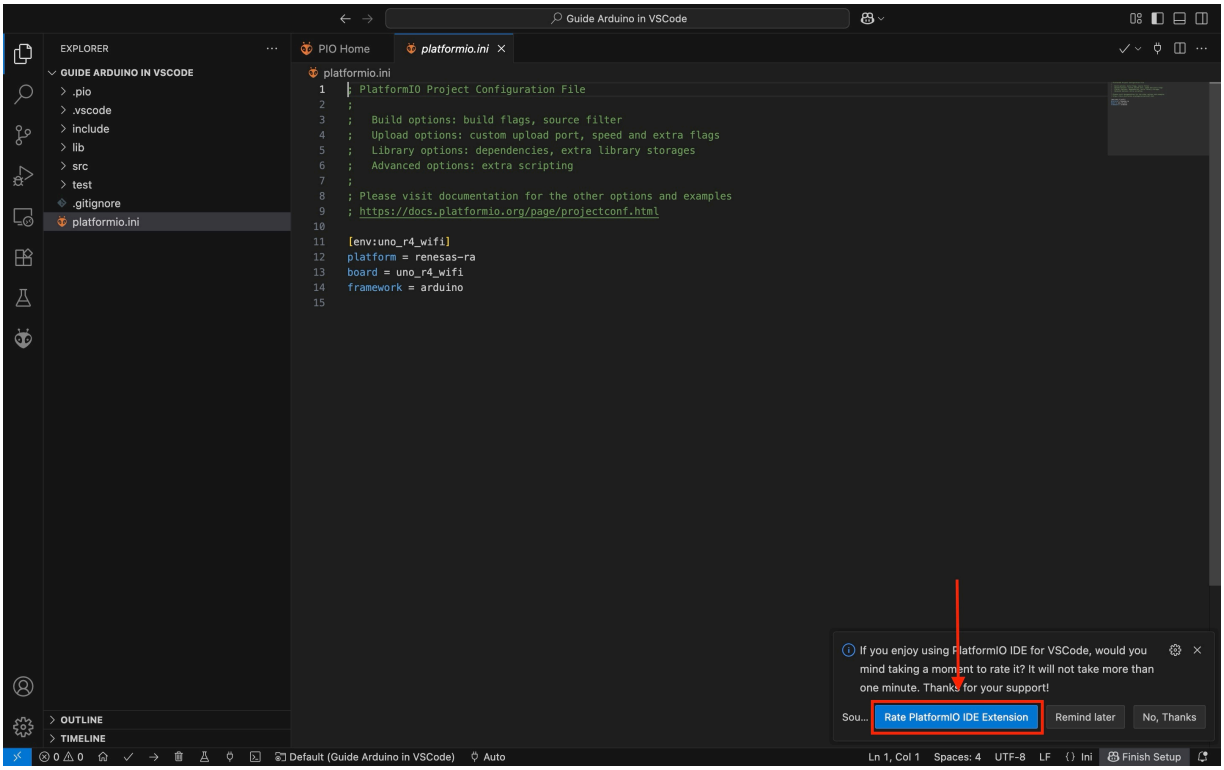
8. Create a new project



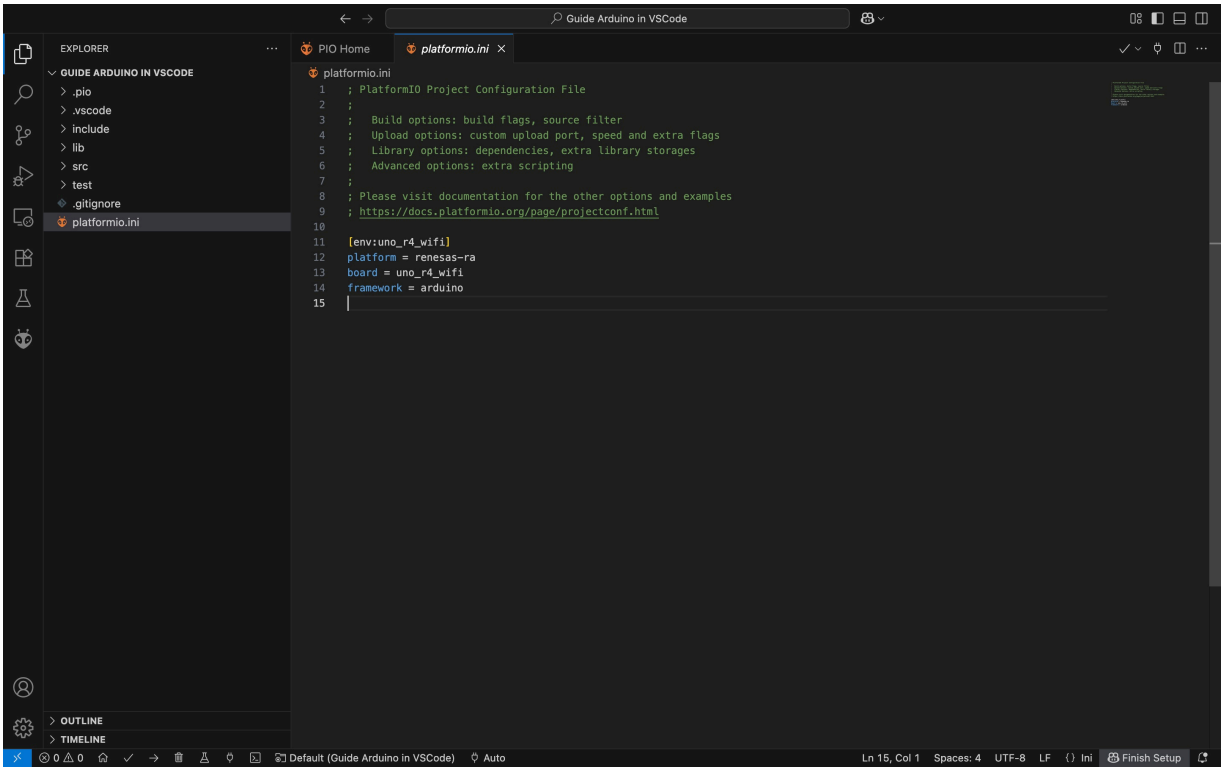
9. Give the project a name, select "Arduino Uno R4 WiFi" in Board, change your code location, and click "Finish"



10. Click on “Rate PlatformIO IDE Extension”



11. Then it’s everything done



Scrabble

In the Scrabble problem, I had to simulate a simplified version of the Scrabble game. The program asks two players to input a word each, then calculates the score of each word based on a predefined point system for every letter. Non-letter characters are ignored, and in the end, the program compares both scores to determine which player wins, or if it's a tie.

Scrabble - Result:

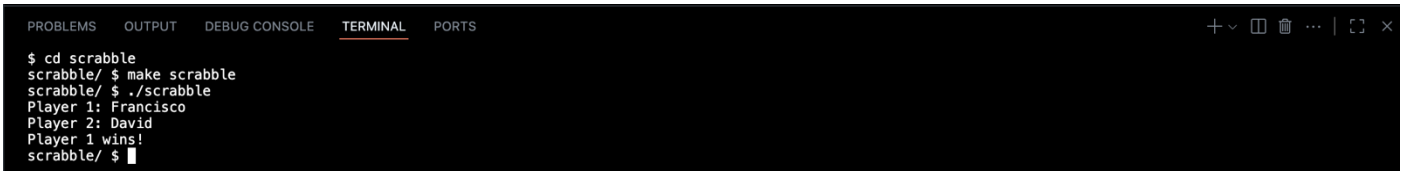
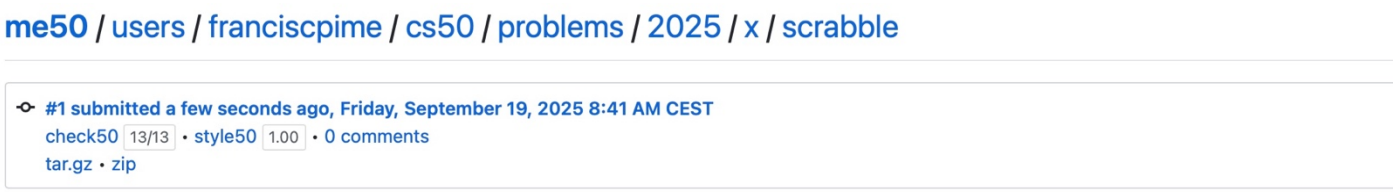


Figure 23 - Scrabble | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/scrabble

Readability

In the Readability problem, I had to analyze a block of text and estimate its U.S. school grade reading level using the Coleman-Liau index. My program counts the number of letters, words, and sentences, applies the readability formula, and then prints the corresponding grade, from “Before Grade 1” all the way up to “Grade 16+” for university-level text.

Readability - Result:

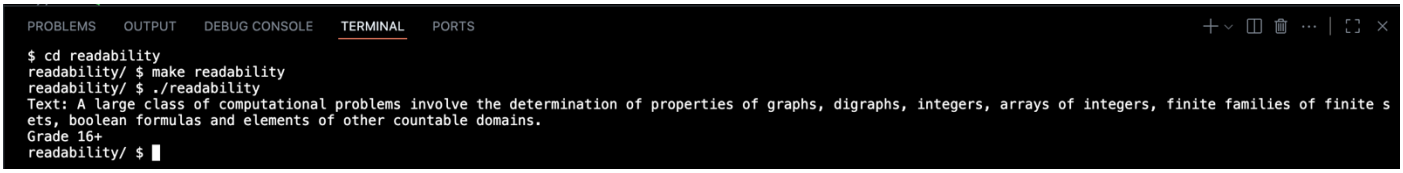
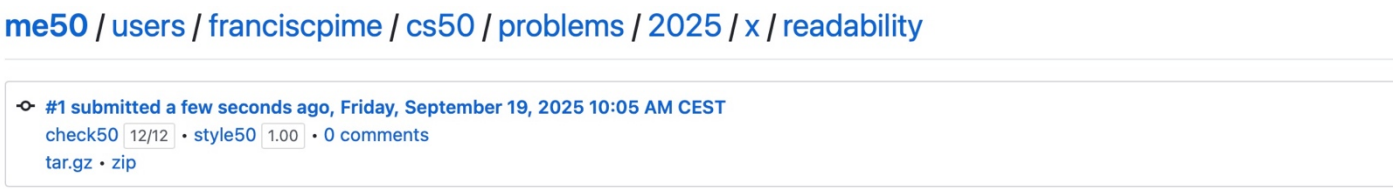


Figure 24 - Readability | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/readability

Caesar

In the Caesar problem, I had to implement the Caesar cipher, a classic encryption technique that shifts each letter in the plaintext by a fixed number of positions in the alphabet. My program first checks that the key provided as a command-line argument is a valid number, then applies the shift to every letter, keeping uppercase and lowercase intact, and leaving non-alphabetic characters unchanged.

Finally, it prints the encrypted result as the ciphertext.

Caesar - Result:

Figure 25 - Caesar | Result

Code

https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/caesar

Morse Code

In this Arduino project, I built a real-time Morse code translator that takes any sentence from the Serial Monitor and converts it into both audio and visual Morse signals. The system uses the Arduino R4's 8x12 LED matrix to display dots and dashes, while a buzzer and an external LED provide synchronized sound and light feedback.

It fully supports letters (A-Z), numbers, and several punctuation symbols, respecting official Morse timing between symbols, letters, and words.

Morse Code - Result:

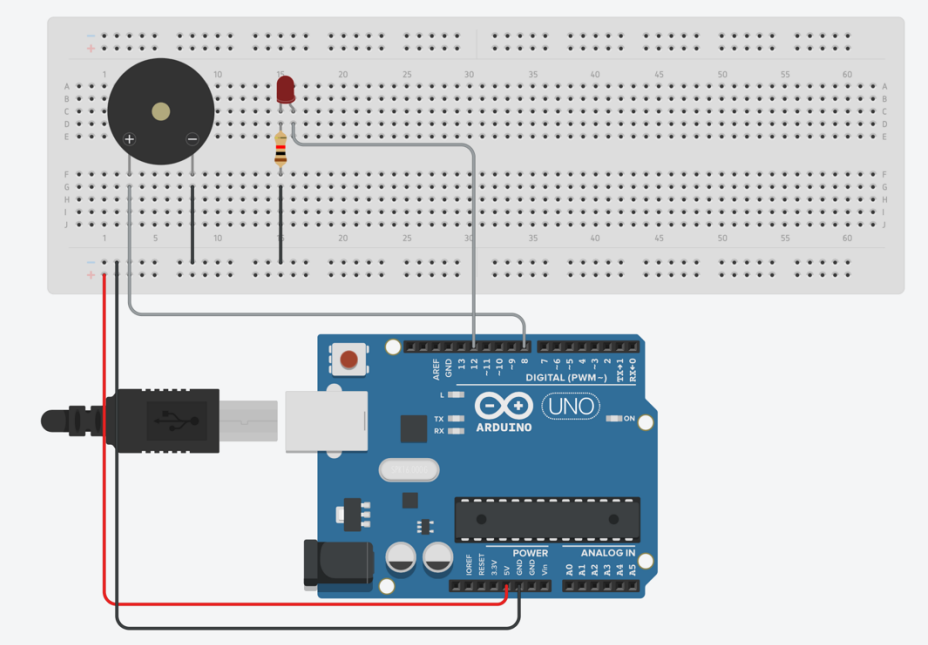


Figure 26 - Morse Code | Circuit View

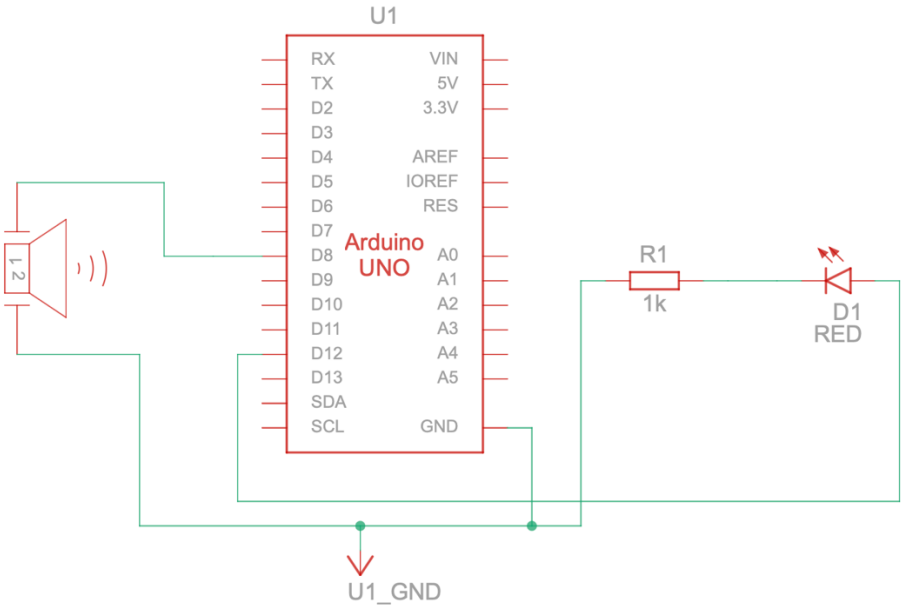


Figure 27 - Morse Code | Schematic View

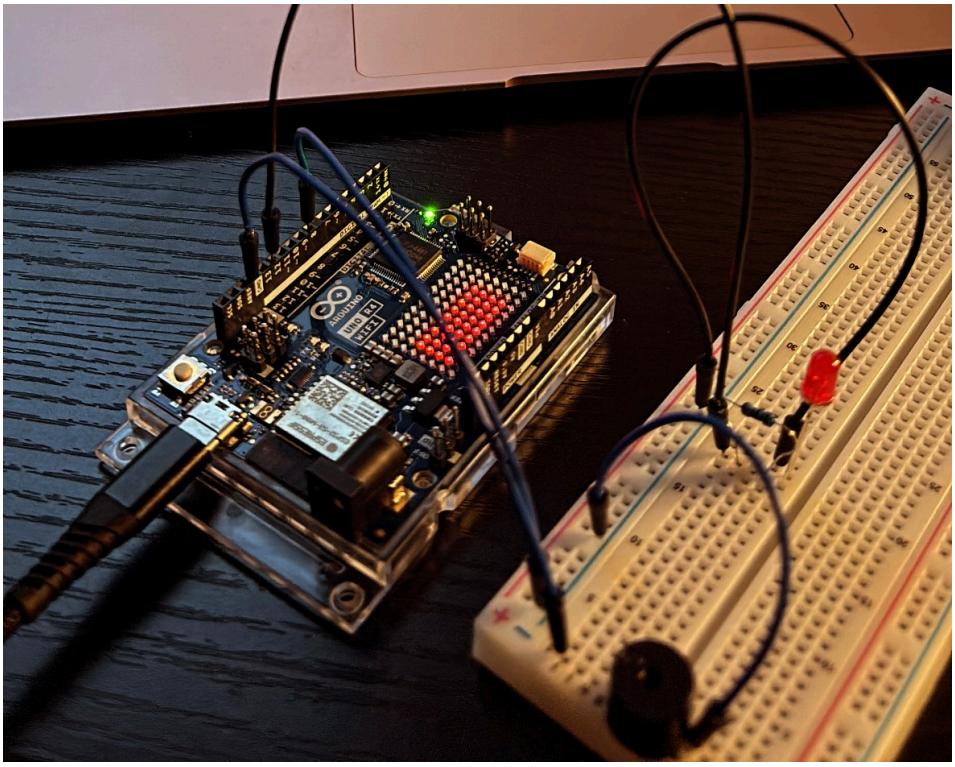


Figure 28 - Morse Code | Real View

Code

Self-Reflection 3

This week was definitely more intense then the previous one. Instead of having just two assignments, we now had three to complete each week. For Computer Science, I worked on Scrabble, Readability, and Caesar, while for Embedded Systems, we had to program something that could send messages in Morse code. It was a big step up in difficulty, especially since I'm still getting used to programming in C.

At first, it felt like a lot to handle – I spent quite a few hours trying to make everything work and understand the logic behind the code. The pressure of finishing everything by Friday started to get to me a bit, but luckily our teachers extended the deadlines to the following week. That extra time really helped me focus on improving my code instead of just rushing to submit something that barely worked. I ended up finishing Caesar and Morse Code on Monday, and that felt pretty satisfying.

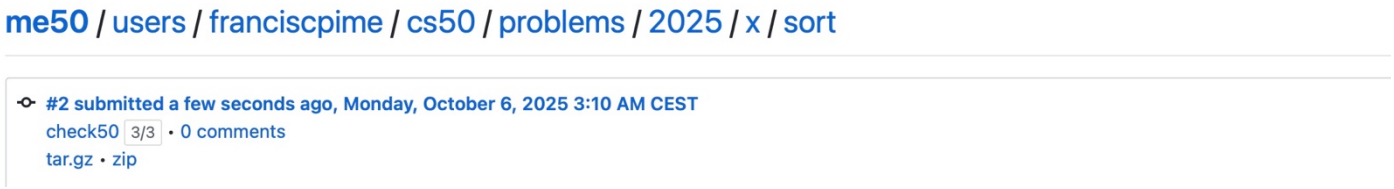
What I learned this week is that time management matters a lot when there is more workload, and that it's okay to ask for help or take more time when things get tough. I also started to notice that I'm getting better at spotting errors and thinking more logically, which makes me feel like I'm actually starting to get that hang of C programming.

Week 4

Sort

This short analysis compares three sorting algorithms, bubble sort, merge sort ans selection sort, based on how their performance changes depending on the order of the input data. By observing how long each program takes to sort lists that are already sorted, reversed, or random, it is possible to infer which sorting method each progrm uses.

```
sort1 uses: bubble sorte
How do you know?:
Because when run on sorted10000.txt it finishes quickly, but when run on reversed10000.txt
it takes much longer.
sort2 uses: merge sort
How do you know?:
Because the runtime is always fast and similar, whether the list is sorted, reversed, or
random.
sort3 uses: selection sort
How do you know?:
Because its runtime is about the same whether the list is sorted or reversed.
```



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/sort

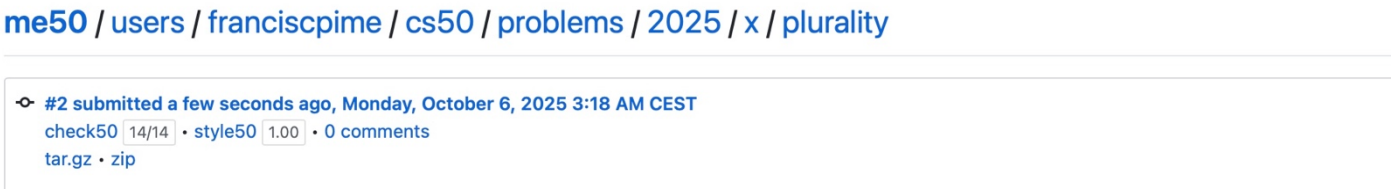
Plurality

In the Plurality problem, I had to simulate a simple voting system where each voter chooses one candidate, and with the most votes wins. The program receives the candidate names as command-line arguments, then asks the user for each vote, updating the vote counts accordingly.
After all votes are collected, it prints the winner, or multiple winners in case of a tie.

Plurality - Result:

Figure 29 - Plurality | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/plurality

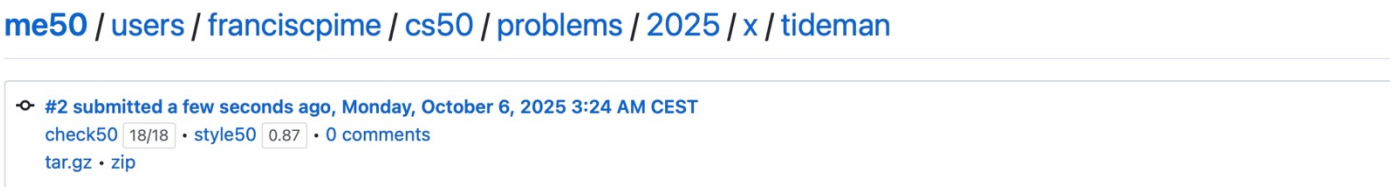
Tideman

In the Tideman problem, I had to implement a ranked choice voting system where each voter ranks the candidates by reference. The program compares every pair of candidates head-to-head, determines the strength of victory, and then “locks in” the strongest pairs without creating cycles in the graph.
In the end, the candidate who is not beaten by anyone is declared the winner.

Tideman - Result:

Figure 30 - Tideman | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/tideman

Traffic Lights

In this Arduino Project, I developed a fully automated traffic light system with pedestrian crossing integration. It uses shift register to drive multiple traffic signals, a 7-segment display to show a live countdown for pedestrians, and a buzzer for audible feedback during crossing.

A physical push button allows pedestrians to request passage, which the system “schedules” based on the current traffic light phase. The logic handles full east-west and north-south light sequencing with precise timing, while safely interrupting the cycle to give pedestrians right of way without causing traffic conflicts.

Traffic Lights - Result:

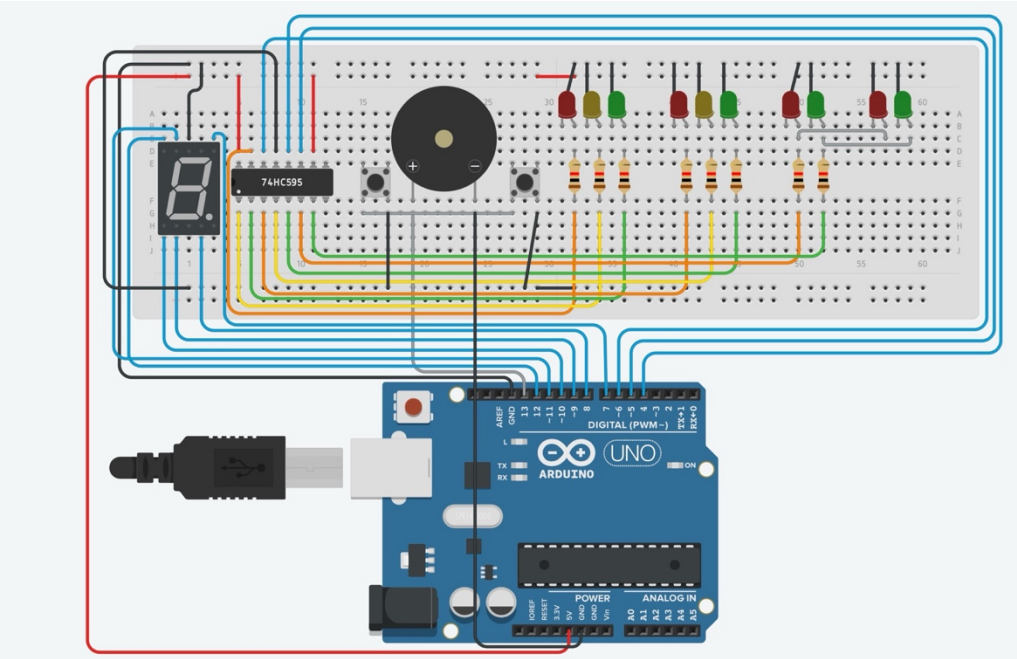


Figure 31 - Traffic Lights – Circuit View

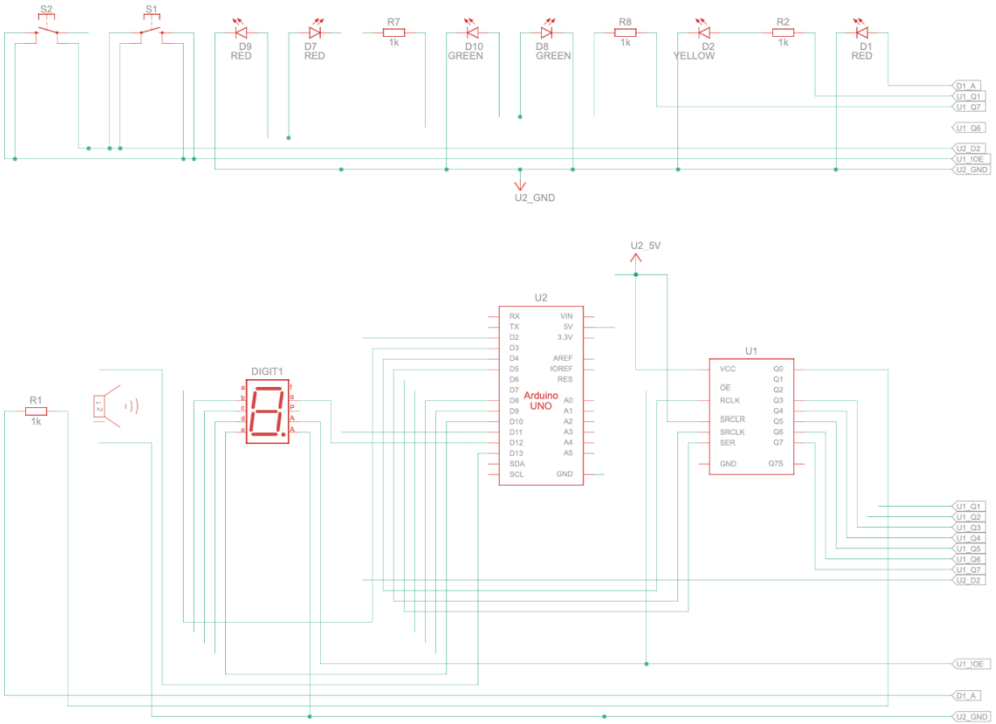


Figure 32 - Traffic Lights – Traffic Lights | Schematic View1

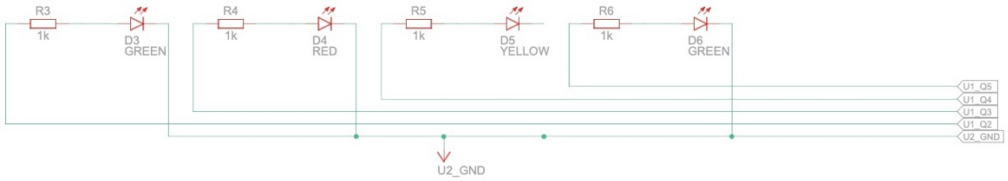


Figure 33 - Traffic Lights | Schematic View2

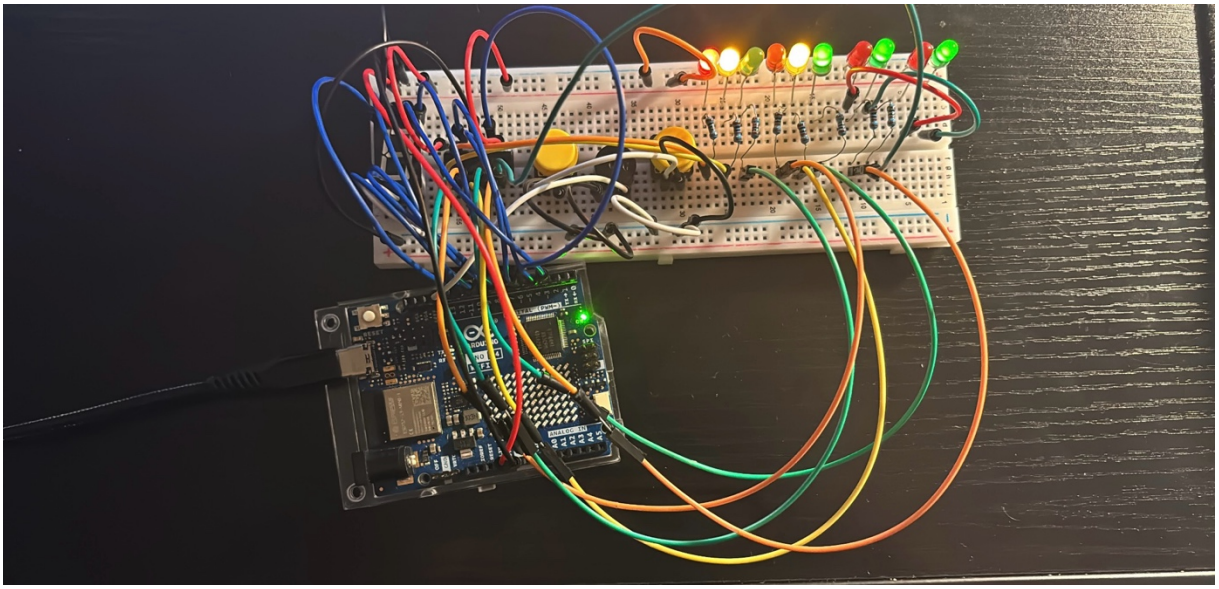


Figure 34 - Traffic Lights | Real View

Code

Self-Reflection 4

This week felt busier than ever. For Computer Science, I had three assignments: Sort, Plurality, and for the first, and only, time, Tideman. Sort actually surprised me because it didn't require any coding, it was all about analyzing and justifying answers, which was a nice change of pace. But Tideman was another story. It was one of the "harder" assignments that could give a U grade if done successfully, and since I had committed to trying it, I wanted to see it through.

Around this time, I also started a part-time job. It doesn't take up too much time, but it still required me to adjust my routine and think more carefully about how I plan my days. Some people told me to be careful not to let work interfere with my studies, and I get that, but I honestly believe that if I manage my time well, both can fit. My strategy is to make the most of the hours I spend at university, from 9:15 to 16:00, so that I can stay on top of my studies.

For Embedded Systems, we had to make a traffic light project using both the book and our teachers' instructions. It was challenging but also fun to see something functional come out of the code. Overall, it was a full week, and I can tell the assignments are getting more demanding. Still, I'm proud that I'm learning to balance my studies, work, and personal goals better each week.

Week 5

Filter – Less

In the Filter Less problem, I had to apply different image filters by manipulationg each pixel's RGB values. Specifically, I implemented four functions: ont to convert the image to grayscale, one for sepia, one to reflect the image horizontally, and one to blur it by averaging neighboring pixels. Each operation works directly on the 2D array of pixels provided by CS50's BMP structure.

Filter Less - Result:

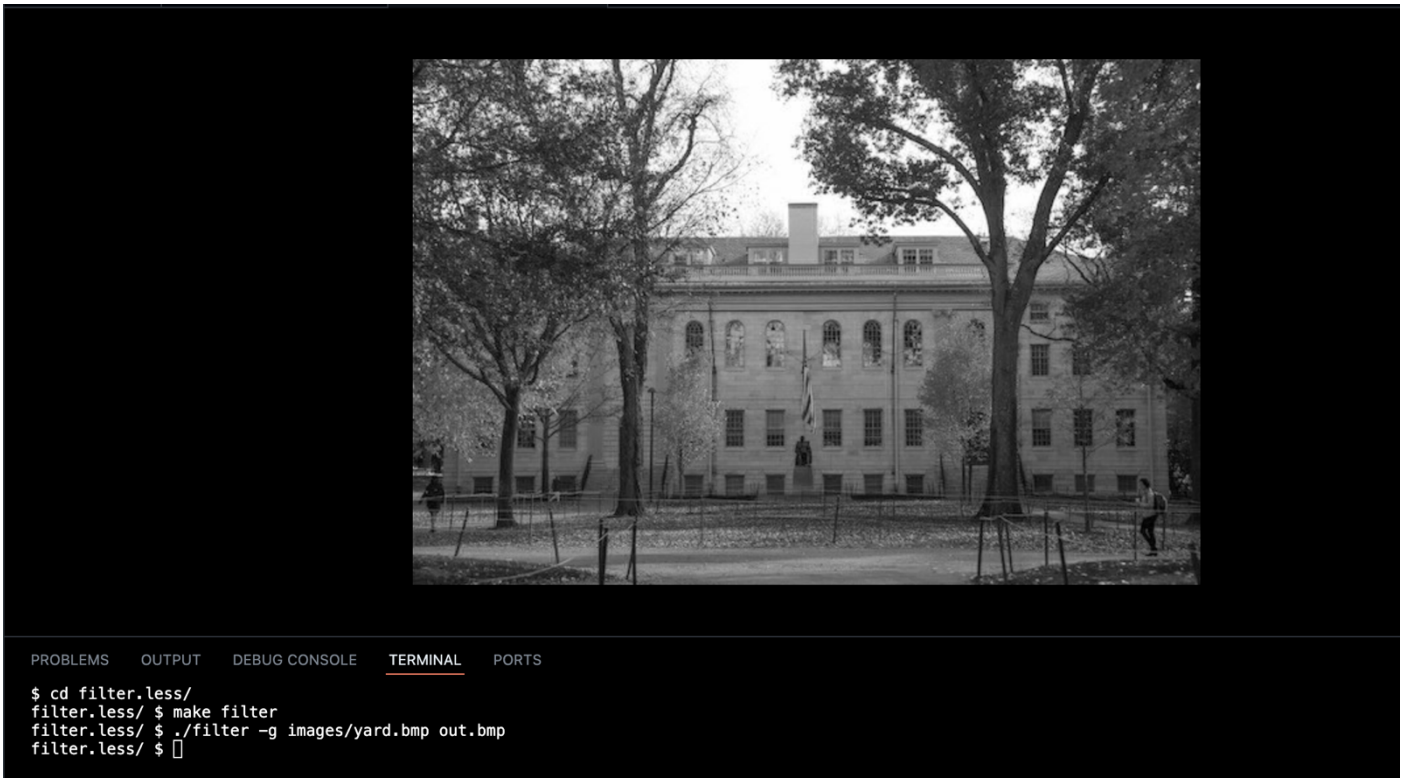


Figure 35 - Filter Less | GrayScale

Figure 36 - Filter Less | Sepia

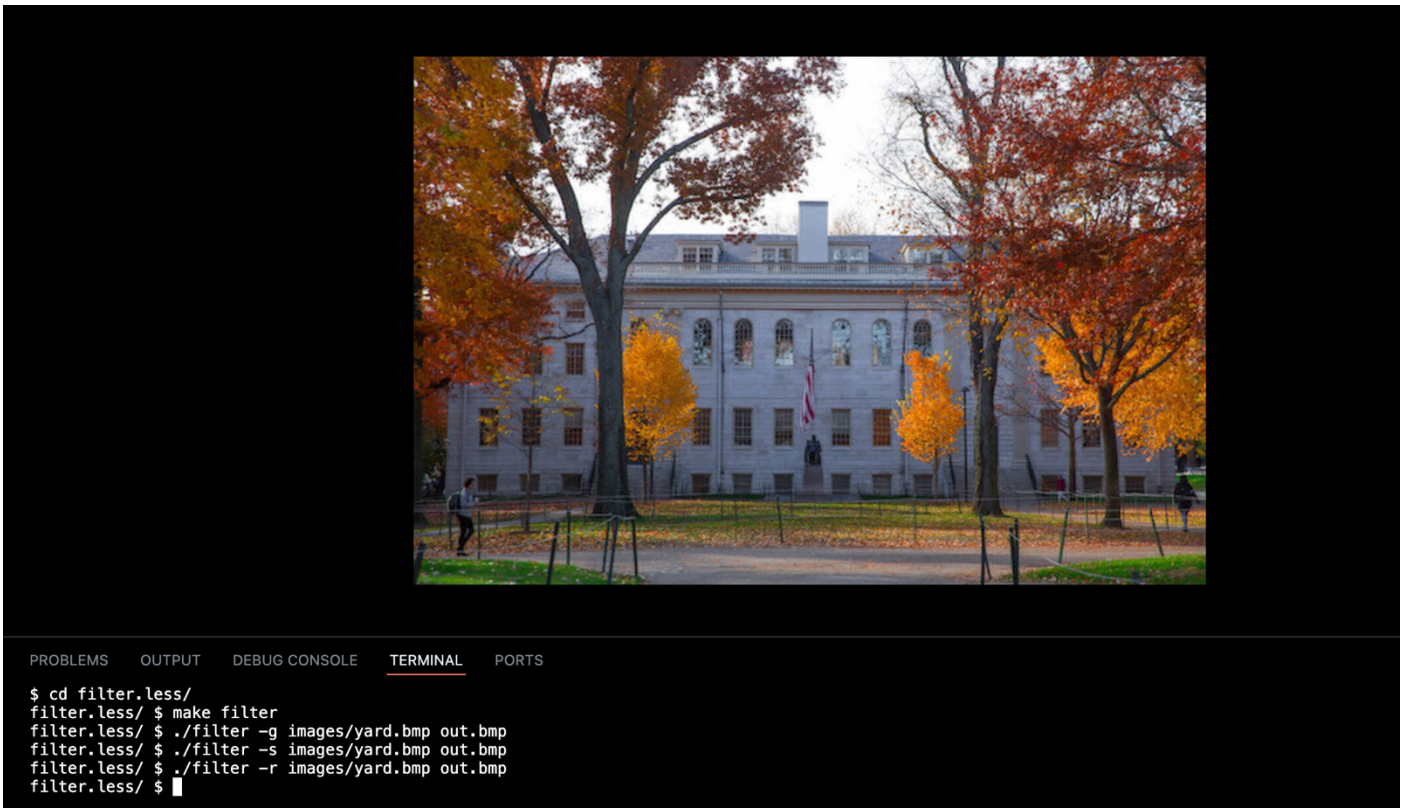


Figure 37 - Filter Less | Reflect

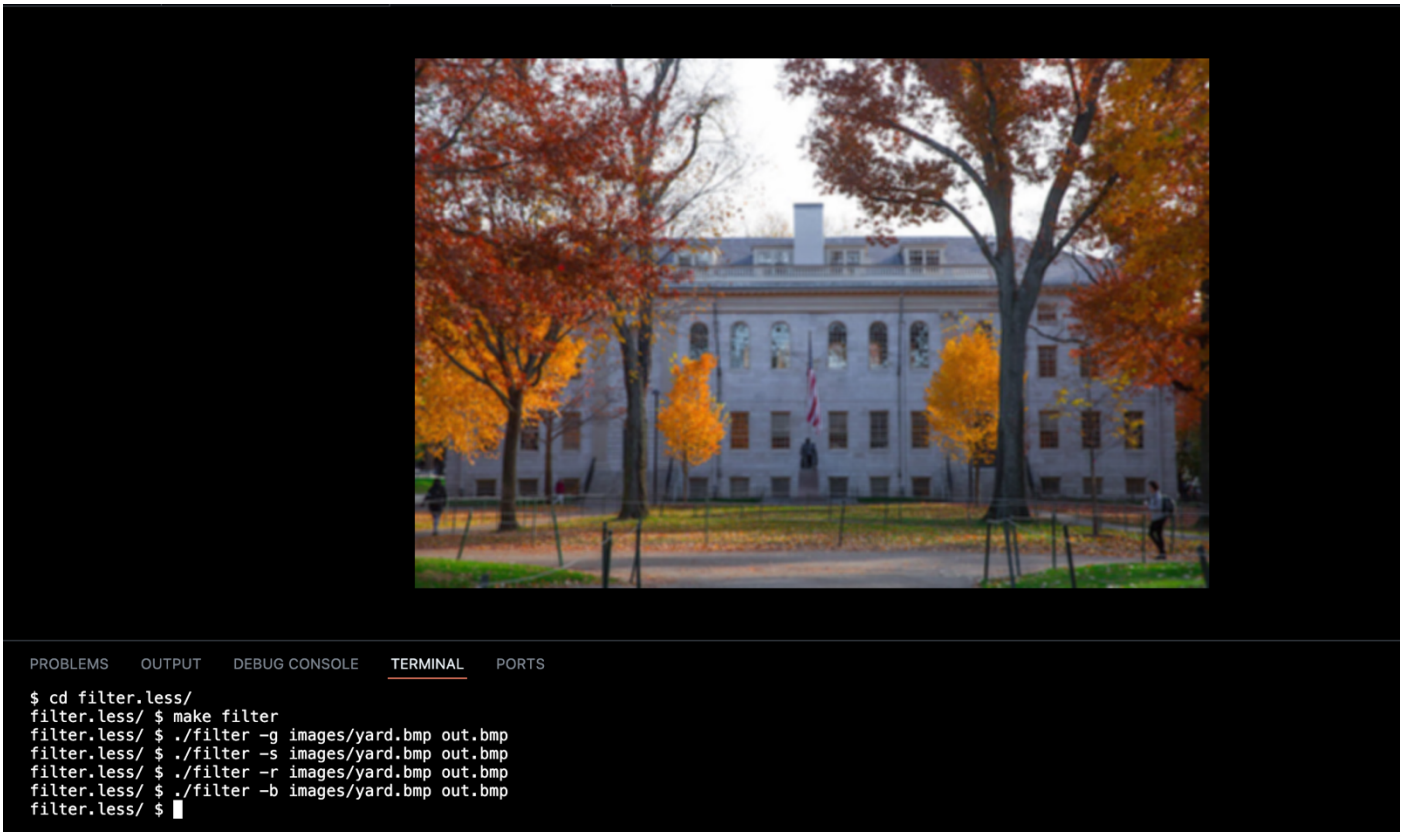
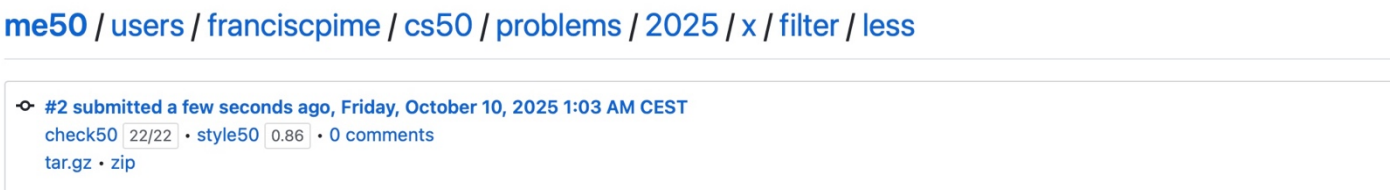


Figure 38 - Filter Less | Blur

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/filter/less

Recover

In the Recover problem, I had to forensically recover JPEG images from a raw memory card file. The program reads the card block by block (512 bytes at a time) and looks for the signature that marks the beginning of a JPEG file. Once a new JPEG header is detected, it starts writing the recovered image to a new file, naming them sequentially like “000.jpg”, “001.jpg”, etc, until all recoverable images are extracted.

Recover - Result before:

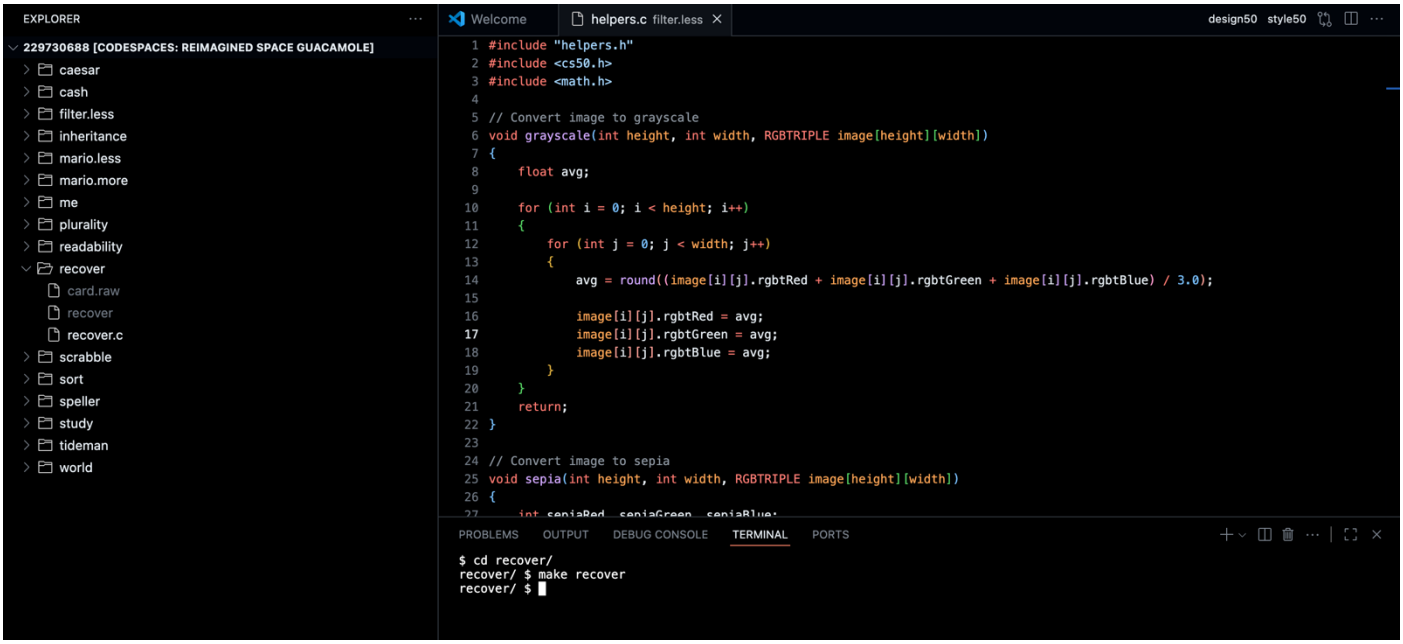


Figure 39 - Recover | Result Before

Result after:

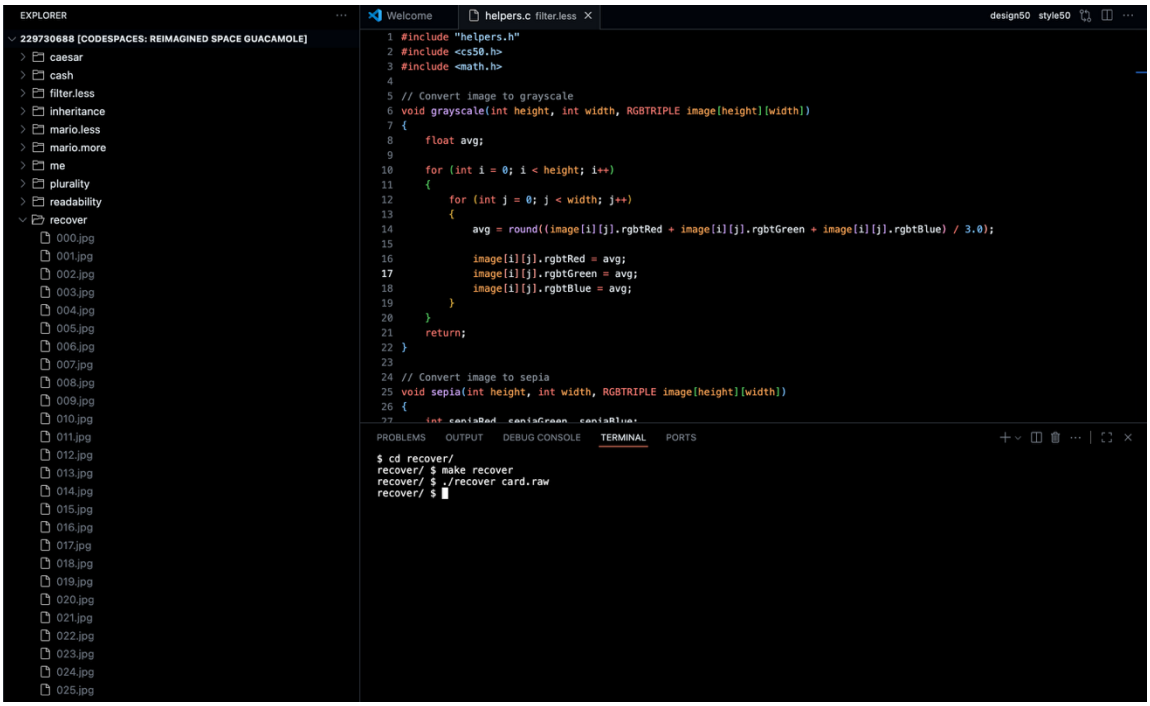


Figure 40 - Recover | Result After

Code

Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/recover

me50 / users / franciscpime / cs50 / problems / 2025 / x / recover

#1 submitted 2 days ago, Thursday, October 9, 2025 2:29 PM CEST
check50 7/7 • style50 0.91 • 0 comments
tar.gz • zip

Displaying Uptime

Displaying Uptime - Result:

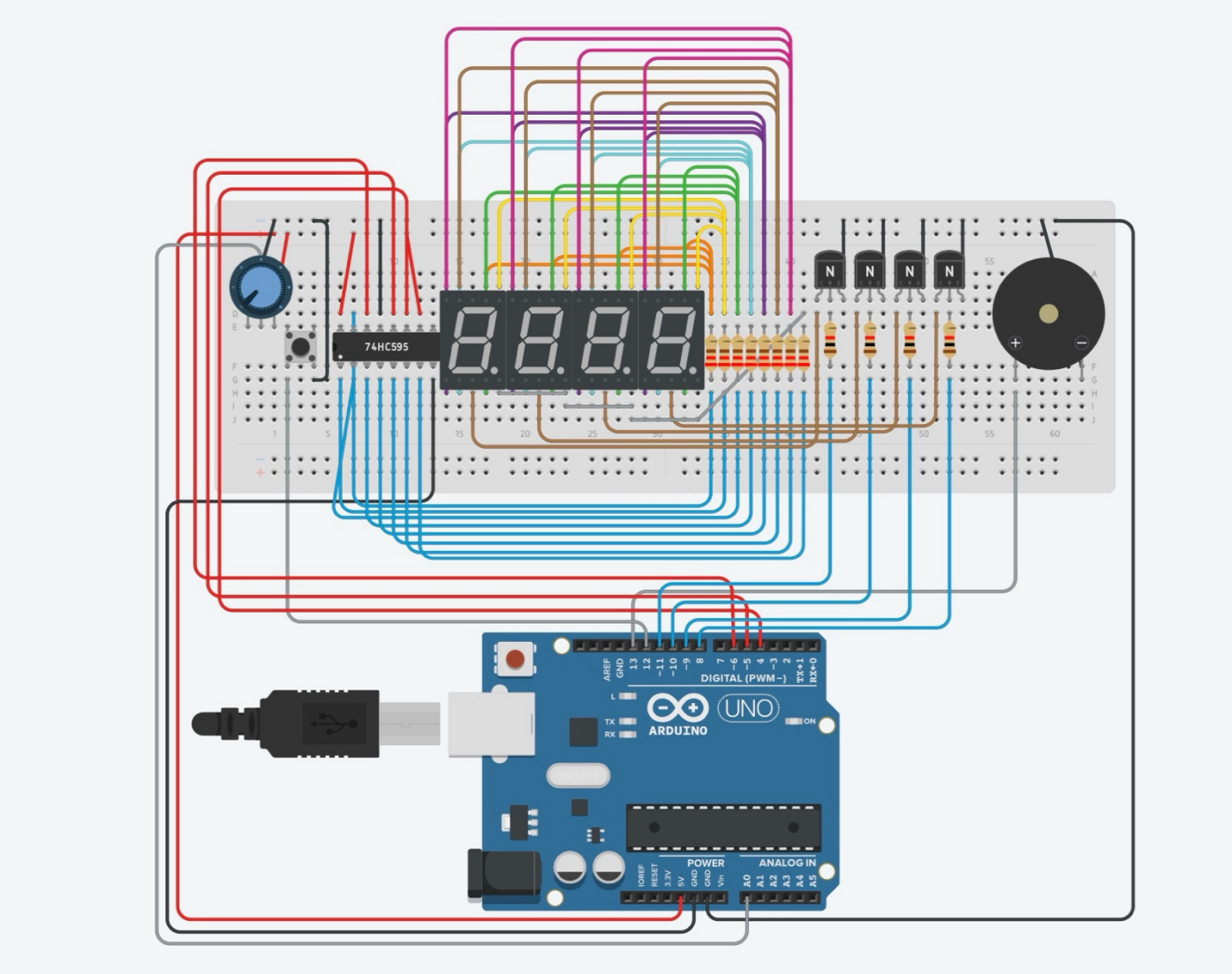


Figure 41 - Displaying Uptime | Circuit View

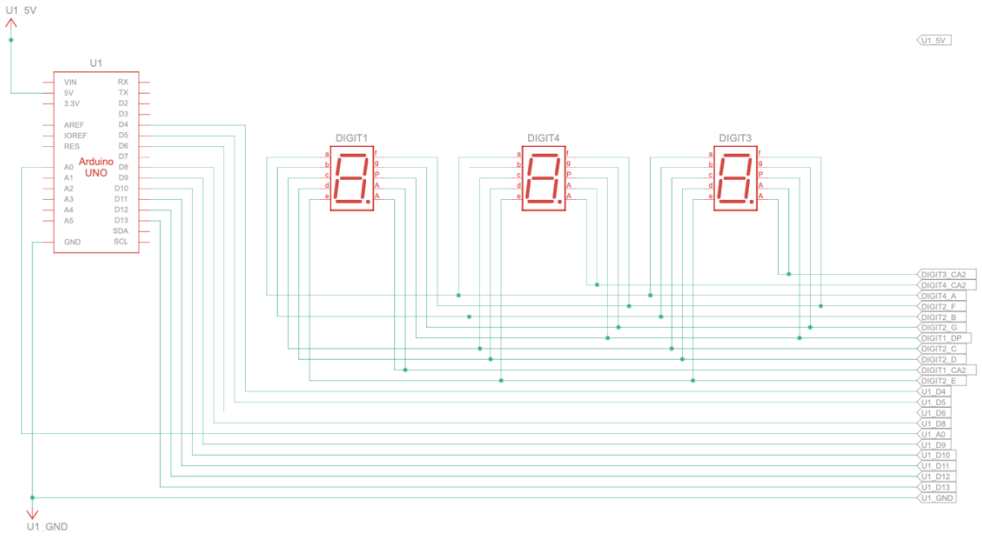


Figure 42 - Displaying Uptime | Schematic View1

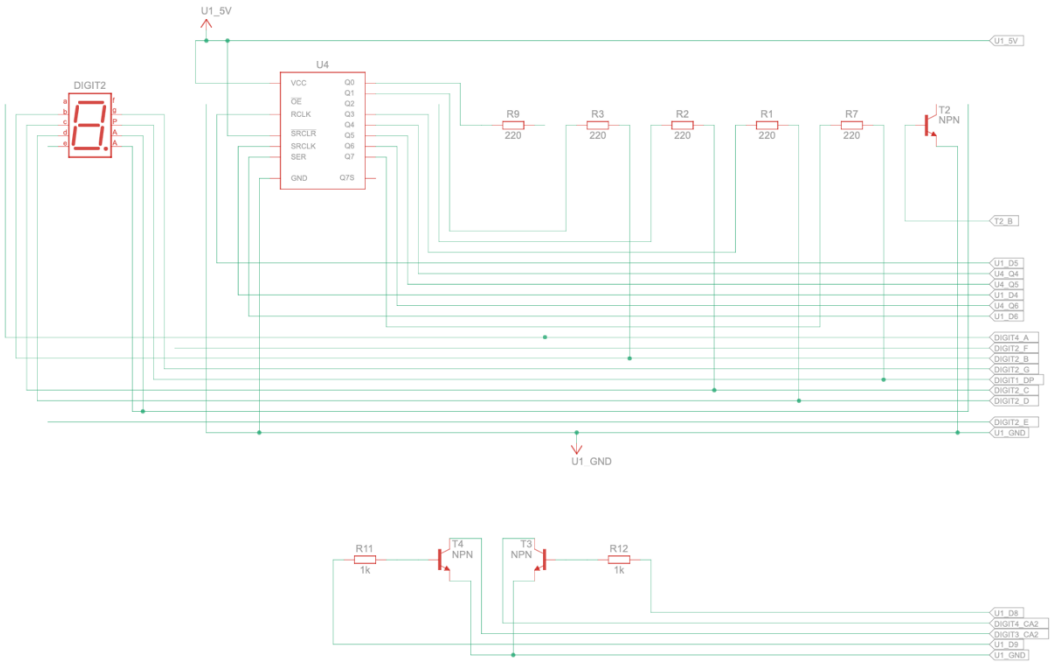


Figure 43 - Displaying Uptime | Schematic View2

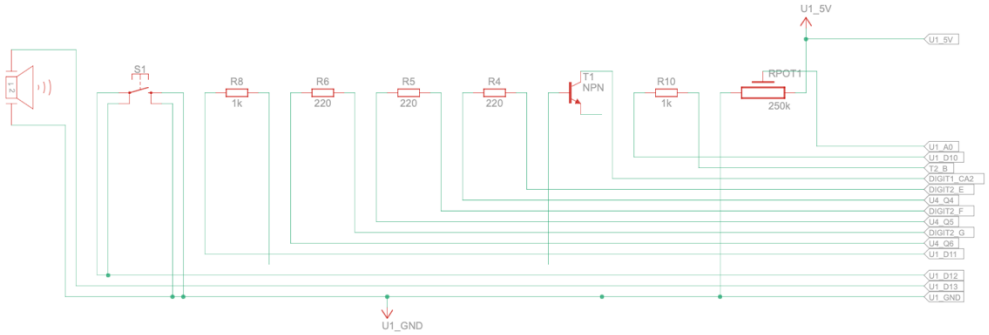


Figure 44 - Displaying Uptime | Schematic View3

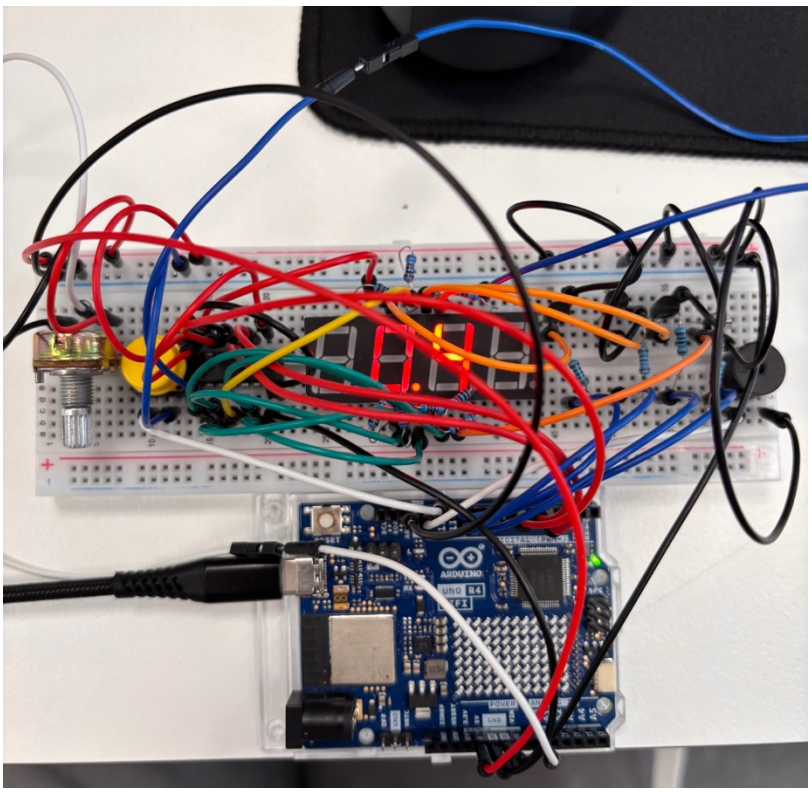


Figure 45 - Displaying Uptime | Real View

Code

Self-Reflection 5

This week was pretty intense but also full of learning. For Computer Science, I had three assignments: Scavenger Hunt, Filter Less, and Recover. The Scavenger Hunt was a short written teste where we had to write five small programs in C, which was actually a good way to see if I really understood the basics. Filter Less and Recover took a lot more time because they required me to think differently and get familiar with new ways of programming. I spent extra hours testing, debugging, and trying to understand why some parts of the code didn't work as I expected, which, honestly, happens quite a lot when learning C.

For Embedded Systems, our task was to create a timer that counted how long the Arduino stayed connected to the computer. It didn't take too long to start, but improving it took some effort. I also used a potentiometer for the first time, which was new and a bit confusing at first. Still, it felt rewarding to make everything function properly in the end.

Even though it was a demanding week, I feel like I made real progress. I'm getting more confident in solving problems on my own and more patient when things don't work right away. Learning how to keep improving my projects instead of just finishing them was probably the biggest takeaway this week.

Week 6

Git Together

This was a collaborative group project where we built a simple interactive Arduino menu over Serial communication that allows the user to choose between performing a sum or a multiplication. The program continuously listens for the user's input and then calls the corresponding function to handle the operation.

We also added a built-in LED indicator so the system visibly shows that it is active and running. In this project I was responsible for implementing the Blick() function, which makes the onboard LED repeatedly toggle on and off while the program is idle, acting as a heartbeat that confirms the board is alive and ready for user input.

Code

Inheritance

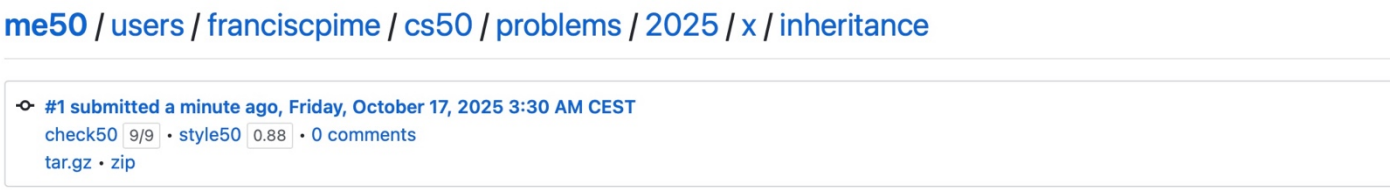
In the inheritance problem, I wrote a program that simulates the inheritance of blood types across multiple generations using dynamic memory allocation and recursion. I generate a family tree where each person has two parents and two blood type alleles, and those alleles are either inherited from the parents or randomly generated if they belong to the oldest generation.

After building the full ancestry, I neatly print out the entire family tree with indentation to show generational structure, and at the end, I free all the allocated memory to avoid leaks.

Inheritance - Result:

Figure 46 - Inheritance | Result

Code



Access link: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/inheritance

Jumbo Clock

Jumbo Clock - Result:

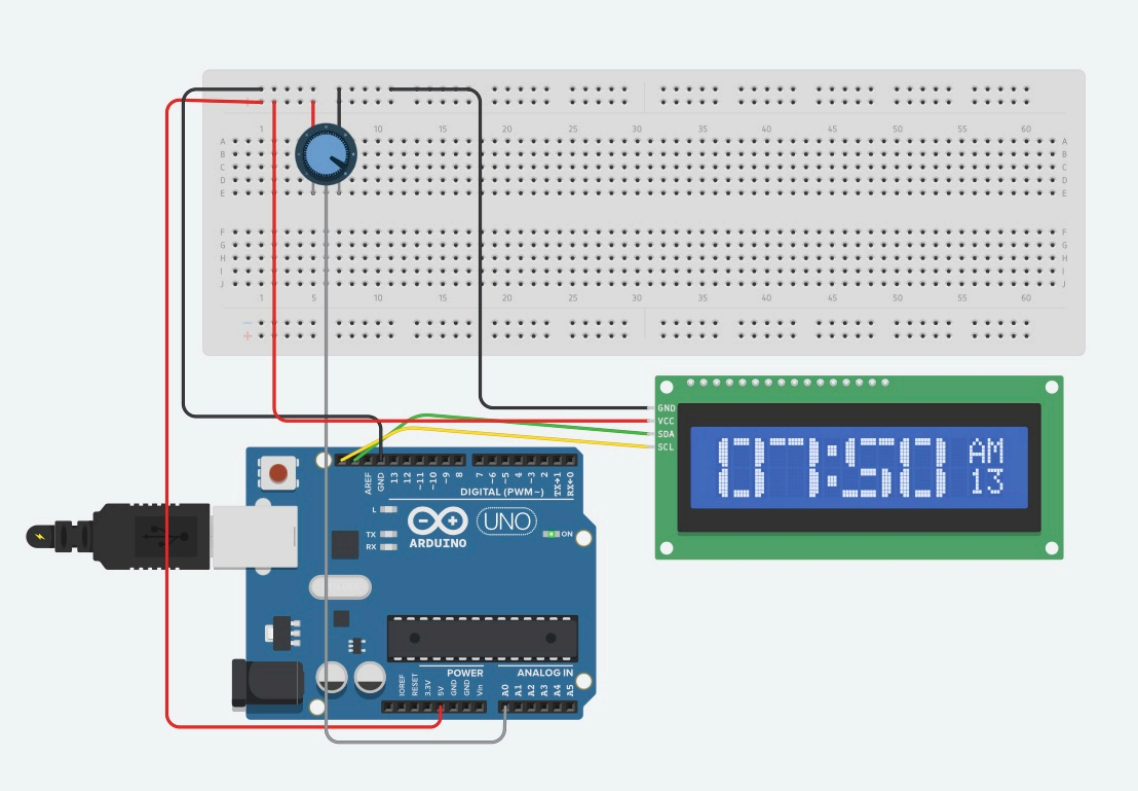


Figure 47 - Jumbo Clock | Circuit View

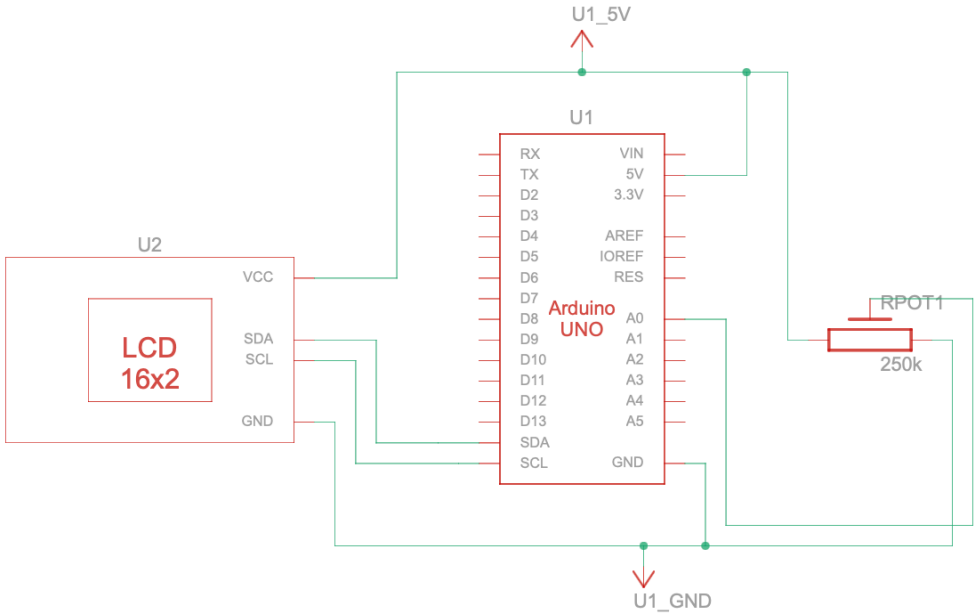


Figure 48 - Jumbo Clock | Schematic View

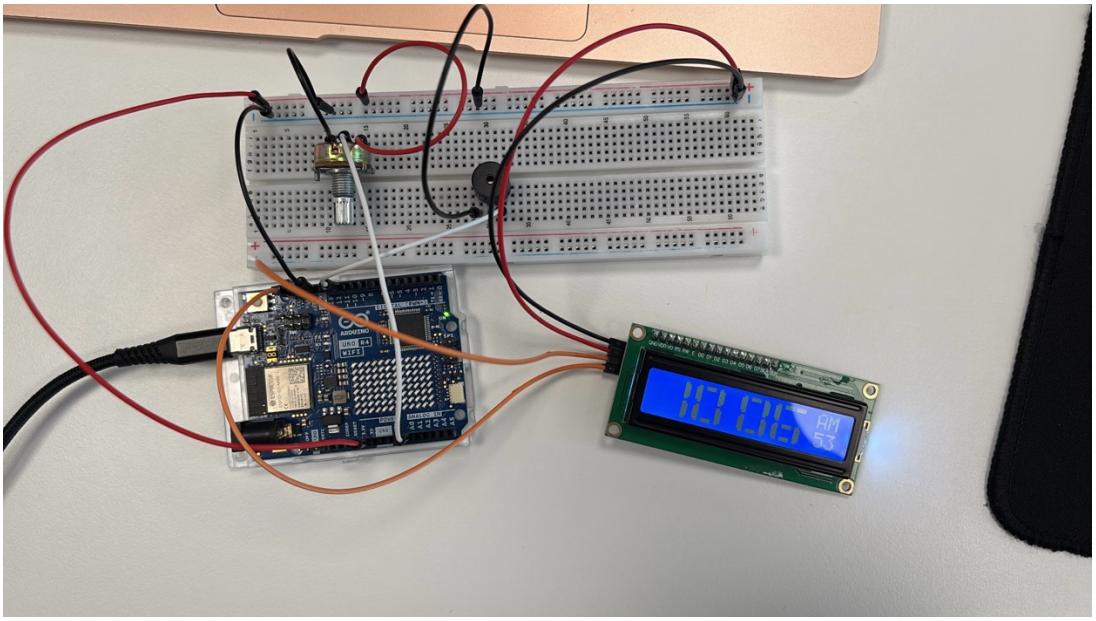


Figure 49 - Jumbo Clock | Real View

Code

Access link: https://www.tinkercad.com/things/bniBAbSxQ3O-jumbo-clock?sharecode=eVpLH83nNGMOlNwUhSlFMvyOt_bfBTGZyG76lV9j5TU

Self-Reflection 6

This week we went back to having two assignments: Git Together and Inheritance. The first one was a group project where the teams were made randomly by the teacher. At first, it was a bit strange working with people I didn't work before, but it turned out to be a good experience. We started by learning how to use GitHub properly, especially for those who weren't very familiar with it, and then we divided the four tasks among the group members. At the end, we combined everything to get the final program. It was interesting to see how different pieces of code came together into one project.

The second assignment was... let's say, quite an experience! Most of it was basically following the given instructions, so the hardest part for me wasn't writing the code, but actually understanding how everything worked. Still, it helped me see how powerful and organized programming can be when we use concepts like inheritance correctly.

For Embedded Systems, we built a real-time clock. I didn't struggle much with the time counter part since I had already done something similar before, but using special characters for the digits was tricky at first, the format kept confusing me. In the end, though, it worked fine. Every time I show my projects to the teachers, I get helpful tips that make me realize how much I'm improving, and that's exactly the goal.

Week 7

Speller

In the Speller problem, I implemented a hash table-based speller checker that can load thousands of words into memory and very efficiently check if a given word exists in the dictionary. I used an array of 26 linked lists (one per letter A-Z), where each node stores a wpord.

The program supports four core operations: loading the dictionaty file into memory, checking if a word exists case-insensitively, returning the total word count, and properly freeing all allocated memory at the end.

Speller - Result:

Figure 50 - Speller | Result

Code

Weather Station

Weather Station - Result:

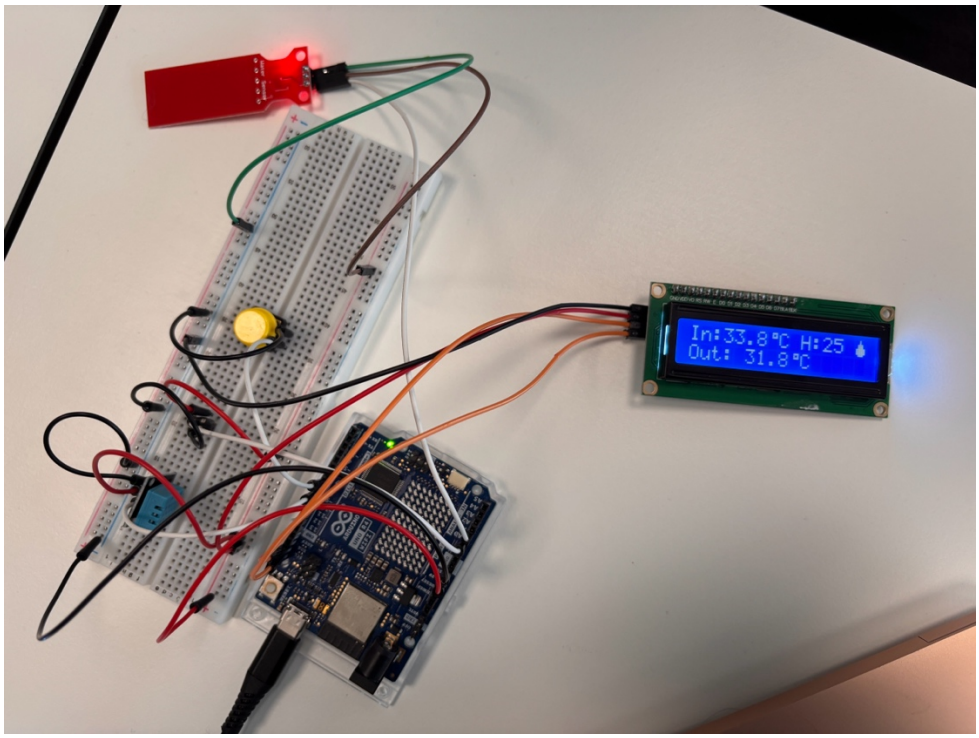


Figure 51 - Weather Station | Real View | Main

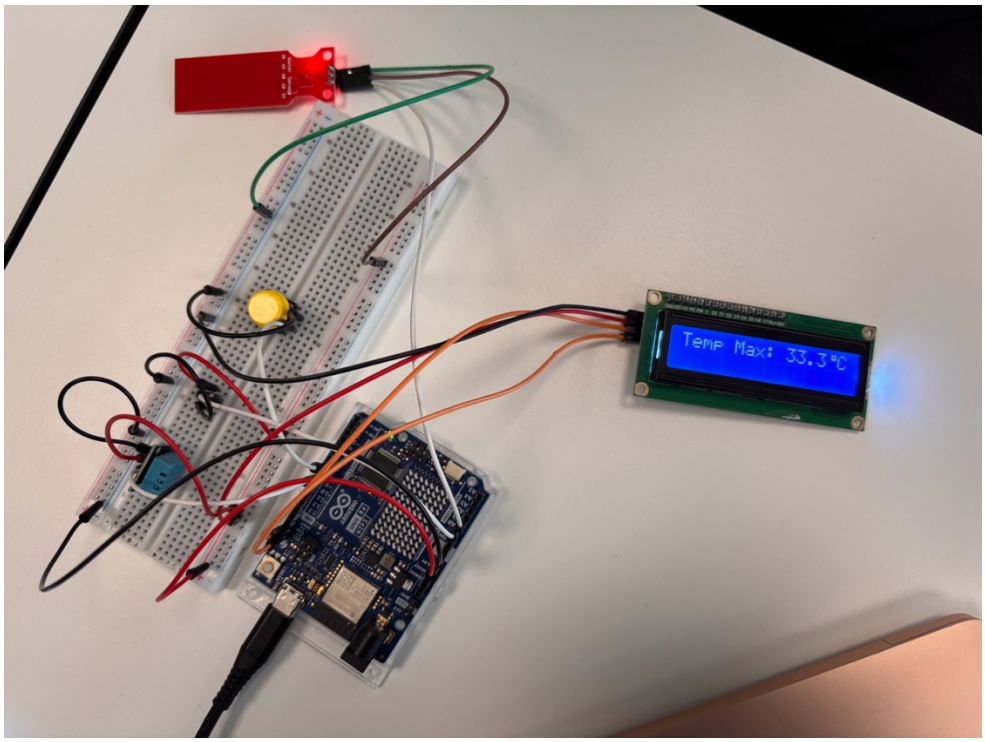


Figure 52 - Weather Station | Real View | MaxT

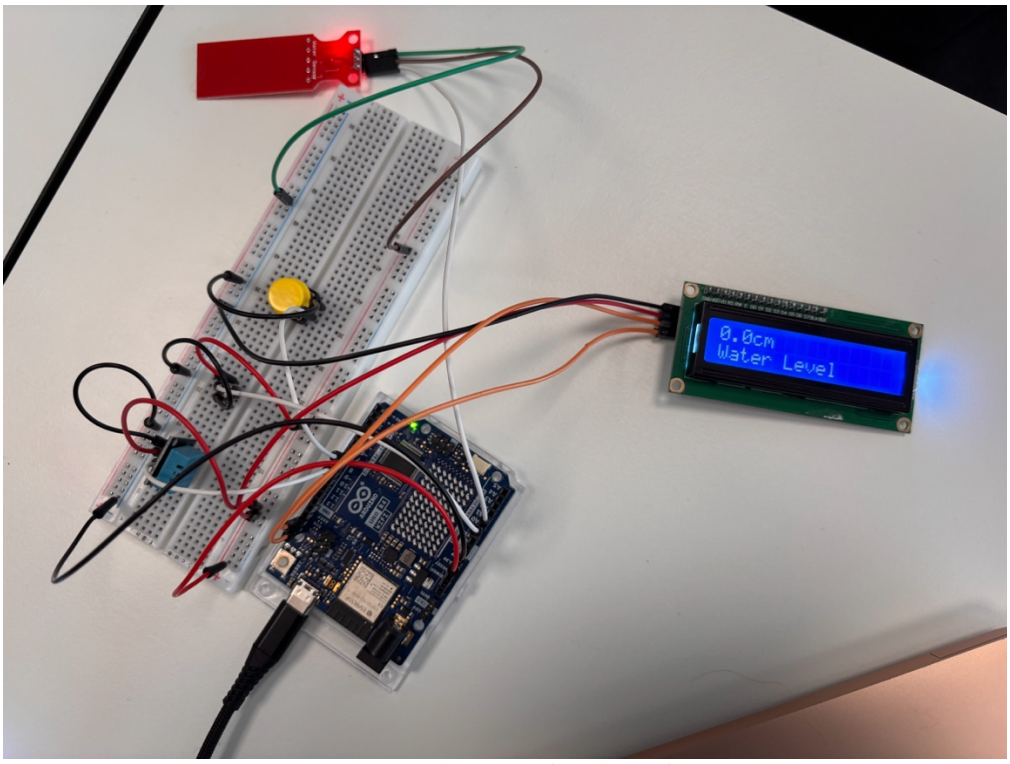


Figure 53 - Weather Station | Real View | Water

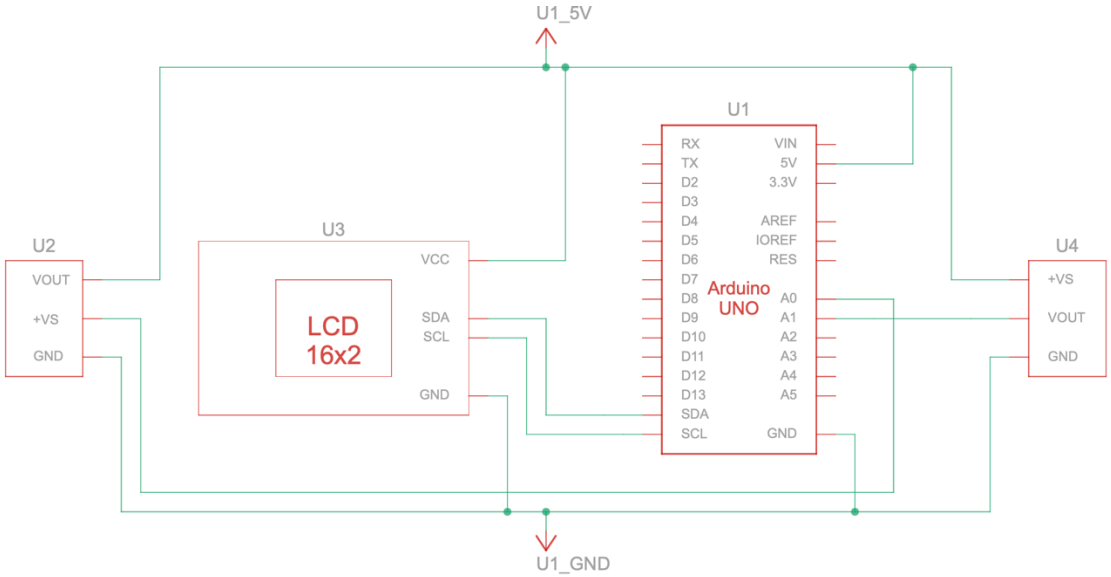


Figure 54 - Weather Station | Schematic View

Code

me50 / users / franciscpime / cs50 / problems / 2025 / x / speller

#1 submitted a few seconds ago, Sunday, October 26, 2025 11:08 PM CET
check50 10/10 • style50 0.97 • 0 comments
tar.gz • zip

Access: https://submit.cs50.io/users/franciscpime/cs50/problems/2025/x/speller

Self-Reflection 7

This week was a bit calmer compared to the previous ones, which was actually nice. I only had one assignment for each subject: Speller for Computer Science and Weather Station for Embedded Systems. Having fewer tasks gave me a bit more time to breathe and focus on understanding the details of each project instead of just rushing to finish everything.

For Computer Science, Speller was a challenging but interesting task. It tested how well I could work with data structures and logic to make the program check words correctly. It took some patience to get everything working, but when I finally saw the program running as expected, it felt great.

I think this assignment helped me realize how far I've come since the first weeks, because I can read and write C code with more confidence now.

For Embedded Systems, the Weather Station project was quite fun because it elt more "real world". It involved collecting and displaying data, which made it different from the usual logic-based projects. I enjoyed seeing the Arduino display the information and understanding how sensors can be used in practical applications.

Overall, this week showed me that even when things slow down a bit, there is still a lot to learn. It also reminded me that progress doesn't always come rom doing more, sometimes it comes from doing thins better and understanding them more deeply.

Codes

Hello, It's Me – Code

```c
#include <stdio.h>
#include <cs50.h>
int main(void)
{
    string firstname = get_string("What's your name? ");
    printf("hello, %s\n", firstname);
    return 0;
}
```

Code – Result

Mario Less - Code

```c
#include <cs50.h>
#include <stdio.h>
int main(void)
{
    int height;
    // ask me for the pyramid height (between 1 and 8)
    do
    {
        height = get_int("Write a number (1 to 8): ");
        if (height < 1 || height > 8)
        {
            // Error message if input is out of range
            printf("Number out of range. Try again.\n");
        }
    } while (height < 1 ||height > 8);           // Repeat until valid input is given
    // Loop through each row of the pyramid
    for (int row = 1; row <= height; row++)
    {
        // Print spaces before the hashes to align the pyramid
        for (int space = 1; space <= height - row; space++)
        {
            printf(" ");
        }
        // Print the hashes for the current row
        for (int hash = 1; hash <= row; hash++)
        {
            printf("#");
        }
        // Move to the next line after finishing one row
        printf("\n");
    }
    return 0;
}
```

Code – Result

Mario More - Code

```c
#include <cs50.h>
#include <stdio.h>
int main (void)
{
    int height;
    // Asks for the pyramid height (between 1 and 8)
    do {
        height = get_int ("Height: ");
        if (height < 1 || height > 8)
        {
            // Error message if input is out of range
            printf ("Number out of range. Try again.\n");
        }
    } while (height < 1 || height > 8);         // Repeat until valid input is given
    // Draw 2 pyramids
    for (int row = 1; row <= height; row++)
    {
        // Print spaces before the hashes to align the pyramid
        for (int space = 1; space <= height - row; space++)
        {
            printf (" ");
        }
        // Print the hashes for the current row
        for (int hash = 1; hash <= row; hash++)
        {
            printf ("#");
        }
        // Space between pyramids
        printf ("  ");
        // Second pyramid
        for (int hash = 1; hash <= row; hash++)
        {
            printf ("#");
        }
        // New line
        printf ("\n");
    }
}
```

Code – Result

Cash - Code

```c
#include <cs50.h>
#include <stdio.h>
int main(void)
{
    int cents;
    // Asks for the amount of change (in cents), making sure it is not negative
    do {
        cents = get_int ("Change owed: ");
    }
    while (cents < 0);
    int coins = 0;                          // Counter for the number of coins used
    int coin = 25;                          // Start with the largest coin
    // Continue until all change has been given
    while (cents > 0)
    {
        // If the remaining changer is enough for the current coin
        if (cents >= coin)
        {
            cents -= coin;       // Substract the coin value from the remaining change
            coins++;             // Increase the coin counter
        }
        // If not, move to the next smaller coin
        else if(coin == 25) {
            coin = 10;                      // Switch from 25 to 10
        }
        else if (coin == 10)
        {
            coin = 5;                       // Switch from 10 to 5
        }
        else if(coin == 5)
        {
            coin = 1;                       // Switch from 5 to 1
        }
    }
    printf("%i\n", coins);
    return 0;
}
```

Code – Result

Mario UNO - Code

```
//-----------------------------------------------------------------------
// Symmetric Pyramid + LED Matrix + Buzzer
// =======================================
//
// Draws two mirrored pyramids diagonally on a 12x8 LED matrix.
// Each level triggers a buzzer tone. The user inputs pyramid
// height (1-8) via Serial Monitor. The pyramids always fit
// the display.
//
// Francisco Pimentel
// 16 / Oct / 2025
// version 2.3.6
//-----------------------------------------------------------------------
#include <ArduinoGraphics.h>        // Graphics support for LED matrix
#include <Arduino_LED_Matrix.h>     // LED matrix control library
#define ON HIGH                     // LED ON state
#define OFF LOW                     // LED OFF state
ArduinoLEDMatrix matrix;            // Create LED matrix object
const int buzzerPin = 11;           // Buzzer pin
int n = 0;                          // Pyramid height
void setup() {
  Serial.begin(9600);               // Initialize serial communication
  matrix.begin();                    // Initialize LED matrix
  matrix.clear();                    // Clear matrix display
  pinMode(buzzerPin, OUTPUT);        // Set buzzer pin as output
  digitalWrite(buzzerPin, OFF);      // Ensure buzzer is initially off
  Serial.println();
  Serial.println("Write a number (1 to 8): ");  // Prompt for pyramid size
}
void loop() {
  if (Serial.available() > 0) {      // Check if user input is available
    n = Serial.parseInt();            // Read integer from Serial
    // Validate input range (1 to 8)
    if (n < 1 || n > 8) {
      Serial.println("Number out of range. Try again.");
      Serial.println("Write a number (1 to 8): ");
      n = 0;                          // Reset invalid input
      return;                         // Wait for new input
    }
    Serial.read();                    // Clear remaining input from buffer
    Serial.println();
    matrix.clear();                   // Clear LED matrix before drawing
    // Print pyramid to Serial Monitor
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= n - i; j++) {  // Leading spaces for alignment
        Serial.print(" ");
      }
      for (int j = 1; j <= i; j++) {      // Left pyramid hashes
        Serial.print("#");
      }
      Serial.print("  ");                  // Space between pyramids
      for (int j = 1; j <= i; j++) {      // Right pyramid hashes
        Serial.print("#");
      }
      Serial.println();                   // Move to next row
    }
```

```
    Serial.println();                       // Extra blank line
    uint8_t frame[8][12] = {0};            // 8x12 frame initialized to all zeros
    // Draw pyramids on LED matrix with buzzer sound
    for (int level = 0; level < n; level++) {
      // Pyramid 1 — upper-left corner
      for (int row = 0; row <= level && row < 8; row++) {  // Row of LED matrix
        for (int col = 0; col <= level - row && col < 12; col++) {  // Column of LED matri
x
          frame[row][col] = 1;                    // Turn on LED
        }
      }
      // Pyramid 2 — lower-right corner (mirrored)
      for (int row = 0; row <= level && row < 8; row++) {
        for (int col = 0; col <= level - row && col < 12; col++) {
          int x = 11 - col;                      // Mirror column horizontally
          int y = 7 - row;                       // Mirror row vertically
          frame[y][x] = 1;                       // Turn on mirrored LED
        }
      }
      matrix.renderBitmap(frame, 12, 8);       // Render pyramids on LED matrix
      int frequency = 300 + (level * 150);     // Calculate buzzer frequency
      tone(buzzerPin, frequency, 120);         // Play tone for 120 ms
      delay(250);                              // Wait 250 ms before next level
    }
    noTone(buzzerPin);                         // Stop buzzer after pyramid
    delay(1000);                               // Wait 1 second before next input
    Serial.println("Write a number (1 to 8): ");  // Prompt again
  }
}
```

Code – Result

Scrabble - Code

```c
#include <cs50.h>
#include <stdio.h>
#include <ctype.h>
// Points for letters A to Z
int POINTS[26] =
{
        1, 3, 3, 2, 1, 4, 2, 4,     // Points for A-H
        1, 8, 5, 1, 3, 1, 1, 3,     // Points for I-P
        10, 1, 1, 1, 1, 4, 4, 8,    // Points for Q-X
        4, 10                       // Points for Y and Z
};
int compute_score (string word);
int main (void)
{
    // Ask Player 1 for a word
    string word1 = get_string ("Player 1: ");
    // Ask Player 2 for a word
    string word2 = get_string ("Player 2: ");
    // Compute scores for both words
    int score1 = compute_score (word1);          // Prompt user and store result in word1
    int score2 = compute_score (word2);         // Prompt user and store result in word2
    // Compare the scores and print the result
    if (score1 > score2)
    {
        printf ("Player 1 wins!\n");
    }
    else if (score1 < score2)
    {
        printf ("Player 2 wins!\n");
    }
    else
    {
        printf ("Tie!\n");
    }
}
int compute_score (string word)
{
    int score = 0;
    // Loop over characters until the null terminator
    for (int character_index = 0; word[character_index] != '\0'; character_index++)
    {
        // Get the current character from the string
        char character = word[character_index];
        // If the character is an lowercase letter
        if (character >= 'a' && character <= 'z')
        {
            // Lowercase becomes to uppercase
            character = toupper(word[character_index]);
        }
        // If the character is an uppercase letter
        if (character >= 'A' && character <= 'Z')
        {
            int index = character - 'A';       // Covert 'A'...'Z' to index 0...25
            score += POINTS[index];
        }
```

```
        // If the character isn't a letter (punctuation, space...), ignore it
    }
    return score;
}
```

Code – Result

Readability - Code

```
#include <cs50.h>
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
bool only_digits (string s);
char rotate (char c, int k);
int main (int argc, string argv[])
{
    // If the program does not receive exactly 2 arguments >> ./caesar and the number
    if (argc != 2)
    {
        printf("Usage: ./caesar key\n");                    // It let us know how to use
        return 1;
    }
    if (!only_digits (argv[1])) // Se o argumento não for só numeros
    {
        printf("Usage: ./caesar key\n");                   // It let us know how to use
        return 1;
    }
    long k_long = strtol(argv[1], NULL, 10); // Converts the string argv[1] to a long int
    // Transform to int and get only the remainder of 26 (so as not to go out of the
alphabet)
int k = (int) (k_long % 26);     string plaintext = get_string ("plaintext: "); // Ask to
write the text you want to encrypt
    printf("ciphertext: "); // Shows that it will now print the encrypted version
    for (int letter_index = 0, length = strlen(plaintext); letter_index < length;
letter_index++) // Go through each letter of the text
    {
        // Applies the rotate function and prints each encrypted letter
        printf("%c", rotate(plaintext[letter_index], k));
    }
    printf("\n");
    return 0;
}
// Function that checks if a string has only numbers
bool only_digits(string s)
{
    for (int character_index = 0, length = strlen(s); character_index < length;
character_index++)              // Iterate through each character of the string
    {
        if (!isdigit((unsigned char) s[character_index]))                 // If any is not
number
        {
            return false;                                       // So what I wrote wasn't just
numbers
```

```
        }
    }
    return true;                                        // All I wrote were numbers
}
// Function that applies the exchange of a normal letter for the encrypted one
char rotate(char c, int k)
{
    if (isupper((unsigned char) c)) // If it is uppercase
    {
        // Count the position of the letter in the alphabet, add k, make sure it's within
26, and
        // turn it back into a letter.
        return (char) ((c - 'A' + k) % 26 + 'A');
    }
    if (islower((unsigned char) c))               // If it is lowercase
    {
        return (char) ((c - 'a' + k) % 26 + 'a'); // Same process as in uppercase
    }
    return c; // If it is not a letter (" " or ?), it doesn't change anything
}
```

Code – Result

Caesar – Code

```
#include <cs50.h>
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
bool only_digits (string s);
char rotate (char c, int k);
int main (int argc, string argv[])
{
    // If the program does not receive exactly 2 arguments >> ./caesar and the number
    if (argc != 2)
    {
        printf("Usage: ./caesar key\n");                  // It let us know how to use
        return 1;
    }
    if (!only_digits (argv[1])) // Se o argumento não for só numeros
    {
        printf("Usage: ./caesar key\n");                 // It let us know how to use
        return 1;
    }
    long k_long = strtol(argv[1], NULL, 10); // Converts the string argv[1] to a long int
    // Transform to int and get only the remainder of 26 (so as not to go out of the
alphabet)
int k = (int) (k_long % 26);     string plaintext = get_string ("plaintext: "); // Ask to
write the text you want to encrypt
    printf("ciphertext: "); // Shows that it will now print the encrypted version
    for (int letter_index = 0, length = strlen(plaintext); letter_index < length;
letter_index++) // Go through each letter of the text
    {
        // Applies the rotate function and prints each encrypted letter
```

```
        printf("%c", rotate(plaintext[letter_index], k));
    }
    printf("\n");
    return 0;
}
// Function that checks if a string has only numbers
bool only_digits(string s)
{
    for (int character_index = 0, length = strlen(s); character_index < length;
character_index++)              // Iterate through each character of the string
    {
        if (!isdigit((unsigned char) s[character_index]))                // If any is not
number
        {
            return false;                                       // So what I wrote wasn't just
numbers
        }
    }
    return true;                                                // All I wrote were numbers
}
// Function that applies the exchange of a normal letter for the encrypted one
char rotate(char c, int k)
{
    if (isupper((unsigned char) c)) // If it is uppercase
    {
        // Count the position of the letter in the alphabet, add k, make sure it's within
26, and
        // turn it back into a letter.
        return (char) ((c - 'A' + k) % 26 + 'A');
    }
    if (islower((unsigned char) c))               // If it is lowercase
    {
        return (char) ((c - 'a' + k) % 26 + 'a'); // Same process as in uppercase
    }
    return c; // If it is not a letter (" " or ?), it doesn't change anything
}
```

Code – Result

Morse Code - Code

```
//---------------------------------------------------------------------
// Morse Code Translator with LED Matrix Display
// =============================================
//
// Converts a user-input message from Serial Monitor into Morse code.
// Displays each dot/dash on an 8x12 LED matrix and emits corresponding
// beeps via a buzzer. Supports letters, numbers, and some symbols.
//
// Features:
// - LED on pin 12 lights with each Morse signal.
// - Dot/dash timing follows Morse code standard.
// - Real-time conversion of sentence to Morse code.
//
// Francisco Pimentel
// 20 / Oct / 2025
// version 2.4.1
//---------------------------------------------------------------------
#include <ArduinoGraphics.h>            // Graphics library for LED matrix
#include <Arduino_LED_Matrix.h>         // LED matrix library
ArduinoLEDMatrix matrix;                // Create LED matrix object
const int ledPin    = 12;               // LED pin to indicate Morse signal
const int buzzerPin = 8;                // Buzzer pin for Morse tones
const int unit      = 50;               // Base Morse time unit (ms)
// Morse timing constants
const int dotTime     = unit;           // Dot duration = 1 unit
const int dashTime    = unit * 3;       // Dash duration = 3 units
const int letterSpace = unit * 3;       // Space between letters = 3 units
const int wordSpace   = unit * 7;       // Space between words = 7 units
// Function declarations
void showDot();
void showDash();
void playMorseChar(char symbol);
// Morse code tables for letters A-Z
const char letters[26][7] = {
  ".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
  ".---", "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.",
  "...", "-", "..-", "...-", ".--", "-..-", "-.--", "--.."
};
// Morse code tables for numbers 0-9
const char numbers[10][6] = {
  "-----", ".----", "..---", "...--", "....-",
  ".....", "-....", "--...", "---..", "----."
};
// Morse code for special characters (.,!?-/@=$&)
const char specials[10][10] = {
  ".-.-.-", "--..--", "..--..", "-.-.--", "-....-",
  "-..-.", ".--.-.", "-...-", "...-..-", ".-..."
};
// LED matrix bitmaps for dot and dash
uint8_t dotBitmap[8][12] = {
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,1,1,1,0,0,0,0,0,0},
```

```
  {0,0,0,1,1,1,0,0,0,0,0,0},
  {0,0,0,1,1,1,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0}
};
uint8_t dashBitmap[8][12] = {
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0},
  {0,1,1,1,1,1,1,1,1,1,0,0},
  {0,1,1,1,1,1,1,1,1,1,0,0},
  {0,1,1,1,1,1,1,1,1,1,0,0},
  {0,1,1,1,1,1,1,1,1,1,0,0},
  {0,0,0,0,0,0,0,0,0,0,0,0}
};
void setup() {
  pinMode(ledPin, OUTPUT);               // LED pin as output
  pinMode(buzzerPin, OUTPUT);            // Buzzer pin as output
  Serial.begin(9600);                    // Initialize Serial communication
  matrix.begin();                        // Initialize LED matrix
  Serial.println("Enter a sentence: ");  // Prompt user for input
  delay(2000);                           // Wait 2 seconds before reading input
}
void loop() {
  if (Serial.available() > 0) {                          // Check if user entered data
    String userMessage = Serial.readStringUntil('\n');   // Read entire line
    userMessage.toUpperCase();                           // Convert to uppercase
    char morseSymbol[8];                                 // Temporary storage for Morse sy
mbols
    // Iterate through each character of the message
    for (int i = 0; i < userMessage.length(); i++) {
      char currentChar = userMessage[i];
      morseSymbol[0] = '\0';                             // Reset Morse symbol
      // ---- Character conversion ----
      if (currentChar >= 'A' && currentChar <= 'Z') {
        strcpy(morseSymbol, letters[currentChar - 'A']);  // Letter Morse code
      } else if (currentChar >= '0' && currentChar <= '9') {
        strcpy(morseSymbol, numbers[currentChar - '0']);  // Number Morse code
      } else {
        switch (currentChar) {                              // Special characters
          case '.': strcpy(morseSymbol, specials[0]); break;
          case ',': strcpy(morseSymbol, specials[1]); break;
          case '?': strcpy(morseSymbol, specials[2]); break;
          case '!': strcpy(morseSymbol, specials[3]); break;
          case '-': strcpy(morseSymbol, specials[4]); break;
          case '/': strcpy(morseSymbol, specials[5]); break;
          case '@': strcpy(morseSymbol, specials[6]); break;
          case '=': strcpy(morseSymbol, specials[7]); break;
          case '$': strcpy(morseSymbol, specials[8]); break;
          case '&': strcpy(morseSymbol, specials[9]); break;
          case ' ': delay(wordSpace); break;                // Space between words
        }
      }
      // ---- Playback and visual output ----
      if (morseSymbol[0] != '\0') {                          // Only valid characters
        Serial.print(currentChar);
        Serial.print(": ");
        // Play each symbol in Morse code
        for (int symbolIndex = 0; morseSymbol[symbolIndex] != '\0'; symbolIndex++) {
          playMorseChar(morseSymbol[symbolIndex]);
```

```
        }
        Serial.println(morseSymbol);
        delay(letterSpace - dotTime);                          // Space between letters
      }
    }
    Serial.println("Message completed!");
    Serial.println("Enter a sentence: ");
  }
}
//---------------------------------------------------------------------
// Display dot on LED matrix
//---------------------------------------------------------------------
void showDot() {
  matrix.clear();                                             // Clear previous display
  matrix.renderBitmap(dotBitmap, 8, 12);                     // Show dot bitmap
}
//---------------------------------------------------------------------
// Display dash on LED matrix
//---------------------------------------------------------------------
void showDash() {
  matrix.clear();                                             // Clear previous display
  matrix.renderBitmap(dashBitmap, 8, 12);                    // Show dash bitmap
}
//---------------------------------------------------------------------
// Play Morse symbol (visual and audible)
//---------------------------------------------------------------------
void playMorseChar(char symbol) {
  if (symbol == '.') {                                        // Dot
    digitalWrite(ledPin, HIGH);                               // LED ON
    tone(buzzerPin, 1000, dotTime);                           // Play dot tone
    showDot();                                                // Display dot on matrix
    delay(dotTime);                                           // Wait dot duration
    digitalWrite(ledPin, LOW);                                // LED OFF
    noTone(buzzerPin);                                        // Stop buzzer
    delay(dotTime);                                           // Short pause
  } else {                                                    // Dash
    digitalWrite(ledPin, HIGH);                               // LED ON
    tone(buzzerPin, 1000, dashTime);                          // Play dash tone
    showDash();                                               // Display dash on matrix
    delay(dashTime);                                          // Wait dash duration
    digitalWrite(ledPin, LOW);                                // LED OFF
    delay(dotTime);                                           // Short pause
    noTone(buzzerPin);                                        // Stop buzzer
  }
}
```

Code – Result

Plurality - Code

```c
#include <cs50.h>
#include <stdio.h>
#include <string.h>
// Max number of candidates
#define MAX 9
// Candidates have name and vote count
typedef struct
{
    string name;                            // Candidate's name
    int votes;                              // Candidate vote counter
} candidate;
candidate candidates[MAX];
int number_of_candidates;
bool vote(string name);
void print_winner(void);
int main(int argc, string argv[])
{
    // Check if the program was called with at least 1 candidate
    if (argc < 2)
    {
        printf("Usage: plurality candidate1, ...2, etc.\n");       // Shows how to use
        return 1;
    }
    // Populate number_of_candidates with the number of candidates (arguments - 1)
    number_of_candidates = argc - 1;
    // If it exceed the defined limit
    if (number_of_candidates > MAX)
    {
        printf("Maximum number of candidates is %i\n", MAX);
        return 2;
    }
    // Copies the argument names to the candidates array and resets the votes
    for (int i = 0; i < number_of_candidates; i++)
    {
        candidates[i].name = argv[i + 1];                 // argv[0] is the name of the
program, so +1
        candidates[i].votes = 0;                          // Initialize votes to 0
    }
    // Ask the user how many voters there will be
    int voter_count = get_int("Number of voters: ");
    // For each voter, ask for the vote and try to register it
    for (int i = 0; i < voter_count; i++)
    {
        string name = get_string("Vote: ");
        // If the vote function returns false >> invalid vote
        if (!vote(name))
        {
            printf("Invalid vote.\n");
        }
    }
    // After all votes, print the winner(s)
    print_winner();
    return 0;
}
bool vote(string name)
```

```
{
    // Iterate over each candidate
    for (int i = 0; i < number_of_candidates; i++)
    {
        // Check if candidate's name matches given name
        if (strcmp(candidates[i].name, name) == 0)
        {
            candidates[i].votes++;                      // If yes, increment candidate's
votes and return true
            return true;
        }
    }
    // If no match, return false
    return false;
}
// Print the winner (or winners) of the election
void print_winner (void)
{
    int max_votes = 0;
    // Find the maximum number of votes
    for (int i = 0; i < number_of_candidates; i++)
    {
        // If the current candidate has more votes than MAX
        if (candidates[i].votes > max_votes)
        {
            max_votes = candidates[i].votes;
        }
    }
    // Second loop: print all candidates whose votes == max_votes
    for (int i = 0; i < number_of_candidates; i++)
    {
        if (candidates[i].votes == max_votes)
        {
            printf("%s\n", candidates[i].name);             // Print the winner's name
        }
    }
}
```

Code – Result

Tideman – Code

```c
#include <cs50.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
// Maximum number of candidates
#define MAX 9
int preferences[MAX][MAX];                      // Stores how many voters prefer candidate i over candidate j
bool locked[MAX][MAX];// Indicates if candidate i won against j and this victory is "locked" in the graph
typedef struct                                  // Structure that stores a pair of candidates: winner and loser
{
    int winner;
    int loser;
} pair;
string candidates[MAX];                         // Array that stores the names of the candidates
pair pairs[MAX * (MAX - 1) / 2];                // Array that stores all pairs (winner, loser)
int pair_count;                                 // Number of pairs that have been found
int candidate_count;                            // Number of candidates participating
bool vote(int rank, string name, int ranks[]);  // Records a voter's preference for a given candidate
void record_preferences(int ranks[]);           // Updates the global preferences matrix based on one voter's rankings
void add_pairs(void);                           // Creates all pairs (winner, loser) based on who is preferred by more voters
void sort_pairs(void);                          // Sorts the pairs in descending order by strength of victory
void lock_pairs(void);                          // Locks the pairs into the directed graph, avoiding any cycles
void print_winner(void);                        // Determines and prints the overall winner (the source of the graph)
bool cycle(int start, int candidate);           // Helper function to detect cycles recursively when locking pairs
int main(int argc, string argv[])
{
    if (argc < 2)
    {
        printf("Usage: tideman [candidate ...]\n");
        return 1;
    }
    candidate_count = argc - 1;                 // Take of the name of the program (./tideman)
    if (candidate_count > MAX)
    {
        printf("Maximum number of candidates is %i\n", MAX);
        return 2;
    }
    // Copies the candidate names from the command line to the candidates array
    for (int candidate_index = 0; candidate_index < candidate_count; candidate_index++)
    {
        candidates[candidate_index] = argv[candidate_index + 1];
```

```
    }
    // Initializes the locked graph: all start as "false" (no one has won yet)
    for (int candidate_a = 0; candidate_a < candidate_count; candidate_a++)
    {
        for (int candidate_b = 0; candidate_b < candidate_count; candidate_b++)
        {
            locked[candidate_a][candidate_b] = false;
        }
    }
    pair_count = 0;
    int voter_count = get_int ("Number of voters: ");
    // For each voter, we will collect the ranking preferences
    for (int vouter_index = 0; vouter_index < voter_count; vouter_index++)
    {
        int ranks[candidate_count];                                                    //
ranks will store this voter's ranking order
        // Ask for the preference for each rank (1st, 2nd, 3rd, ...)
        for (int rank_position = 0; rank_position < candidate_count; rank_position++)
        {
            string name = get_string("Rank %i: ", rank_position + 1);
            if (!vote (rank_position, name, ranks))
            {
                printf ("Invalid vote.\n");
                return 3;
            }
        }
        record_preferences(ranks);                  // After collecting this voter's
ranks, update the preferences matrix
        printf("\n");                               // Just to visually separate voters
    }
    add_pairs();                                    // Create all winner-loser pairs
    sort_pairs();                                   // Sort the pairs by strength of
victory
    lock_pairs();                                   // Lock the victories in the graph
(without creating cycles)
    print_winner();                                 // Show who won the election
}
/* Checks if the voted name exists among the candidates
If it exists, stores the position of that candidate in the ranks array */
bool vote(int rank, string name, int ranks[])
{
    for (int candidate_index = 0; candidate_index < candidate_count; candidate_index++)
    {
        if (strcmp(candidates[candidate_index], name) == 0)
        {
            ranks[rank] = candidate_index;                      // Store the candidate's
index in this rank
            return true;
        }
    }
    return false;
}
void record_preferences(int ranks[])                            // Updates the preferences
matrix with a single voter's choices
{
    // Go through each candidate the voter ranked
    for (int higher_rank = 0; higher_rank < candidate_count; higher_rank++)
    {
        // For each candidate ranked ahead, increase the count against those ranked behind
```

```
        for (int lower_rank = higher_rank + 1; lower_rank < candidate_count; lower_rank++)
        {
            int preferred = ranks[higher_rank];                         // Candidate
ranked higher
            int less_preferred = ranks[lower_rank];                     // Candidate
ranked lower
            preferences[preferred][less_preferred]++;                   // Increment 1
vote
        }
    }
}
void add_pairs(void)                // Creates winner-loser pairs from the preferences
matrix
{
    pair_count = 0;
    // Compare all candidates against each other
    for (int candidate_a = 0; candidate_a < candidate_count; candidate_a++)
    {
        for (int candidate_b = candidate_a + 1; candidate_b < candidate_count;
candidate_b++)
        {
            // If more people prefer i over j
            if (preferences[candidate_a][candidate_b] >
preferences[candidate_b][candidate_a])
            {
                pairs[pair_count].winner = candidate_a;
                pairs[pair_count].loser = candidate_b;
                pair_count++;
            }
            // If more people prefer j over i
            else if (preferences[candidate_b][candidate_a] >
preferences[candidate_a][candidate_b])
            {
                pairs[pair_count].winner = candidate_b;
                pairs[pair_count].loser = candidate_a;
                pair_count++;
            }
            // If it's a tie, do not create a pair
        }
    }
}
// Sorts the pairs by strength of victory (difference in votes)
void sort_pairs(void)
{
    // Bubble sort: compare neighboring pairs and swap if they are out of order
    for (int first_pair = 0; first_pair < pair_count - 1; first_pair++)
    {
        for (int next_pair = 0; next_pair < pair_count - first_pair - 1; next_pair++)
        {
            int strength1 = preferences[pairs[next_pair].winner][pairs[next_pair].loser];
            int strength2 = preferences[pairs[next_pair + 1].winner][pairs[next_pair +
1].loser];
            // If the second pair is stronger, swap the order
            if (strength1 < strength2)
            {
                pair temp_pair = pairs[next_pair];
                pairs[next_pair] = pairs[next_pair + 1];
                pairs[next_pair + 1] = temp_pair;
            }
```

```
        }
    }
}
// Recursive function that checks if adding a connection creates a cycle
bool cycle(int start, int candidate)
{
    // If the current candidate is the same as the start, a cycle is formed
    if (candidate == start)
    {
        return true;
    }
    // For each candidate, see if there is an arrow leading to a cycle
    for (int candidate_index = 0; candidate_index < candidate_count; candidate_index++)
    {
        // If there is a connection and it continues the cycle, return true
        if (locked[candidate][candidate_index] && cycle(start, candidate_index))
        {
            return true;
        }
    }
    return false;
}
// Locks the pairs in the graph, ensuring no cycles are created
void lock_pairs(void)
{
    for (int pair_index = 0; pair_index < pair_count; pair_index++)
    {
        int winner = pairs[pair_index].winner;
        int loser = pairs[pair_index].loser;
        // Only lock the connection if it does not form a cycle
        if (!cycle(winner, loser))
        {
            locked[winner][loser] = true;
        }
    }
}
// Prints the winner of the election
void print_winner(void)
{
    // The winner is the candidate with no arrows pointing at them
    for (int candidate_index = 0; candidate_index < candidate_count; candidate_index++)
    {
        bool has_arrow_pointing = false;
        // Check if any candidate has won against this one
        for (int opponent_index = 0; opponent_index < candidate_count; opponent_index++)
        {
            if (locked[opponent_index][candidate_index])
            {
                has_arrow_pointing = true;
                break;
            }
        }
        // If no one defeated this candidate, they are the final winner
        if (!has_arrow_pointing)
        {
            printf("%s\n", candidates[candidate_index]);
            return;
        }
    }
```

```
}
```

Code – Result

Traffic Lights - Code

```
//-------------------------------------------------------------------------
// Traffic Light and Pedestrian Crossing System
// ============================================
//
// Controls a traffic light with pedestrian crossing support.
//
// Features:
// - Uses shift registers to control east, north, and pedestrian lights.
// - 7-segment display shows countdown timer for pedestrians.
// - Buzzer provides sound feedback during crossing.
// - Pedestrian button requests crossing.
// - Lights follow timed sequence, handling pedestrian requests.
//
// Francisco Pimentel
// 04 / Oct / 2025
// version 2.3.6
//-------------------------------------------------------------------------
#define ON HIGH                                 // Define HIGH as ON
#define OFF LOW                                 // Define LOW as OFF
#define pedbut 2                                 // Pedestrian button pin
// Traffic light color bitmaps for shift register
#define Red    0b00000100                        // Red light
#define Yellow 0b00000010                        // Yellow light
#define Green  0b00000001                        // Green light
// Shift register pins
const int STCP = 4;                               // Latch pin
int SHCP = 5;                                     // Clock pin
int DS   = 6;                                     // Data pin
int buzzer = 13;                                  // Buzzer pin
long timer;                                       // Stores start time of traffic light cycle
// 7-segment display segment pins (A-G)
const int LED[] = {3, 7, 8, 9, 10, 11, 12};
// Lookup table: segments to display digits 0-9 (g f e d c b a)
const byte SEG[] = {
  0b00111111, 0b00000110, 0b01011011, 0b01001111, 0b01100110,
  0b01101101, 0b01111101, 0b00000111, 0b01111111, 0b01101111
};
volatile int PED1 = 0;                             // Pedestrian button state flag (0=not pres
sed)
// Function declarations
void showNumber(int digit);
void colors(int east, int north, int pedestrian);
void PedRequest();
void ifPedestrians();
void setup()
{
  Serial.begin(9600);                              // Initialize serial for debugging
  pinMode(STCP, OUTPUT);                           // Latch pin as output
  pinMode(SHCP, OUTPUT);                           // Clock pin as output
  pinMode(DS, OUTPUT);                             // Data pin as output
  pinMode(buzzer, OUTPUT);                         // Buzzer as output
```

```
  pinMode(pedbut, INPUT_PULLUP);                // Button as input with internal pullup
  // Set 7-segment display pins as outputs
  for (int i = 0; i < 7; i++)
  {
    pinMode(LED[i], OUTPUT);                    // Each segment pin as output
  }
  attachInterrupt(digitalPinToInterrupt(pedbut), PedRequest, FALLING); // Interrupt for bu
tton press
  timer = millis();                             // Initialize traffic light timer
}
void loop()
{
  ifPedestrians();                              // Handle pedestrian requests
  long timePassed = millis() - timer;           // Time since current cycle started
  // Traffic light phases
  if (timePassed <= 9000)
  {
    colors(Green, Red, Red);                    // East green, North red, Pedestrian red
  }
  else if (timePassed <= 12000)
  {
    colors(Yellow, Red, Red);                   // East yellow, North red, Pedestrian red
  }
  else if (timePassed <= 13000)
  {
    colors(Red, Red, Red);                      // All red
  }
  else if (timePassed <= 15000)
  {
    colors(Red, Red + Yellow, Red);             // East red, North red+yellow, Pedestrian
red
  }
  else if (timePassed <= 19000)
  {
    colors(Red, Green, Red);                    // East red, North green, Pedestrian red
  }
  else if (timePassed <= 22000)
  {
    colors(Red, Yellow, Red);                   // East red, North yellow, Pedestrian red
  }
  else if (timePassed <= 23000)
  {
    colors(Red, Red, Red);                      // All red
  }
  else if (timePassed <= 25000)
  {
    colors(Red + Yellow, Red, Red);             // East red+yellow, North red, Pedestrian
red
  }
  else if (timePassed <= 27000)
  {
    colors(Green, Red, Red);                    // East green, North red, Pedestrian red
  }
  else
  {
    timer = millis();                            // Reset traffic light cycle
  }
}
//---------------------------------------------------------------------
```

```
// Display digit on 7-segment
//-----------------------------------------------------------------------
void showNumber(int digit)
{
  if (digit > 9) digit = 9;                    // Limit digits to 0-9
  byte activeSegments = SEG[digit];            // Get segment pattern
  for (int segmentIndex = 0; segmentIndex < 7; segmentIndex++)
  {
    if (activeSegments & (1 << segmentIndex))
    {
      digitalWrite(LED[segmentIndex], HIGH);    // Turn on segment
    }
    else
    {
      digitalWrite(LED[segmentIndex], LOW);     // Turn off segment
    }
  }
}
//-----------------------------------------------------------------------
// Set traffic light colors using shift register
//-----------------------------------------------------------------------
void colors(int east, int north, int pedestrian)
{
  if (pedestrian == 0b00000100) pedestrian = 0b00000010; // Adjust for shift register
  digitalWrite(STCP, LOW);                      // Prepare latch
  shiftOut(DS, SHCP, LSBFIRST, (east << 5) + (north << 2) + pedestrian); // Shift data
  digitalWrite(STCP, HIGH);                     // Latch output
}
//-----------------------------------------------------------------------
// Interrupt for pedestrian button
//-----------------------------------------------------------------------
void PedRequest()
{
  PED1 = 1;                                     // Set pedestrian flag
}
//-----------------------------------------------------------------------
// Handle pedestrian crossing with countdown and buzzer
//-----------------------------------------------------------------------
void ifPedestrians()
{
  if(PED1)
  {
    long timePassed = millis() - timer;         // Time since cycle start
    int remainingTime = 0;                      // Remaining seconds in current phase
    // Determine remaining seconds for current light phase
    if (timePassed <= 9000) remainingTime = 9 - timePassed / 1000;
    else if (timePassed <= 12000) remainingTime = 12 - timePassed / 1000;
    else if (timePassed <= 13000) remainingTime = 13 - timePassed / 1000;
    else if (timePassed <= 15000) remainingTime = 15 - timePassed / 1000;
    else if (timePassed <= 19000) remainingTime = 19 - timePassed / 1000;
    else if (timePassed <= 22000) remainingTime = 22 - timePassed / 1000;
    else if (timePassed <= 23000) remainingTime = 23 - timePassed / 1000;
    else if (timePassed <= 25000) remainingTime = 25 - timePassed / 1000;
    else if (timePassed <= 27000) remainingTime = 27 - timePassed / 1000;
    else remainingTime = 9;
    if (remainingTime < 0) remainingTime = 0;
    // Countdown on 7-segment display
    for(int i = remainingTime; i >= 0; i--)
    {
```

```
      showNumber(i);                                // Show digit
      delay(1000);                                  // Wait 1 second
    }
    colors(Red, Red, Green);                        // Pedestrian green
    // Buzzer blinking for pedestrian crossing
    for(int i = 0; i < 150; i++)
    {
      if(i % 2) noTone(buzzer);                     // Alternate buzzer off
      else tone(buzzer, 600);                       // Alternate buzzer on
      delay(100);
    }
    PED1 = 0;                                       // Reset flag
    timer = millis();                               // Reset timer to keep traffic sequence
  }
}
```

Code – Result

Filter Less - Code

```
#include <helpers.h>
#include <cs50.h>
#include <math.h>
// Convert image to grayscale
void grayscale(int height, int width, RGBTRIPLE image[height][width])
{
    float avg;
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            avg = round((image[i][j].rgbtRed + image[i][j].rgbtGreen + image[i][j].rgbtBlue)
/ 3.0);
            image[i][j].rgbtRed = avg;
            image[i][j].rgbtGreen = avg;
            image[i][j].rgbtBlue = avg;
        }
    }
    return;
}
// Convert image to sepia
void sepia(int height, int width, RGBTRIPLE image[height][width])
{
    int sepiaRed, sepiaGreen, sepiaBlue;
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            sepiaRed = round(.393 * image[i][j].rgbtRed + .769 * image[i][j].rgbtGreen +
.189 * image[i][j].rgbtBlue);
            sepiaGreen = round(.349 * image[i][j].rgbtRed + .686 * image[i][j].rgbtGreen +
.168 * image[i][j].rgbtBlue);
            sepiaBlue = round(.272 * image[i][j].rgbtRed + .534 * image[i][j].rgbtGreen +
.131 * image[i][j].rgbtBlue);
            // Limitar valores a 255
            if (sepiaRed > 255) {
```

```
                image[i][j].rgbtRed = 255;
            } else {
                image[i][j].rgbtRed = sepiaRed;
            }
            if (sepiaGreen > 255) {
                image[i][j].rgbtGreen = 255;
            } else {
                image[i][j].rgbtGreen = sepiaGreen;
            }
            if (sepiaBlue > 255) {
                image[i][j].rgbtBlue = 255;
            } else {
                image[i][j].rgbtBlue = sepiaBlue;
            }
        }
    }
    return;
}
// Reflect image horizontally
void reflect(int height, int width, RGBTRIPLE image[height][width])
{
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width / 2; j++)
        {
            RGBTRIPLE temp = image[i][j];
            image[i][j] = image[i][width - j - 1];
            image[i][width - j - 1] = temp;
        }
    }
    return;
}
// Blur image
void blur(int height, int width, RGBTRIPLE image[height][width])
{
    RGBTRIPLE copy[height][width];
    // Copiar imagem original
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            copy[i][j] = image[i][j];
        }
    }
    // Apply blur
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            int totalRed = 0, totalGreen = 0, totalBlue = 0;
            int counter = 0;
            for (int di = -1; di <= 1; di++)
            {
                for (int dj = -1; dj <= 1; dj++)
                {
                    int new_i = i + di;
                    int new_j = j + dj;
                    if (new_i >= 0 && new_i < height && new_j >= 0 && new_j < width)
                    {
```

```
                        totalRed += copy[new_i][new_j].rgbtRed;
                        totalGreen += copy[new_i][new_j].rgbtGreen;
                        totalBlue += copy[new_i][new_j].rgbtBlue;
                        counter++;
                    }
                }
            }
            image[i][j].rgbtRed = round((float) totalRed / counter);
            image[i][j].rgbtGreen = round((float) totalGreen / counter);
            image[i][j].rgbtBlue = round((float) totalBlue / counter);
        }
    }
    return;
}
```

Code – Result

Recover - Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
// Each block in the raw file is 512 bytes
#define BLOCK_SIZE 512
int main(int argc, char *argv[])
{
    // Check if the user provided exactly one file
    if (argc != 2)
    {
        printf("Usage: ./recover filename\n");  // Show how to run the program
        return 1;                               // Error
    }
    // Open the input file for reading
    FILE *raw_file = fopen(argv[1], "r");
    if (raw_file == NULL)
    {
        printf("Could not open file.\n");
        return 1;
    }
    // Buffer to store one 512-byte block at a time
    uint8_t buffer[BLOCK_SIZE];
    // Pointer to the JPEG file the user is currently creating
    FILE *img = NULL;
    // Counter for naming the recovered JPEG files
    int file_count = 0;
    // Read through the file block by block
    while (fread(buffer, BLOCK_SIZE, 1, raw_file))
    {
        // Check if this block contains the start of a JPEG
        if (buffer[0] == 0xff &&
            buffer[1] == 0xd8 &&
            buffer[2] == 0xff &&
            (buffer[3] & 0xf0) == 0xe0)
        {
            // If a JPEG file was already open, close it
            if (img != NULL)
```

```
            {
                fclose(img);
            }
            // Create a new filename like 000.jpg, 001.jpg, ...
            char filename[8];
            sprintf(filename, "%03i.jpg", file_count);
            file_count++;
            // Open the new JPEG file for writing
            img = fopen(filename, "w");
        }
        // If we have an open JPEG, write this block to it
        if (img != NULL)
        {
            fwrite(buffer, BLOCK_SIZE, 1, img);
        }
    }
    // Close the last JPEG if one is open
    if (img != NULL)
    {
        fclose(img);
    }
    // Close the raw input file
    fclose(raw_file);
}
```

Code - Result

Displaying Uptime - Code

```
//-----------------------------------------------------------------------
// 4-Digit Timer with Display Sweep Effect
// ======================================
//
// Implements a real-time timer with a 4-digit 7-segment display
// and smooth sweeping effect. Uses shift registers and a buzzer.
//
// Features:
// - Counts MM:SS up to 99:59 and wraps around.
// - Potentiometer adjusts sweeping speed of digits.
// - Button pauses/resumes timer.
// - Buzzer beeps every second.
// - Multiplexed 7-segment display with optional decimal point.
//
// Francisco Pimentel
// 08 / Oct / 2025
// version 2.3.6
//-----------------------------------------------------------------------
#define ON HIGH                                          // Define HIGH as ON for clarity
#define OFF LOW                                          // Define LOW as OFF for clarity
// Pin assignments for 4-digit display and peripherals
const int DIGIT_PIN1      = 11;                         // Digit 1 control
const int DIGIT_PIN2      = 10;                         // Digit 2 control
const int DIGIT_PIN3      = 9;                          // Digit 3 control
const int DIGIT_PIN4      = 8;                          // Digit 4 control
const int SHCP            = 4;                          // Shift register clock pin
const int STCP            = 5;                          // Shift register latch pin
const int DS              = 6;                          // Shift register data pin
const int POTENTIOMETER   = A0;                         // Potentiometer for sweep speed
const int BUZZER_PIN      = 13;                         // Buzzer pin
const int BUTTON_PIN      = 12;                         // Pause/resume button pin
// Store current digit values for display
int currentDigits[4];
int digitPinsArray[4] = {DIGIT_PIN1, DIGIT_PIN2, DIGIT_PIN3, DIGIT_PIN4};  // Array for lo
oping through digits
// 7-segment display patterns (bits a-g)
const byte SEGMENT_PATTERNS[10] = {
  0b00111111,   // 0
  0b00000110,   // 1
  0b01011011,   // 2
  0b01001111,   // 3
  0b01100110,   // 4
  0b01101101,   // 5
  0b01111101,   // 6
  0b00000111,   // 7
  0b01111111,   // 8
  0b01101111    // 9
};
// Timer state
unsigned long lastSecondUpdateTime = 0;   // Timestamp of last second increment
int currentSeconds = 0;                    // Current seconds count
int currentMinutes = 0;                    // Current minutes count
// Button & pause state
bool timerPaused = false;                  // Pause flag
bool lastButtonState = HIGH;               // Last button reading for edge detection
void setup()
```

```
{
  Serial.begin(9600);                       // Initialize serial for debugging
  pinMode(SHCP, OUTPUT);                    // Shift register clock as output
  pinMode(STCP, OUTPUT);                    // Shift register latch as output
  pinMode(DS, OUTPUT);                      // Shift register data as output
  pinMode(POTENTIOMETER, INPUT);            // Potentiometer as analog input
  pinMode(BUZZER_PIN, OUTPUT);              // Buzzer as output
  pinMode(BUTTON_PIN, INPUT_PULLUP);        // Button as input with internal pullup
  digitalWrite(BUZZER_PIN, OFF);            // Ensure buzzer is initially off
  digitalWrite(SHCP, OFF);                  // Initialize shift register clock low
  digitalWrite(STCP, OFF);                  // Initialize shift register latch low
  digitalWrite(DS, OFF);                    // Initialize shift register data low
  // Initialize digit pins to OFF
  for (int digit = 0; digit < 4; digit++)
  {
    pinMode(digitPinsArray[digit], OUTPUT);  // Set each digit control as output
    digitalWrite(digitPinsArray[digit], OFF);// Turn off all digits initially
  }
}
void loop()
{
  unsigned long currentMillis = millis();        // Get current time in ms
  // Read button and detect falling edge for pause/resume
  bool buttonReading = digitalRead(BUTTON_PIN);  // Read current button state
  if (lastButtonState == HIGH && buttonReading == LOW)
  {
    timerPaused = !timerPaused;                  // Toggle pause state
  }
  lastButtonState = buttonReading;               // Update last button state
  // ---- Real-time timer counting ----
  if (!timerPaused && currentMillis - lastSecondUpdateTime >= 1000)
  {
    lastSecondUpdateTime = currentMillis;       // Update last second timestamp
    currentSeconds++;                            // Increment seconds
    tone(BUZZER_PIN, 1800, 80);                 // Short beep for each second
    if (currentSeconds >= 60)                    // Check for minute overflow
    {
      currentSeconds = 0;                        // Reset seconds
      currentMinutes++;                           // Increment minutes
      if (currentMinutes >= 100)                 // Wrap at 99 minutes
      {
        currentMinutes = 0;                      // Reset minutes
      }
    }
  }
  // ---- Display sweep speed control ----
  int potentiometerLevel = analogRead(POTENTIOMETER);           // Read potentiometer
  int digitDisplayDelay = map(potentiometerLevel, 0, 1023, 1, 30);  // Map to delay value
  digitDisplayDelay = 1 + (int)(pow((float)digitDisplayDelay, 1.2)); // Exponential smooth
ing
  updateDisplay(digitDisplayDelay);            // Refresh display with sweep
}
//--------------------------------------------------------------------
// Split 4-digit number into individual digits
//--------------------------------------------------------------------
void splitNumberIntoDigits(long number)
{
  currentDigits[0] = number / 1000;           // Thousands
  number %= 1000;
```

```
  currentDigits[1] = number / 100;            // Hundreds
  number %= 100;
  currentDigits[2] = number / 10;             // Tens
  currentDigits[3] = number % 10;             // Units
}
//-----------------------------------------------------------------------
// Update 7-segment display with sweeping effect
//-----------------------------------------------------------------------
void updateDisplay(int digitDelay)
{
  int totalTime = (currentMinutes * 100) + currentSeconds;  // Combine minutes and seconds
  splitNumberIntoDigits(totalTime);                         // Break into digits
  // Loop through each digit for multiplexing/sweep effect
  for (int digitIndex = 0; digitIndex < 4; digitIndex++)
  {
    byte pattern = SEGMENT_PATTERNS[currentDigits[digitIndex]];  // Pattern for current di
git
    if (digitIndex == 1)                                      // Optional decimal point
    {
      pattern |= 0b10000000;                                  // Turn on DP for tens dig
it
    }

    // Send pattern to shift register
    digitalWrite(STCP, LOW);                                   // Prepare latch
    shiftOut(DS, SHCP, MSBFIRST, pattern);                     // Send segment pattern
    digitalWrite(STCP, HIGH);                                  // Latch output
    // Activate current digit briefly
    digitalWrite(digitPinsArray[digitIndex], ON);              // Turn on current digit
    delay(digitDelay);                                         // Display duration for s
weeping effect
    digitalWrite(digitPinsArray[digitIndex], OFF);             // Turn off digit before
next
  }
}
```

Code - Result

Git Together - Code

```
/*
// Program that offers a simple serial menu for sum
// and multiplication operations
*/
#include <Arduino.h>
bool call_sums = false;                         // Flag to trigger sum function when true
bool call_multiply = false;                     // Flag to trigger multiply function when tr
ue
void sums();                                    // Function prototype: sum two numbers

void multiply();                                // Function prototype: multiply two numbers
void blink();                                   // Function prototype: blink onboard LED
int menu_choice;                                // Stores the user menu choice from Serial i
nput
const int ledPin = LED_BUILTIN;                 // Onboard LED pin reference
void setup()
{
  Serial.begin(9600);                           // Initialize Serial communication
  pinMode(ledPin, OUTPUT);                      // Configure LED pin as output
  Serial.println("-----Menu-----");             // Print menu header
  Serial.println("1. Calculate Sums");          // Option 1: sum
  Serial.println("2. Multiplication ");         // Option 2: multiply
  Serial.println("Enter your choice: ");        // Prompt user
  Serial.println("--------------");              // Visual separator
}
void loop()
{
  blink();                                      // Continuously blink LED to show program i
s running
  if(Serial.available() > 0)                    // Check if user entered something in Seria
l
  {
    menu_choice = Serial.parseInt();            // Read user input as integer
    if(menu_choice == 1)                        // If user selects Sums
    {
      Serial.read();                            // Clear extra characters (like newline)
      call_sums = true;                         // Set flag to call sum function
    }
    else if(menu_choice == 2)                   // If user selects Multiplication
    {
      Serial.read();                            // Clear extra characters
      call_multiply = true;                     // Set flag to call multiply function
    }
    else                                        // Invalid user input
    {
      Serial.read();                            // Clear buffer
      Serial.println("Invalid Input, pick 1 or 2"); // Error message
    }
  }
  if (call_sums == true)                        // If flag is active
  {
    sums();                                     // Run sum function
    call_sums = false;                          // Reset flag afterwards
  }
  if (call_multiply == true)                    // If flag is active
```

```
  {
    multiply();                                   // Run multiply function
    call_multiply = false;                        // Reset flag afterwards
  }
}
void blink()
{
  digitalWrite(ledPin, HIGH);                     // LED ON
  delay(1000);                                    // Wait 1 second
  digitalWrite(ledPin, LOW);                      // LED OFF
  delay(1000);                                    // Wait 1 second
}
void sums()
{
  int num1;
  int num2;
  Serial.println("Enter 2 numbers: ");          // Ask user for two numbers
  while (Serial.available() == 0) {}              // Wait until input received
  num1 = Serial.parseInt();                       // Read first number
  Serial.read();                                  // Clear buffer
  while (Serial.available() == 0) {}              // Wait for second number
  num2 = Serial.parseInt();                       // Read second number
  int result_s = num1 + num2;                     // Add numbers
  Serial.print("The sum is: ");                   // Output label
  Serial.println(result_s);                       // Output result
  Serial.read();                                  // Clear remaining buffer
  Serial.println("-----Menu-----");               // Reprint menu
  Serial.println("1. Calculate Sums");
  Serial.println("2. Multiplication ");
  Serial.println("Enter your choice: ");
  Serial.println("--------------");
}
void multiply()
{
  int num1;
  int num2;
  Serial.println("Enter 2 numbers: ");          // Ask user for two numbers
  while (Serial.available() == 0) {}              // Wait until input received
  num1 = Serial.parseInt();                       // Read first number
  Serial.read();                                  // Clear buffer
  while (Serial.available() == 0) {}              // Wait for second number
  num2 = Serial.parseInt();                       // Read second number
  int result_s = num1 * num2;                     // Multiply numbers
  Serial.print("The result is: ");                // Output label
  Serial.println(result_s);                       // Output result
  Serial.read();                                  // Clear buffer
  Serial.println("-----Menu-----");               // Reprint menu
  Serial.println("1. Calculate Sums");
  Serial.println("2. Multiplication ");
  Serial.println("Enter your choice: ");
  Serial.println("--------------");
}
```

Code - Result

Inheritance - Code

```c
#define _DEFAULT_SOURCE
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
// Definition of the type that represents a person
typedef struct person
{
    /* Pointers to the two parents: parents[0] and parents[1]
    may be NULL if we are in the oldest generation */
    struct person *parents[2];
    char alleles[2];                                    // Stores both alleles of
this person, e.g.: {'A','O'}
} person;
/* Program parameters */
const int GENERATIONS = 3;                              // Number of generations t
o create (0 = child, 1 = parents, 2 = grandparents, ...)
const int INDENT_LENGTH = 4;                            // How many spaces to use
to indent each generation when printing
person *create_family(int generations);                 // Recursively creates a p
erson and their ancestors
void print_family(person *p, int generation);           // Prints the family tree
with indentation
void free_family(person *p);                            // Recursively frees alloc
ated memory
char random_allele();                                   // Randomly chooses 'A', '
B', or 'O'
int main(void)
{
    srandom(time(0));
    /* Create the family structure with the defined number of generations.
       The function returns a pointer to the youngest person (generation 0). */
    person *p = create_family(GENERATIONS);
    // Print the family tree starting from generation 0
    print_family(p, 0);
    // Free all memory allocated by create_family
    free_family(p);
    return 0;
}
person *create_family(int generations)
{
    /* Allocate memory for a new person
       sizeof(person) returns how many bytes the struct requires */
    person *new_person = malloc(sizeof(person));
    // If malloc fails, return NULL to signal allocation error
    if (new_person == NULL)
    {
        return NULL;
    }
    // Recursive case: still need to generate ancestors
    if (generations > 1)
    {
        /* Create both parents recursively: each call creates a sub-tree
           with (generations - 1) generations. */
        person *parent0 = create_family(generations - 1);
```

```
        person *parent1 = create_family(generations - 1);
        // Assign the pointers to the parents
        new_person->parents[0] = parent0;
        new_person->parents[1] = parent1;
        /* Inherit alleles: each allele of new_person comes randomly from one
           of the two alleles of the respective parent. */
        new_person->alleles[0] = parent0->alleles[random() % 2];
        new_person->alleles[1] = parent1->alleles[random() % 2];
    }
    else
    {
        // Base case: oldest generation (no parents)
        new_person->parents[0] = NULL; // No known parents
        new_person->parents[1] = NULL;
        // Choose two random alleles because there are no parents to inherit from
        new_person->alleles[0] = random_allele();
        new_person->alleles[1] = random_allele();
    }
    return new_person;
}
void free_family(person *p)
{
    // Base case: if the pointer is NULL, there's nothing to free
    if (p == NULL)
    {
        return;
    }
    // Recursively free the parents (if they exist)
    free_family(p->parents[0]);
    free_family(p->parents[1]);
    // Finally, free the person struct itself
    free(p);
}
void print_family(person *p, int generation)
{
    // If p is NULL, we reached the end of a branch, nothing to print
    if (p == NULL)
    {
        return;
    }
    // Print indentation spaces: each generation indents by INDENT_LENGTH spaces
    for (int i = 0; i < generation * INDENT_LENGTH; i++)
    {
        printf(" ");
    }
    // Depending on the generation, print a different label (child, parent, grandparent, e
tc.)
    if (generation == 0)
    {
        printf("Child (Generation %i): blood type %c%c\n", generation, p->alleles[0], p-
>alleles[1]);
    }
    else if (generation == 1)
    {
        printf("Parent (Generation %i): blood type %c%c\n", generation, p->alleles[0], p-
>alleles[1]);
    }
    else
    {
```

```
    /* For older generations, print the prefix "Great-" repeated
    (generation - 2) times before "Grandparent" */
    for (int i = 0; i < generation - 2; i++)
        {
        printf("Great-");
        }
        printf("Grandparent (Generation %i): blood type %c%c\n", generation, p-
>alleles[0], p->alleles[1]);
    }
    /* After printing the current person, proceed to their parents (left, right)
    and increase generation so that parents are printed more indented. */
    print_family(p->parents[0], generation + 1);
    print_family(p->parents[1], generation + 1);
}
char random_allele()
{
    int r = random() % 3; // r will be 0, 1, or 2
    if (r == 0)
    {
        return 'A';
    }
    else if (r == 1)
    {
        return 'B';
    }
    else
    {
        return 'O';
    }
}
```

Code - Result

Jumbo Clock - Code

```
/* ------------------------------------------------
  Jumno Clock with LCD Sweep Effect
  ------------------------------------------------
  Displays a large-format HH:MM:SS clock on a 16x2 LCD
  using custom characters. Colon blinks every second.
  Sweep effect controlled by potentiometer. Buzzer ticks
  each second. User can set time via Serial Monitor.
  Francisco Pimentel
  13 / Oct / 2025
  version 2.3.6
------------------------------------------------ */
#include <LCD_I2C.h>        // Library for controlling I2C LCD
#include "FspTimer.h"       // Library for custom timers
#define OFF LOW             // Logical OFF
#define ON HIGH             // Logical ON
FspTimer MyTimer;           // Create timer object
LCD_I2C lcd(0x27, 16, 2);   // LCD at I2C address 0x27, 16x2
// Initialize random time for testing
int hours = random(24), minutes = random(60), seconds = random(60);
const int POTENTIOMETER = A0;         // Potentiometer input pin
const int BUZZER_PIN = 11;            // Buzzer pin
unsigned long previousMillis = 0;     // Tracks last second update
const unsigned long interval = 1000;  // 1-second update interval
bool showColon = true;                // Controls blinking colon
// Function prototypes
void drawBigDigit(byte digit, int column);  // Draw a single large digit
void updateTime();                          // Update clock time
void drawClock();                           // Draw entire clock
/*-----------------------------------------------
Custom LCD characters for big numbers
8 custom characters (5x8 matrix each)
-----------------------------------------------*/
byte segments[8][8] = {
  {0b00110,0b01110,0b01110,0b01110,0b01110,0b01110,0b01110,0b00110}, // 0: Top-left
  {0b11111,0b11111,0b00000,0b00000,0b00000,0b00000,0b00000,0b00000}, // 1: Top bar
  {0b01100,0b01110,0b01110,0b01110,0b01110,0b01110,0b01110,0b01100}, // 2: Top-right
  {0b00110,0b01110,0b01110,0b01110,0b01110,0b01110,0b01110,0b00110}, // 3: Bottom-left
  {0b00000,0b00000,0b00000,0b00000,0b00000,0b00000,0b11111,0b11111}, // 4: Bottom bar
  {0b01100,0b01110,0b01110,0b01110,0b01110,0b01110,0b01110,0b01100}, // 5: Bottom-right
  {0b11111,0b11111,0b00000,0b00000,0b00000,0b00000,0b11111,0b11111}, // 6: Top+Bottom bars
  {0b00000,0b00000,0b01110,0b01110,0b01110,0b00000,0b00000,0b00000}  // 7: Colon
};
/*-----------------------------------------------
Big digit map (0–9)
Each digit uses 2 rows × 3 segments
255 = empty segment
-----------------------------------------------*/
const byte bigNum[10][2][3] = {
  {{0,1,2},{3,4,5}},           // 0
  {{255,255,2},{255,255,5}},   // 1
  {{1,1,2},{3,6,4}},           // 2
  {{1,1,2},{4,6,5}},           // 3
  {{0,4,2},{255,255,5}},       // 4
  {{0,1,1},{4,6,5}},           // 5
  {{0,1,1},{3,6,5}},           // 6
```

```
  {{1,1,2},{255,255,5}},       // 7
  {{0,1,2},{3,6,5}},           // 8
  {{0,1,2},{4,6,5}}            // 9
};
void setup() {
  Serial.begin(9600);                 // Initialize Serial communication
  lcd.begin();                        // Initialize LCD
  lcd.backlight();                    // Turn on backlight
  pinMode(POTENTIOMETER, INPUT);      // Potentiometer as input
  pinMode(BUZZER_PIN, OUTPUT);        // Buzzer as output
  digitalWrite(BUZZER_PIN, OFF);      // Ensure buzzer is off initially
  // Create custom characters on LCD
  for (int charIndex = 0; charIndex < 8; charIndex++) {
    lcd.createChar(charIndex, segments[charIndex]);
  }
  Serial.println("Enter time as 4 digits (e.g., 0930)."); // Prompt user
}
void loop() {
  tone(BUZZER_PIN, 1800, 80);             // Short tick every loop iteration
  unsigned long currentMillis = millis(); // Track current time
  // Read potentiometer (0-1023)
  int potValue = analogRead(POTENTIOMETER);
  // Map pot to sweep delay
  int sweepDelay = map(potValue, 0, 1023, 1, 60);
  sweepDelay = (int)(pow((float)sweepDelay, 1.3)); // Exponential smoothing for visual eff
ect
  // Update clock every 1 second
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    updateTime();                          // Increment time by 1 second
    // Sweep effect if pot above 100
    if (potValue > 100) {
      for (int col = 0; col < 16; col++) {
        lcd.setCursor(col, 0);           // Clear top row
        lcd.print(" ");
        lcd.setCursor(col, 1);           // Clear bottom row
        lcd.print(" ");
        delay(sweepDelay);               // Small delay to create sweeping animation
      }
    }
    drawClock();                         // Draw updated clock
  }
  // Manual time setting via Serial input
  if (Serial.available() >= 4) {
    String input = Serial.readStringUntil('\n'); // Read input string
    input.trim();                                // Remove whitespace/newline
    if (input.length() == 4 && input.toInt() >= 0 && input.toInt() <= 2359) {
      int newHours = input.substring(0, 2).toInt();    // First two digits = hours
      int newMinutes = input.substring(2, 4).toInt();  // Last two digits = minutes
      if (newHours < 24 && newMinutes < 60) {          // Validate range
        hours = newHours;                               // Update hours
        minutes = newMinutes;                           // Update minutes
        seconds = 0;                                    // Reset seconds
        Serial.print("Time set to: ");
        if (hours < 10) Serial.print("0");             // Leading zero for hours < 10
        Serial.print(hours);
        Serial.print(":");
        if (minutes < 10) Serial.print("0");           // Leading zero for minutes < 10
        Serial.println(minutes);
```

```
      } else {
        Serial.println("Invalid time format!");      // Out-of-range input
      }
    } else {
      Serial.println("Enter exactly 4 digits (0000 to 2359)."); // Invalid length
    }
  }
}
// Draw one large digit on LCD at specified column
void drawBigDigit(byte digit, int column) {
  lcd.setCursor(column, 0);                    // Top row
  for (int colIndex = 0; colIndex < 3; colIndex++) {
    byte cell = bigNum[digit][0][colIndex];  // Get segment index
    if (cell != 255) lcd.write(cell); else lcd.print(" "); // Draw segment or blank
  }
  lcd.setCursor(column, 1);                    // Bottom row
  for (int colIndex = 0; colIndex < 3; colIndex++) {
    byte cell = bigNum[digit][1][colIndex];  // Get segment index
    if (cell != 255) lcd.write(cell); else lcd.print(" "); // Draw segment or blank
  }
}
// Increment clock time by 1 second
void updateTime() {
  seconds++;                                  // Increment seconds
  if (seconds >= 60) { seconds = 0; minutes++; } // Carry over to minutes
  if (minutes >= 60) { minutes = 0; hours++; }   // Carry over to hours
  if (hours >= 24) { hours = 0; }                // Wrap around 24h
  showColon = !showColon;                         // Toggle colon visibility
}
// Draw full clock (HH:MM:SS) with AM/PM
void drawClock() {
  // --- Calculate hours digits ---
  int h1 = hours / 10;                          // First digit (24h format)
  int h2 = hours % 10;                          // Second digit (24h format)

  int h1PM = (hours > 12 ? (hours - 12) / 10 : h1); // First digit for PM
  int h2PM = (hours > 12 ? (hours - 12) % 10 : h2); // Second digit for PM
  // --- Calculate minutes digits ---
  int m1 = minutes / 10;                        // First digit of minutes
  int m2 = minutes % 10;                        // Second digit of minutes
  // --- Draw hours ---
  if (hours < 12) {                             // AM
    drawBigDigit(h1, 0);                        // First hour digit
    drawBigDigit(h2, 3);                        // Second hour digit
  } else {                                      // PM
    drawBigDigit(h1PM, 0);                      // First PM digit
    drawBigDigit(h2PM, 3);                      // Second PM digit
  }
  // --- Draw blinking colon ---
  lcd.setCursor(6, 0);                          // Top colon
  if (showColon) lcd.write(7); else lcd.print(" ");
  lcd.setCursor(6, 1);                          // Bottom colon
  if (showColon) lcd.write(7); else lcd.print(" ");
  // --- Draw minutes ---
  drawBigDigit(m1, 7);                          // First minute digit
  drawBigDigit(m2, 10);                         // Second minute digit
  // --- Draw AM/PM ---
  lcd.setCursor(14, 0);                          // Top row (AM/PM)
  // --- Draw AM/PM ---
```

```
  lcd.setCursor(13, 0);           // Linha 0, coluna 13
  lcd.print(hours < 12 ? "AM" : "PM"); // Escreve AM ou PM
  // --- Draw seconds ---
  lcd.setCursor(14, 1);                          // Seconds position
  if (seconds < 10) lcd.print("0");             // Leading zero
  lcd.print(seconds);                            // Print seconds
}
```

Code - Result

Speller – Code

```
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include "dictionary.h"
// Represents a node in a hash table
typedef struct node
{
    char word[LENGTH + 1];
    struct node *next;
} node;
int num_words = 0;
// Choose number of buckets in hash table
const unsigned int N = 26;
// Hash table
node *table[N];
// Returns true if word is in dictionary, else false
bool check(const char *word)
{
    int hash_value = hash(word);
    node *cursor = table[hash_value];
    while (cursor != NULL)
    {
        if (strcasecmp(cursor->word, word) == 0)
        {
            return true;
        }
        else
        {
            cursor = cursor->next;
        }
    }
    return false;
}
// Hashes word to a number
unsigned int hash(const char *word)
{
    // Improve this hash function
    return toupper(word[0]) - 'A';
}
// Loads dictionary into memory, returning true if successful, else false
bool load(const char *dictionary)
```

```
{
    for (int i = 0; i < N; i++)
    {
        table[i] = NULL;
    }
    FILE *dict_file = fopen(dictionary, "r");
    if (dict_file == NULL) {
        printf("Was not able to open dictionary\n");
        return false;
    }
    char buffer[45];
    while (fscanf(dict_file, "%s", buffer) != EOF)
    {
        node *new_word = malloc(sizeof(node));
        int hash_value = hash(buffer);
        strcpy(new_word->word, buffer);
        new_word->next = table[hash_value];
        table[hash_value] = new_word;
        num_words++;
    }
    fclose(dict_file);
    return true;
}
// Returns number of words in dictionary id loaded, else 0 if not yet loaded
unsigned int size(void)
{
    return num_words;
}
// Unloads dictionary from memory, returning true if successful, else false
bool unload(void)
{
    for (int i = 0; i < N; i++)
    {
        node *temp = table[i];
        node *cursor = table[i];
        while(cursor != NULL)
        {
            cursor = cursor->next;
            free(temp);
            temp = cursor;
        }
    }
    return true;
}
```

Code - Result

Weather Station - Code

```
//-------------------------------------------------------------------
// Weather Station - DHT11 + LM35 + Water Level + Humidity Drop
// ============================================================
//
// This project simultaneously uses:
// - A DHT11 sensor to measure indoor temperature and humidity
// - An LM35 sensor to measure outdoor temperature
// - A water level sensor
// Data is displayed on an I2C LCD display, switching screens with a button.
//
// Francisco Pimentel
// 27 / Oct / 2025
// version 2.3.6
//-------------------------------------------------------------------
#include <DHT.h>                                  // Include DHT sensor library
#include <LCD_I2C.h>                              // Include I2C LCD library
#define DHT_PIN   8                               // DHT11 sensor pin
#define DHT_TYPE  DHT11                           // DHT sensor type
#define LM35_PIN  A0                              // LM35 analog pin
#define WATER_PIN A1                              // Water level sensor analog pin
#define BUTTON_PIN 7                              // Button pin to switch LCD screens
LCD_I2C lcd(0x27, 16, 2);                         // Create LCD object
DHT dht(DHT_PIN, DHT_TYPE);                       // Create DHT object
int readingLM35;                                  // Variable to store LM35 reading
float humInt, tempInt;                            // Indoor humidity and temperature
float lastTempInt = NULL, lastHumInt = NULL, lastTempExt = NULL, lastWater = NULL; // Last
 displayed values
float tempMax = NULL;
// Custom degree symbol
byte degreeSymbol[8] = { 0b00111, 0b00101, 0b00111, 0b00000, 0b00000, 0b00000, 0b00000, 0b
00000 };
// Custom drop symbol for humidity
byte dropSymbol[8] = { 0b00100, 0b00100, 0b01110, 0b01110, 0b11111, 0b11111, 0b11111, 0b01
110 };
bool showWater = false;                           // Flag to toggle between water/temperature
 menus
bool showMaxTemp = false;                         // Flag to show maximum temperature screen
bool lastButtonState = true;                      // Stores last button state for debounce
unsigned long lastUpdate = 0;                     // Timer for non-blocking updates
const unsigned long interval = 2000;              // Update interval each 2 seconds
void updateLCDValue(int col, int row, float newValue, float &lastValue, bool isTemp);
float waterLevelCM(int adcValue);
void setup() {
  Serial.begin(9600);                                 // Initialize serial communication
  lcd.begin();                                        // Initialize the LCD
  lcd.backlight();                                    // Turn on the LCD backlight
  dht.begin();                                        // Initialize the DHT11 sensor
  pinMode(BUTTON_PIN, INPUT_PULLUP);                  // Set button as input with pullup
  lcd.createChar(0, degreeSymbol);                    // Create custom degree symbol
  lcd.createChar(1, dropSymbol);                      // Create custom drop symbol
  lcd.clear();                                        // Clear LCD screen
  lcd.setCursor(0, 0);                                // Set cursor to line 0
  lcd.print("Getting");                               // Display first part of startup messa
ge
  lcd.setCursor(1, 0);                                // Set cursor to line 1
```

```
  lcd.print("the data...");                          // Display second part of startup mess
age
  delay(1500);                                       // Wait 1.5 seconds
  lcd.clear();                                       // Clear LCD after startup message
  // Initialize labels for temperature menu
  lcd.setCursor(0, 0);
  lcd.print("In:");
  lcd.setCursor(0, 1);
  lcd.print("Out:");
}
void loop() {
  // Read button state for menu toggle
  bool buttonState = digitalRead(BUTTON_PIN);
  if (lastButtonState == HIGH && buttonState == LOW) {
    // Cycle through 3 screens: Initial >> Max Temp >> Water >> Initial
    if (!showMaxTemp && !showWater) {
      showMaxTemp = true;
      showWater = false;
    } else if (showMaxTemp) {
      showMaxTemp = false;
      showWater = true;
    } else if (showWater) {

      showWater = false;
      showMaxTemp = false;
      lastTempInt = NULL;                            // Reset last values for initial scree
n
      lastHumInt = NULL;
    }
    lcd.clear();                                     // Clear LCD for new menu
    delay(50);                                       // Simple debounce
  }
  lastButtonState = buttonState;                     // Save current button state
  unsigned long currentMillis = millis();            // Get current time
  if (currentMillis - lastUpdate >= interval) {
    lastUpdate = currentMillis;                      // Update timer
    if (showWater) {
      // Water level menu
      int rawWater = analogRead(WATER_PIN);              // Read water sensor
      float waterCM = waterLevelCM(rawWater);            // Convert ADC to cm
      lcd.setCursor(0, 0);                               // Set cursor to line 0
      if (rawWater <= 0) {                               // Don't display value if no water
 contact
        lcd.print("   ");                                // Blank
      } else {
        updateLCDValue(0, 0, waterCM, lastWater, false);  // Display water value
      }
      lcd.setCursor(3, 0);                               // Set cursor for unit
      lcd.print("cm");                                   // Print unit
      lcd.setCursor(0, 1);                               // Set cursor for label
      lcd.print("Water Level");                          // Print label
    } else if (showMaxTemp) {
      // Max temperature screen
      tempInt = dht.readTemperature();                   // Read current temperature
      if (tempInt) {
        if (!tempMax || tempInt > tempMax) {
          tempMax = tempInt;                             // Update maximum if higher
        }
      }
```

```
      lcd.setCursor(0, 0);
      lcd.print("Temp Max:");
      if (tempMax) {
        lcd.setCursor(10, 0);
        lcd.print(tempMax, 1);
        lcd.write(byte(0));
        lcd.print("C");
      } else {
        lcd.setCursor(10, 0);
        lcd.print("     ");                              // Blank if no max temperature yet
      }
    } else {
      // Temperature / Humidity menu
      humInt = dht.readHumidity();
      tempInt = dht.readTemperature();
      readingLM35 = analogRead(LM35_PIN);
      float tempExt = (readingLM35 * 5.0 * 100.0) / 1023.0;
      // Print to serial monitor
      Serial.print("Interno: ");
      Serial.print(tempInt);
      Serial.print(" °C  ");
      Serial.print(humInt);
      Serial.println(" %");
      Serial.print("Externo: ");
      Serial.print(tempExt);
      Serial.println(" °C");
      Serial.println("---");
      // Line 0: In: tempInt H: humInt [drop at column 15]
      lcd.setCursor(0,0);
      lcd.print("In:");   // Columns 0-2
      // Display indoor temperature
      if (tempInt) {
          lcd.setCursor(3, 0);                    // Column 3
          lcd.print(tempInt, 1);                  // Example: 23.5
          lcd.write(byte(0));                     // Degree symbol
          lcd.print("C");                         // Columns 3-8
      } else {
          lcd.setCursor(3,0);
          lcd.print("      ");                    // Clear if no reading
      }
      // Display indoor humidity
      if (humInt) {
          lcd.setCursor(10,0);                    // Column 9
          lcd.print("H:");
          lcd.print((int)humInt);
      } else {
          lcd.setCursor(9,0);
          lcd.print("   ");                       // Clear if no reading
      }
      // Display drop symbol at column 15
      lcd.setCursor(15,0);                        // Column 15
      lcd.write(byte(1));
      // Display outdoor temperature
      updateLCDValue(5, 1, tempExt, lastTempExt, true);
    }
  }
}
// Update LCD value smoothly without blinking
void updateLCDValue (int col, int row, float newValue, float &lastValue, bool isTemp) {
```

```
  if (!lastValue || abs(newValue - lastValue) >= 0.1) {

    lcd.setCursor(col, row);
    lcd.print("     ");                                    // Clear previous value
    lcd.setCursor(col, row);
    lcd.print(newValue, 1);                                // Print new value
    if (isTemp) {
      lcd.write(byte(0));                                  // Degree symbol
      lcd.print("C");
    }

    lastValue = newValue;                                  // Update last value
  }
}
// Convert ADC reading to water level in cm
float waterLevelCM (int adcValue) {
  if (adcValue >= 450 && adcValue < 500) {
    return 1;
  }

  if (adcValue >= 500 && adcValue < 520) {
    return 2;
  }

  if (adcValue >= 520 && adcValue < 530) {
    return 2.5;
  }

  if (adcValue >= 530 && adcValue < 540) {
    return 3;
  }
  if (adcValue >= 540 && adcValue < 550) {
    return 3.5;
  }

  if (adcValue >= 550) {
    return 4;
  }

  return 0;
}
```

Code - Result

Bibliography

Ibrahim, D. (2023). Mastering the Arduino Uno R4: Programming and projects for the Minima and WiFi (1ªed.). Elektor

Check, check, triple check

P1. Computer Science

Name	Student nr.	Group	Year
Francisco Porte Pimentel	5708915	A	2025/2026

The student is responsible for safe keeping of this card

Assignments

Wk	Subject	Rating	Date	Signature
1.1	Scratch	(V) O	12 SEP 2025	
1.2	Mario less **and/or** Mario more	(U) (G) V O	12 SEP 2025	
	Cash **and/or** Credit	U (G) V O	12 SEP 2025	
1.3	Scrabble	(V) O	19 SEP 2025	
	Readability	(V) O	19 SEP 2025	
	Ceasar **and/or** Substitution	U (G) V O	22 SEP 2025	
1.4	Sort	(V) O	06 OCT 2025	
	Plurality	(V) O	06 OCT 2025	
	Runoff **and/or** Tideman	(U) G V O	06 OCT 2025	
1.5	Scavenger hunt	U (G) V O	06 OCT 2025	
	Filter less **and/or** Filter more	U (G) V O	10 OCT 2025	
	Recover	(V) O	10 OCT 2025	
1.6	Git Together	(V) O	27 OCT 2025	
	Inheritance	(V) O	10 OCT 2025	
1.7	Speller	(V) O	27 OCT 2025	
	Check, check, triple check	(V) O	29/oct/25	Francisco
1.8	Completing Portfolio			
T1	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

This card is proof that the above assignments are (partially) completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week, a week before, or a week after
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment

NHL STENDEN university of applied sciences

P1. Embedded Systems

Name	Student nr.	Group	Year
Francisco Porte Pimentel	5708915	A	2025/2026

The student is responsible for safe keeping of this card

Assignments

Wk	Ch	Subject	Rating	Date	Signature
1.1	1	CuteBot Race game	(V) O	5 SEP 2025	
1.2	2	Mario Uno *(example 1 to 18)*	(U) G (V) O	12 SEP 2025	M. Bara
1.3	3	Morse code generator *(project 1 to 13)*	U G (V) O	23 SEP 2025	M. Bara
1.4	3	(Advanced) Traffic lights *(proj. 14 to 20)*	U (G) V O	06 OCT 2025	Bara
1.5	4	Displaying Uptime *(project 1 to 6)*	(U) G V O	9 OCT 2025	M. Bara
1.6	5	(Realtime) Jumbo Clock *(project 1 to 6)*	U (G) (V) O	14 OCT 2025	M. Bara
1.7	6	Weather Station *(project 1 to 12)*	U (G) V O	28 OCT 2025	M. Bara
1.8	-	Completing Portfolio			
T1	-	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

Feedback:

use comments
1.5 beeper + speed
1.6 + refresh display
1.2 + buzzer & animation of matrix display
1.7 + waterlevel

Voldaan
28 OCT 2025

This card is proof that the above assignments are (partially) completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week, a week before, or a week after
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment